00

DSA · FOUNDATIONS

Foundations

> Complexity, recursion, మరియు ప్రతి data structure లోపల ఏం జరుగుతోంది – ఇవి తెలియకుండా ఏ pattern నీ నిజంగా అర్థం చేసుకోలేం.

ప్రతి problem కి: **ఏ pattern ఇది** → ఎందుకు ఆ pattern → dry run → optimal JavaScript code → complexity → edge cases. `DSA_Patterns_Telugu.pdf` pattern-first దృష్టి; ఈ file ఆ patterns ని నిజమైన LeetCode problems మీద అమలు చేస్తుంది.

తెలుగు + ENGLISH

YASWANTH · REFERENCE

> ఈ document చదివిన తర్వాత DSA fundamentals మళ్ళీ జీవితంలో మర్చిపోలేవు. DSA అస్సలు తెలియని వ్యక్తిని – ZERO నుండి – Senior Software Engineer (SSE) interview crack చేసే స్థాయికి తీసుకెళ్లడమే లక్ష్యం. ప్రతి concept కి ఒక vivid real-life analogy, ఎందుకు/ఎప్పుడు వాడాలి, internal working, Big-O, gotchas, మరియు runnable JavaScript code – అన్నీ ఉంటాయి. "ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు."
>
> **ఇది foundations doc.** ఇక్కడ complexity, recursion, sorting/searching, patterns, మరియు అన్ని core data structures ని *మొదటి నుండి* నేర్చుకుంటావు. తర్వాత actual LeetCode problems ని pattern-wise drill చేయడానికి companion docs వాడు: `DSA_01_Arrays_Strings_Telugu.md`, `DSA_02_TwoPointers_SlidingWindow_Telugu.md`, `DSA_03_HashMap_Telugu.md`, `DSA_04_Stack_Queue_Telugu.md`, `DSA_05_LinkedList_Telugu.md`, `DSA_06_Trees_Telugu.md`, `DSA_07_Graphs_Telugu.md`, `DSA_08_Heap_Greedy_Telugu.md`, `DSA_09_Backtracking_Telugu.md`, `DSA_10_DynamicProgramming_Telugu.md` – ఇవి **LeetCode Top-Interview-150** ని pattern-by-pattern drill చేస్తాయి.
>
> **లక్ష్యం:** Programming (JavaScript) తెలిసిన కానీ DSA అస్సలు తెలియని engineer ని absolute basics నుండి interview-ready వరకు తీసుకెళ్లడం. Best-teacher style – intuition first, formalism తర్వాత.

విషయ సూచిక (Table of Contents)

Part 1 – Mindset & Complexity (ఆలోచనా విధానం మరియు complexity)

1. DSA అంటే ఏమిటి, SSE interview కి ఎందుకు (data structure vs algorithm, ఎలా approach చేయాలి)
2. Problem-solving Framework (UMPIRE) – ఏ problem కైనా step-by-step ఆలోచనా విధానం
3. Time Complexity & Big-O – deep (ఎందుకు, ఎలా లెక్కించాలి, అన్ని classes, rules)
4. Space Complexity + Amortized Analysis (call stack space, in-place, amortized)
5. Common Complexity Cheat-sheet (data structures + sorting + input-size → complexity)

Part 2 – Recursion & Core Techniques

6. Recursion from scratch (base case, call stack, recursion tree, recursion vs iteration)
7. Sorting essentials (bubble/selection/insertion, merge sort, quick sort, counting sort, JS sort gotcha, stability)
8. Searching essentials (linear vs binary search, binary search template + variants)
9. The Pattern Catalog (two pointers, sliding window, prefix sum, BFS/DFS, backtracking, DP, greedy, D&C, heap/top-K, union-find + recognition signals)

Part 3 – Data Structures FROM SCRATCH

10. Arrays & Dynamic Arrays (memory layout, index O(1), insert/delete, JS arrays reality)
11. Strings (immutability, char operations, JS string methods for DSA)
12. Hash Map & Hash Set (hashing, collisions, O(1) avg, JS Map/Set/object)
13. Linked List (singly/doubly, node/pointer, vs array, common ops: reverse/traverse)
14. Stack (LIFO, JS via array, uses: matching/undo/DFS)
15. Queue & Deque (FIFO, circular, uses: BFS/scheduling)
16. Trees & Binary Trees & BST (terminology, traversals, BST property + operations)
17. Heap / Priority Queue (min/max heap, array representation, heapify, JS implementation)
18. Graph (adjacency list vs matrix, BFS + DFS templates, when graphs appear)
19. Trie (prefix tree, insert/search, uses: autocomplete/word search)
20. Union-Find / Disjoint Set (union by rank + path compression, uses: connectivity/cycles)

Part 4 – Interview Craft

21. Interview లో ఎలా communicate చేయాలి (clarify → code → test → complexity, decision guide, revision strategy)

Part 1 – Mindset & Complexity

> DSA నేర్చుకోవడం అంటే కొన్ని syntax తిరగేయడం కాదు – అది ఒక **కొత్త ఆలోచనా విధానం** (way of thinking). ఈ Part లో DSA అంటే ఏమిటి, ఏ problem కైనా ఎలా approach చేయాలి (UMPIRE framework), మరియు ప్రతి solution ని ఎలా measure చేయాలి (time & space complexity) – ఇవి నేర్చుకుంటాం. ఇవి పునాది; ఇవి పక్కాగా ఉంటే మిగతా అంతా సులభం.

---

1. DSA అంటే ఏమిటి, SSE interview కి ఎందుకు

వివరణ

DSA = Data Structures + Algorithms. రెండు వేర్వేరు విషయాలు, కానీ కలిసి పనిచేస్తాయి:

- **Data Structure (DS)** = data ని **memory లో ఎలా organize/store** చేయాలి. ఉదా: array, hash map, linked list, tree, graph, heap.
- **Algorithm** = ఆ data మీద ఒక problem solve చేయడానికి **step-by-step విధానం** (recipe). ఉదా: binary search, merge sort, BFS, dynamic programming.

ఒక్క వాక్యంలో: **Data structure = ఏ డబ్బాల్లో పెట్టాలి; Algorithm = ఆ డబ్బాతో ఏం చేయాలి.**

రెండూ ఎందుకు కలిసి నేర్చుకోవాలి? ఎందుకంటే **సరైన data structure ఎంచుకుంటే algorithm చాలా వేగం అవుతుంది.** ఉదాహరణకి "ఈ number list లో ఉందా?" అని వెతకాలంటే:

- Array లో ఉంటే → ప్రతి element check చేయాలి → O(n) (slow).
- Hash set లో ఉంటే → నేరుగా O(1) (instant).

అంటే **అదే problem, వేరే data structure** → **1000x వేగం.** ఇదే DSA యొక్క అసలు శక్తి.

Real-life Scenario

> **DSA = వంటగది (kitchen) organization.** ఊహించు – నీ దగ్గర 500 మసాలా డబ్బాలు ఉన్నాయి.
>
> - **Data structure** = ఆ డబ్బాలని ఎలా అమర్చావు. అన్నీ ఒక పెద్ద బస్తాలో గుమ్మరించావా (array – వెతకడానికి అన్నీ తిరగేయాలి)? లేక alphabet ప్రకారం labeled shelf లో పెట్టావా (hash map – నేరుగా "H" shelf కి వెళ్ళి haldi తీయవచ్చు)?
> - **Algorithm** = ఒక కూర వండే recipe – ఏ మసాలా, ఎంత, ఏ order లో వేయాలి.
>
> మంచి organization (data structure) ఉంటే, వంట (algorithm) వేగంగా, తక్కువ శ్రమతో అవుతుంది. చెత్త organization ఉంటే, సరైన recipe ఉన్నా, ప్రతిసారి డబ్బా వెతకడంలోనే గంట పోతుంది. **SSE interview అడిగేది సరిగ్గా ఇదే – "ఈ problem కి ఏ organization (DS) + ఏ recipe (algorithm) optimal?"**

SSE interview కి ఎందుకు ఇంత ముఖ్యం?

Senior/SSE level లో companies నీ దగ్గర ఇవి చూస్తాయి:

ఏం చూస్తారు	ఎందుకు
Optimal solution	Senior గా నువ్వు రాసే code millions of users ని handle చేస్తుంది. $O(n^2)$ vs O(n) తేడా production లో crash vs smooth.
Trade-off ఆలోచన	"Time తగ్గించడానికి extra memory వాడొచ్చా?" – ఇది real engineering decision. Interview దీన్ని test చేస్తుంది.
Communication	Senior అంటే నీ ఆలోచనని team కి explain చేయగలగాలి. Interview లో నువ్వు ఎలా *think aloud* చేస్తావో చూస్తారు.
Fundamentals depth	Framework లు మారతాయి, కానీ hash map, tree, recursion ఎప్పటికీ మారవు. ఇవి నీ foundation strong అని చూపిస్తాయి.
Edge cases	Empty input, single element, duplicates, overflow – senior వీటిని miss అవ్వకూడదు.

అంటే DSA interview అనేది "నీకు trick తెలుసా?" test కాదు – **"నువ్వు ఒక problem ని structured గా, optimal గా, communicate చేస్తూ solve చేయగలవా?"** అనే test. ఇదే exact skill నువ్వు job లో రోజూ వాడతావు.

ఎలా approach చేయాలి (mindset)

మొదటి రోజు నుండే ఈ 4 అలవాట్లు పెంచుకో – ఇవి నిన్ను "rote-learner" నుండి "problem-solver" గా మారుస్తాయి:

1. **Intuition first, code last.** Problem చదవగానే code రాయకు. ముందు "manually ఎలా solve చేస్తాను?" అని కాగితం మీద ఆలోచించు. Brain లో clear అయ్యాకే code.
2. **Pattern recognition.** ప్రతి problem కొత్తది కాదు. చాలావరకు 10-15 patterns లో ఏదో ఒకటే (two pointers, sliding window, BFS...). Pattern గుర్తిస్తే, solution 80% అయ్యింది.
3. **Brute force → optimize.** ముందు ఏదో ఒక పని చేసే (working) solution రాయి, అది slow అయినా. తర్వాత "ఎక్కడ time waste అవుతోంది?" అని optimize చేయి. Interview లో కూడా ఇదే approach మెచ్చుతారు.
4. **Complexity ఎప్పుడూ మనసులో.** ప్రతి solution కి "ఇది ఎంత fast? ఎంత memory?" అని అడుక్కో. ఇది అలవాటైతే, నువ్వు automatic గా better solutions వైపు వెళతావు.

Code – అదే problem, రెండు data structures, భారీ తేడా

```
// Problem: ఒక numbers array లో target ఉందా లేదా చెప్పు.

// ---- విధానం 1: Array లో linear search → O(n) ----
function existsInArray(arr, target) {
  for (let i = 0; i < arr.length; i++) {   // ప్రతి element ని చూడాలి
    if (arr[i] === target) return true;    // worst case: n comparisons
  }
  return false;
}

// ---- విధానం 2: Set (hash-based) లో lookup → O(1) average ----
function existsInSet(set, target) {
  return set.has(target);                  // ఒక్క step — నేరుగా hash lookup
}

const nums = [5, 3, 9, 1, 7, 2, 8];
console.log(existsInArray(nums, 8));       // true  (7 comparisons పట్టింది)

const numSet = new Set(nums);              // ఒకసారి build → O(n)
console.log(existsInSet(numSet, 8));       // true  (1 step మాత్రమే!)

// అదే result. కానీ 10 lakh elements ఉంటే:
// array = 10 lakh comparisons; set = 1 step. ఇదే DSA యొక్క power.
```

Data Structure vs Algorithm – గుర్తుంచుకోవడానికి

అంశం	Data Structure	Algorithm
అర్థం	Data ని organize/store చేసే విధానం	Problem solve చేసే step-by-step విధానం
ప్రశ్న	"ఎక్కడ, ఎలా పెట్టాలి?"	"ఏం చేయాలి, ఏ order లో?"
ఉదాహరణలు	array, hash map, tree, heap, graph	binary search, merge sort, BFS, DP
analogy	డబ్బా / అల్మారా	వంట recipe
measure	operations కి Big-O (search, insert...)	overall time & space complexity
మారుతుందా?	Data shape ని బట్టి ఎంచుకుంటాం	DS + problem ని బట్టి design చేస్తాం

Key Points

- **DSA = Data Structures (ఎలా store) + Algorithms (ఏం చేయాలి).** రెండూ కలిసి పనిచేస్తాయి.
- **సరైన DS ఎంచుకుంటే algorithm dramatically వేగం** అవుతుంది (array O(n) vs set O(1)).
- SSE interview test చేసేది trick కాదు – **optimal solution + trade-off ఆలోచన + communication + fundamentals.**
- Mindset: **intuition first** → **pattern recognition** → **brute force then optimize** → **complexity ఎప్పుడూ మనసులో.**
- Framework లు మారతాయి; **fundamentals (hash map, tree, recursion) ఎప్పటికీ మారవు** – అందుకే ఇవి worth mastering.

Interview దృష్టి

Q: DSA ఎందుకు నేర్చుకోవాలి, real job లో framework వాడతాం కదా? A: Frameworks abstractions ని ఇస్తాయి, కానీ లోపల అవన్నీ DSA మీదే నడుస్తాయి. Database index = B-tree; React reconciliation = tree diff; rate limiter = queue/hash map. Bug debug చేయాలన్నా, system scale చేయాలన్నా fundamentals తప్పనిసరి. Interview ఇదే signal కోసం చూస్తుంది.

Q: మొదటి రోజు నుండి ఎలా practice చేయాలి? A: ప్రతి problem కి UMPIRE (తర్వాతి topic) follow చేయి, brute force ముందు రాయి, తర్వాత complexity చూసి optimize చేయి, పరిష్కారాన్ని ఒక pattern కి tag చేయి. Random గా 500 problems కంటే, pattern-wise 150 problems (Top-Interview-150) చాలా better.

---

2. Problem-solving Framework (UMPIRE)

వివరణ

Interview లో problem చూడగానే panic అయ్యి code రాయడం మొదలుపెడితే – 90% సార్లు తప్పు దారిలో వెళతావు. దీనికి పరిష్కారం: ప్రతి problem కి **అదే fixed steps** follow చేయడం. అత్యంత popular framework = **UMPIRE**:

అక్షరం	Step	ఏం చేయాలి
U	**Understand**	Problem ని పూర్తిగా అర్థం చేసుకో. Clarifying questions అడుగు. Inputs, outputs, constraints, edge cases నోట్ చేసుకో.
M	**Match**	ఇది ఏ known pattern/category కి match అవుతుంది? (sorting? two pointers? graph? DP?)
P	**Plan**	Solution ని plain English/Telugu లో step-by-step గా చెప్పు (pseudocode). Code కాదు – logic.
I	**Implement**	ఇప్పుడే code రాయి. Plan ని నమ్మకంగా translate చేయి.
R	**Review**	Code ని ఒక example తో line-by-line "dry run" చేయి. Edge cases test చేయి. Bugs వెతుకు.

అక్షరం	Step	ఏం చేయాలి
E	**Evaluate**	Time & space complexity చెప్పు. Better అవ్వగలదా? Trade-offs discuss చేయి.

Cricket లో **umpire** ప్రతి ball కి అదే process follow చేస్తాడు – hasty కాదు, systematic. నువ్వూ ప్రతి problem కి ఇలా.

Real-life Scenario

> **UMPIRE = డాక్టర్ diagnosis process.** మంచి డాక్టర్ నువ్వు లోపలికి రాగానే surgery చేయడు:
>
> 1. **Understand** – "ఎక్కడ నొప్పి? ఎప్పటి నుండి? జ్వరం ఉందా?" (clarifying questions – symptoms, history).
> 2. **Match** – "ఈ symptoms ఏ రకమైన జబ్బుకి సరిపోతాయి?" (pattern matching – తనకు తెలిసిన diseases తో compare).
> 3. **Plan** – "మొదట blood test, తర్వాత X-ray, తర్వాత ఇలా treat చేద్దాం." (treatment plan).
> 4. **Implement** – actual treatment/prescription.
> 5. **Review** – "మందు పనిచేస్తోందా? side effects ఉన్నాయా?" (follow-up).
> 6. **Evaluate** – "ఇంకా better/తక్కువ ఖర్చు treatment ఉందా?" (optimize).
>
> **నేరుగా surgery (code) చేసే డాక్టర్ ప్రమాదకరం.** Interview లో నేరుగా code రాసే candidate కూడా అంతే. Interviewer నీ *process* ని చూస్తాడు, కేవలం final code ని కాదు.

ప్రతి step ని లోతుగా

U – Understand (అత్యంత ముఖ్యమైన step):

- Input format ఏమిటి? (sorted array? నెగటివ్‌లు ఉన్నాయా? duplicates?)
- Output ఏం కావాలి? (index? value? boolean? అన్ని solutions?)
- Constraints – n ఎంత పెద్దది? ($n \le 100$ అయితే $O(n^2)$ ఓకే; $n \le 10^6$ అయితే $O(n \log n)$ కావాలి – constraint నీకు expected complexity చెప్తుంది!)
- Edge cases – empty input? single element? అన్నీ same? overflow?
- Ambiguity clear చేసుకో – "input లో ఒకటే valid answer ఉంటుందా, లేక multiple?"

M – Match: గుర్తుపట్టడానికి signals (Topic 9 లో పూర్తి catalog): "sorted array" → binary search / two pointers; "subarray/substring" → sliding window; "top K" → heap; "all combinations" → backtracking; "shortest path" → BFS; "overlapping subproblems" → DP.

P – Plan: Pseudocode రాయి. ఉదా: "hash map లో ప్రతి number ని దాని index తో పెట్టు; ప్రతి number కి `target - number` map లో ఉందా చూడు." ఇది code కాదు, కానీ logic పూర్తిగా clear.

I – Implement: ఇప్పుడు plan ని JS లోకి translate చేయి. Plan strong అయితే ఇది mechanical.

R – Review: ఒక చిన్న example (ఉదా [2,7,11], target 9) తీసుకుని, code ని కాగితం మీద execute చేయి. ప్రతి variable value track చేయి. Off-by-one, edge cases పట్టుకో.

E – Evaluate: "ఇది O(n) time, O(n) space. Space తగ్గించాలంటే sort చేసి two-pointer వాడొచ్చు కానీ అప్పుడు O(n log n) time." – ఇలా trade-off చెప్పడం senior signal.

Code – UMPIRE ని "Two Sum" మీద apply

```
// PROBLEM: numbers array + target. మొత్తం target అయ్యే రెండు indices return చేయి.
// ఖచ్చితంగా ఒకే answer ఉంటుంది; అదే element రెండుసార్లు వాడకూడదు.

// U — Understand:
//   input: nums = [2,7,11,15], target = 9
//   output: [0,1]  (ఎందుకంటే nums[0]+nums[1] = 2+7 = 9)
//   constraint: ఖచ్చితంగా ఒక solution; negatives రావచ్చు.
//   edge: length ≥ 2 అని assume.

// M — Match:
//   "ఒక pair కావాలి, sum = target" → complement lookup → HASH MAP pattern.

// P — Plan:
//   1. ఖాళీ hash map (value → index).
//   2. ప్రతి number x కి: need = target − x.
//   3. need ఇప్పటికే map లో ఉంటే → [map[need], currentIndex] return.
//   4. లేకపోతే x ని map లో పెట్టి కొనసాగు.

// I — Implement:
function twoSum(nums, target) {
  const seen = new Map();              // value → index
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];     // ఈ number కి ఏ jodi కావాలి?
    if (seen.has(need)) {              // ఆ jodi ఇప్పటికే చూశామా?
      return [seen.get(need), i];      // అవును → రెండు indices
    }
    seen.set(nums[i], i);              // లేదు → దీన్ని గుర్తుపెట్టుకో
  }
  return [];                           // (constraint ప్రకారం ఇక్కడికి రాదు)
}

// R — Review (dry run [2,7,11,15], target 9):
//   i=0: x=2, need=7, map ఖాళీ → 7 లేదు → map={2:0}
//   i=1: x=7, need=2, map లో 2 ఉంది (index 0) → return [0,1]  ✓
console.log(twoSum([2, 7, 11, 15], 9));   // [0, 1]
console.log(twoSum([3, 2, 4], 6));        // [1, 2]
console.log(twoSum([3, 3], 6));           // [0, 1]  (duplicate values ఓకే)

// E — Evaluate:
//   Time: O(n) — ప్రతి element ఒకసారే చూస్తాం.
//   Space: O(n) — worst case n entries map లో.
//   Brute force (nested loop) O(n²) కంటే ఇది చాలా better.
//   Trade-off: extra O(n) space వాడి time ని O(n²)→O(n) కి తగ్గించాం.
```

UMPIRE vs "నేరుగా code" – తేడా

అంశం	నేరుగా code రాయడం	UMPIRE follow చేయడం
Direction	తరచూ తప్పు దారి, మధ్యలో reset	ముందే సరైన దారి fix

అంశం	నేరుగా code రాయడం	UMPIRE follow చేయడం
Edge cases	చివర్లో miss అవుతాయి	Understand step లోనే capture
Communication	Interviewer కి నీ ఆలోచన కనపడదు	ప్రతి step think-aloud → strong signal
Bugs	Random గా వస్తాయి, debug కష్టం	Review step లో దొరుకుతాయి
Optimization	తట్టదు లేదా మర్చిపోతావు	Evaluate step లో guaranteed discussion
Stress	Panic ఎక్కువ	Process ఉంది కాబట్టి calm

Key Points

- **UMPIRE = Understand, Match, Plan, Implement, Review, Evaluate.** ప్రతి problem కి ఇదే fixed process.
- **నేరుగా code రాయకు** – ముందు Understand + Plan. ఇది time save చేస్తుంది, waste కాదు.
- **Constraint (n range) నీకు expected complexity చెప్తుంది** – $n \le 10^6$ అంటే O(n log n) target.
- **Review = dry run** ఒక చిన్న example తో line-by-line. Bugs ఇక్కడ దొరుకుతాయి.
- **Evaluate లో complexity + trade-off** చెప్పడం SSE level signal.
- Interviewer final code కంటే నీ **process + communication** ని ఎక్కువ చూస్తాడు.

Interview దృష్టి

Q: Problem అర్థమైంది అనుకున్నా, ఎక్కడ మొదలుపెట్టాలో తట్టడం లేదు – ఏం చేయాలి? A: Match step లో ఆగిపో. Input shape చూడు: sorted? → binary search/two pointers. Subarray? → sliding window. Tree/grid? → DFS/BFS. ఏదీ match కాకపోతే brute force రాసి, "ఎక్కడ repeated work జరుగుతోంది?" అని optimize చేయి. Interviewer కి "ఇలా ఆలోచిస్తున్నా" అని చెప్తూ ఉండు – silence కంటే wrong-but-vocal better.

Q: Time అయిపోతోంది, ఇంకా brute force దగ్గరే ఉన్నా – ఏం చేయాలి? A: Working brute force + "ఇది $O(n^2)$, దీన్ని hash map తో O(n) కి తీసుకురావచ్చు" అని optimization plan చెప్పడం, కేవలం half-written optimal solution కంటే చాలా better. పని చేసే code + clear optimization path = strong.

---

3. Time Complexity & Big-O (deep)

వివరణ

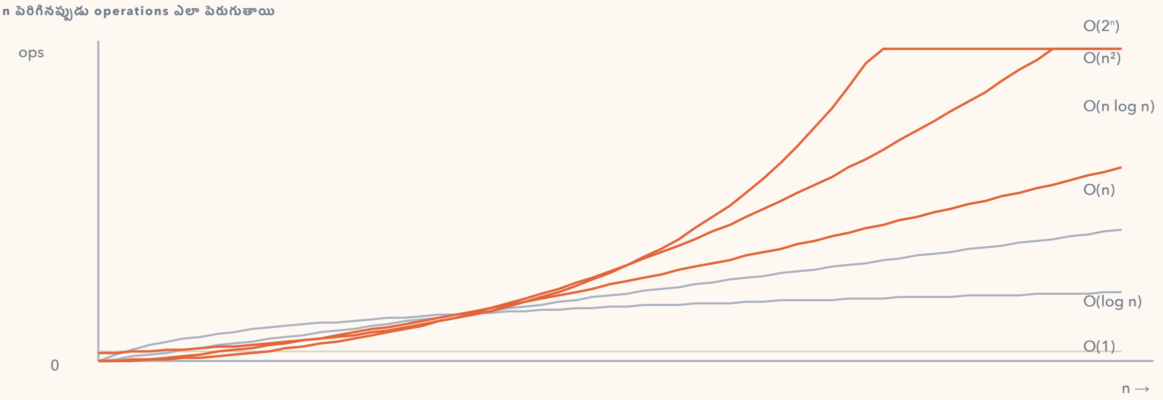
Time complexity = "input size పెరిగితే, ఈ algorithm ఎన్ని *steps* చేస్తుంది?" అని describe చేసే విధానం. దీన్ని **Big-O notation** తో రాస్తాం: `O(...)`.

ముఖ్యమైన విషయం – **Big-O అంటే seconds కాదు.** ఎందుకంటే seconds hardware ని బట్టి మారతాయి (నీ laptop vs server). బదులుగా Big-O measure చేసేది: **input `n` పెరిగే కొద్దీ steps ఎంత వేగంగా పెరుగుతాయి (growth rate).** ఇది hardware-independent, అందుకే universal language.

ఉదాహరణ: ఒక loop `n` సార్లు తిరిగితే → n steps → **O(n)**. Nested loop (loop లో loop) → n×n → **$O(n^2)$**. `n` రెట్టింపు అయితే O(n) steps రెట్టింపు అవుతాయి, కానీ $O(n^2)$ steps నాలుగు రెట్లు అవుతాయి. ఇదే "growth rate."

BIG-O · వేగం ఎలా వస్తుంది

n పెరిగినప్పుడు operations ఎలా పెరుగుతాయి



n = 10^5 దగ్గర ఏమవుతుంది – ఈ సంఖ్యలు గుర్తుంచుకోండి

$O(n) = 10^5$ · $O(n \log n) = 1.7 \times 10^6$ · $O(n^2) = 10^{10}$ ← ఇది timeout

సెకనుకి సుమారు 10^8 operations అనుకోవాలి.

అందుకే constraint చూడగానే – 10^5 అంటే $O(n \log n)$ కావాలి, 10^3 అయితే $O(n^2)$ సరిపోతుంది.

Constraint → algorithm map: $n \le 10$ అయితే O(n!) కూడా సరే · $n \le 20 \to O(2^n)$ · $n \le 10^3 \to O(n^2)$ · $n \le 10^5 \to O(n \log n)$ · $n \le 10^8 \to O(n)$ లేదా O(log n). ఇది interview lo constraint చూసి algorithm ని వెనక్కి ఊహించే పద్ధతి.

Real-life Scenario

> **Big-O = "ఎంత మంది guests వస్తే వంట ఎంత కష్టం అవుతుంది?"** అనే prediction.
>
> - **O(1)** – ఎంత మంది వచ్చినా ఒకే switch నొక్కి light వేయడం. Guests 10 అయినా 1000 అయినా same effort.
> - **O(n)** – ప్రతి guest కి ఒక plate పెట్టడం. 10 guests = 10 plates; 100 guests = 100 plates. Linear గా పెరుగుతుంది.
> - **$O(n^2)$** – ప్రతి guest ప్రతి ఇతర guest తో కరచాలనం (handshake). 10 guests = 45 handshakes; 100 guests = ~5000. Guests 10x అయితే handshakes ~100x!
> - **O(log n)** – ఫోన్ డైరెక్టరీలో పేరు వెతకడం. ప్రతిసారి సగం pages తీసేస్తావు. 1000 pages → కేవలం ~10 steps.
>
> **నువ్వు 10 guests కి వండేటప్పుడు O(n) vs $O(n^2)$ తేడా కనపడదు. కానీ 1 million guests? $O(n^2)$ algorithm పండగకి పూర్తవదు.** అందుకే Big-O ముఖ్యం – పెద్ద scale లో ఏం జరుగుతుందో ముందే చెప్తుంది.

ఎలా లెక్కించాలి – 3 simple rules

Rule 1: Constants ని drop చేయి. O(2n) → O(n). O(500) → O(1). ఎందుకంటే $n \to \infty$ దగ్గర constant అర్థరహితం; growth rate మాత్రమే ముఖ్యం. రెండు loops వరుసగా (`for...; for...`) = O(n) + O(n) = O(2n) = **O(n)**.

Rule 2: Non-dominant terms ని drop చేయి. $O(n^2 + n) \to O(n^2)$. ఎందుకంటే n పెద్దగా అయినప్పుడు n^2 ముందు n చాలా చిన్నది ($n=1000 \to n^2=10^6$, $n=10^3$ – negligible). ఎప్పుడూ **అత్యంత వేగంగా పెరిగే**

term మాత్రమే ఉంచు.

Rule 3: Different inputs = different variables. రెండు వేర్వేరు arrays (size a, b) మీద loops అయితే O(a+b), O(a×b) – **O(n²) కాదు.** ఇది interview లో చాలా మంది తప్పు చేస్తారు.

Nested loops = multiply; sequential loops = add.

అన్ని ముఖ్యమైన complexity classes (వేగం క్రమంలో)

Big-O	పేరు	ఉదాహరణ	n=1000 దగ్గర ~steps
O(1)	Constant	array index, hash lookup, push/pop	1
O(log n)	Logarithmic	binary search, balanced BST	~10
O(n)	Linear	array traverse, linear search	1,000
O(n log n)	Linearithmic	merge sort, quick sort, heap sort	~10,000
O(n²)	Quadratic	nested loops, bubble sort	1,000,000
O(n³)	Cubic	triple nested loops	1,000,000,000
O(2ⁿ)	Exponential	subsets, naive fibonacci	astronomical (10^{300})
O(n!)	Factorial	permutations, brute-force TSP	ఊహకందదు

గుర్తుంచుకో (fast → slow): `O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)`

Code – ప్రతి class కి ఒక ఉదాహరణ

```
// O(1) – Constant: input ఎంత పెద్దదైనా ఒకే step
function first(arr) {
  return arr[0];                         // index access – ఎప్పుడూ 1 step
}

// O(n) – Linear: ప్రతి element ఒకసారి
function sum(arr) {
  let total = 0;
  for (const x of arr) total += x;       // n iterations
  return total;
}

// O(n²) – Quadratic: ప్రతి pair
function allPairs(arr) {
  const pairs = [];
  for (let i = 0; i < arr.length; i++) {       // n సార్లు
    for (let j = i + 1; j < arr.length; j++) { // లోపల ~n సార్లు
      pairs.push([arr[i], arr[j]]);            // మొత్తం ~n²/2 → O(n²)
    }
  }
  return pairs;
}

// O(log n) – Logarithmic: ప్రతిసారి సగం తగ్గుతుంది
```

```
function countHalvings(n) {
  let steps = 0;
  while (n > 1) {                        // n → n/2 → n/4 → ... → 1
    n = Math.floor(n / 2);              // log₂(n) iterations
    steps++;
  }
  return steps;
}

// O(n log n) — Linearithmic: బయట n, లోపల log n
function nLogN(arr) {
  for (let i = 0; i < arr.length; i++) {       // O(n) బయట
    let n = arr.length;
    while (n > 1) n = Math.floor(n / 2);       // O(log n) లోపల
  }                                            // → O(n log n)
}

console.log(countHalvings(1000));        // 9  (~log₂ 1000)
console.log(allPairs([1, 2, 3]));        // [[1,2],[1,3],[2,3]]
```

Best / Average / Worst case

ఒకే algorithm కి input ని బట్టి time మారొచ్చు:

- **Best case (Ω, Omega):** అత్యంత అనుకూల input. ఉదా: linear search లో target మొదటి element అయితే O(1).
- **Average case (Θ, Theta):** typical input మీద expected time. Linear search లో ~n/2 → O(n).
- **Worst case (O, Big-O):** అత్యంత ప్రతికూల input. Linear search లో target చివర్లో/లేకపోతే O(n).

Interview default = worst case (Big-O). ఎందుకంటే production లో "అన్నీ బాగా జరిగితే" మీద rely అవ్వకూడదు; worst case guarantee ముఖ్యం. అడిగితేనే best/average discuss చేయి.

```
// Linear search — best O(1), worst O(n)
function search(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;     // best: target arr[0] → 1 step
  }
  return -1;                             // worst: లేదు → n steps
}
```

Big-O ఎలా చదవాలి — quick reference

Code pattern	Complexity
ఒక్క statement, index access, hash op	O(1)
ఒక loop (n సార్లు)	O(n)
రెండు nested loops (n × n)	O(n²)
Input ని ప్రతిసారి సగం చేసే loop	O(log n)
Sort చేయడం	O(n log n)

Code pattern	Complexity
Loop లో binary search	O(n log n)
Recursion, ప్రతి call 2 calls చేస్తే, depth n	$O(2^n)$
అన్ని permutations generate	O(n!)

Key Points

- **Big-O = growth rate**, seconds కాదు; hardware-independent, worst-case focus.
- **3 rules:** constants drop, non-dominant terms drop, different inputs = different variables.
- **Nested loops = multiply (×), sequential loops = add (+).**
- క్రమం గుర్తుంచుకో: `O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)`.
- **Interview default = worst case.** అడిగితేనే best/average.
- `O(n log n)` = మంచి sorting/optimal comparison-based limit; `O(2ⁿ)/O(n!)` = brute force → optimize అవసరం.

Interview దృష్టి

Q: రెండు separate loops ఉంటే $O(n^2)$ అవుతుందా? A: కాదు. వరుసగా (nested కాదు) ఉంటే O(n) + O(n) = O(2n) = O(n). $O(n^2)$ అనేది ఒక loop లోపల మరో loop ఉన్నప్పుడే. ఇది చాలా common తప్పు.

Q: `arr.includes()` లేదా `arr.indexOf()` complexity ఎంత? A: O(n) – internally linear scan. అందుకే loop లోపల `includes` వాడితే మొత్తం $O(n^2)$ అవుతుంది. దీనికి బదులు Set (`has()` O(1)) వాడితే O(n) కి తగ్గుతుంది. ఇది interview లో తరచూ దాగి ఉండే trap.

Q: log ఏ base? A: Big-O లో base అర్థరహితం (constant factor), అందుకే `log₂` అయినా `log₁₀` అయినా O(log n) అనే రాస్తాం. కానీ DSA లో ఎక్కువగా base-2 (ప్రతిసారి సగం).

---

4. Space Complexity + Amortized Analysis

వివరణ

Space complexity = "input size పెరిగితే, ఈ algorithm ఎంత *extra memory* వాడుతుంది?" అనేది. Time లాగే Big-O తో రాస్తాం.

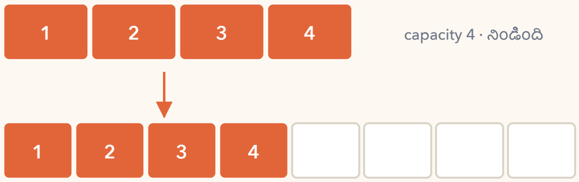
ముఖ్యమైన subtlety: **input ని store చేయడానికి పట్టే memory ని లెక్కించం** (అది మనం create చేసినది కాదు). మనం లెక్కించేది **auxiliary space** – algorithm తను *అదనంగా* create చేసే memory (కొత్త arrays, hash maps, recursion call stack).

రెండు రకాలు:

1. **Auxiliary space** – extra data structures (arrays, maps, sets మనం create చేసేవి).
2. **Call stack space** – recursion లో ప్రతి pending function call stack లో memory పడుతుంది. ఇది చాలా మంది మర్చిపోతారు!

AMORTIZED ANALYSIS · "ఒక్కేసారి ఖరీదు, సగటున చౌక"

DYNAMIC ARRAY · నిండినప్పుడు రెట్టింపు చేయడం



capacity 4 · నిండింది

capacity 8 · అంతా copy – O(n)

ఒక్క operation చూస్తే

ఆ ఒక్క push O(n) – అన్నిటినీ copy చేసింది.
"కాబట్టి push O(n)" అనడం తప్పు.

వరుసగా n pushes చూస్తే

Copy ఖర్చు: $1 + 2 + 4 + \ldots + n < 2n$
n pushes కి మొత్తం O(n) → ఒక్కోదానికి O(1) amortised

ఎందుకు రెట్టింపు, ఒక్కొక్కటి కాదు

ప్రతిసారీ capacity ని +1 చేస్తే – n pushes కి n copies → $O(n^2)$.
రెట్టింపు చేయడం వల్ల copies మధ్య దూరం విపరీతంగా పెరుగుతుంది.
Interview lo "amortised O(1)" అనే మాట వాడితే – ఈ వాదన కూడా చెప్పగలగాలి.

Real-life Scenario

> **Space complexity = వంట చేసేటప్పుడు ఎన్ని అదనపు గిన్నెలు వాడతావు?**
>
> - **O(1) space (in-place)** – ఉన్న గిన్నెలోనే అన్నీ కలిపి వండటం. Guests 10 అయినా 1000 అయినా, నువ్వు వాడే అదనపు గిన్నెలు fixed (ఒకటి, రెండు). Input పెరిగినా extra memory పెరగదు.
> - **O(n) space** – ప్రతి dish కి ఒక కొత్త గిన్నె. 10 dishes = 10 గిన్నెలు; 100 dishes = 100 గిన్నెలు. Input తో పాటు memory linear గా పెరుగుతుంది.
> - **Recursion call stack** – నువ్వు ఒక recipe మధ్యలో ఆపి, "ముందు sauce చేయాలి" అని పక్కన పెట్టి, sauce recipe మొదలుపెడితే – రెండు recipes ఒకేసారి "గుర్తుంచుకోవాలి." 10 levels deep వెళితే 10 recipes memory లో pending. ఇదే call stack.

Auxiliary space – ఉదాహరణలు

```
// O(1) space — extra memory input size తో పెరగదు
function sumInPlace(arr) {
  let total = 0;                        // ఒకే variable — fixed memory
  for (const x of arr) total += x;      // కొత్త array/map ఏదీ లేదు
  return total;                         // → O(1) auxiliary space
}

// O(n) space — input అంత కొత్త memory create
function doubled(arr) {
  const result = [];                    // కొత్త array
  for (const x of arr) result.push(x * 2);  // n elements → O(n) space
  return result;
}

// O(n) space — hash map input అంత grow అవుతుంది
function countFreq(arr) {
  const freq = new Map();               // worst case n distinct keys
  for (const x of arr) freq.set(x, (freq.get(x) || 0) + 1);
```

```
  return freq;                           // → O(n) space
}
```

Call stack space – recursion యొక్క దాచిన memory

Recursion లో ప్రతి function call, return అయ్యేదాకా **call stack** మీద ఉంటుంది. అంటే depth d కి **O(d) space** – code లో కొత్త array కనపడకపోయినా!

```
// ఈ recursion లో array/map ఏది లేదు, కానీ...
function factorial(n) {
  if (n <= 1) return 1;                  // base case
  return n * factorial(n - 1);           // n calls stack మీద pending
}
// factorial(5): factorial(5)→(4)→(3)→(2)→(1) — ఏకకాలంలో 5 frames stack మీద
// → O(n) SPACE (call stack), కేవలం O(1) కాదు!

// Iterative version → O(1) space (stack లేదు)
function factorialIter(n) {
  let result = 1;
  for (let i = 2; i <= n; i++) result *= i;  // ఒకే frame
  return result;
}
```

గుర్తుంచుకో: recursion depth = call stack space. Balanced tree traversal → O(log n) stack; skewed/linear recursion → O(n) stack (deep అయితే *stack overflow* వచ్చే ప్రమాదం).

In-place algorithms

In-place = input ని modify చేస్తూ, O(1) (లేదా O(log n)) extra space మాత్రమే వాడే algorithm. Memory-constrained scenarios లో valuable. ఉదా: array ని reverse చేయడం two pointers తో.

```
// In-place reverse — O(1) space (కొత్త array లేదు)
function reverseInPlace(arr) {
  let left = 0, right = arr.length - 1;
  while (left < right) {
    [arr[left], arr[right]] = [arr[right], arr[left]];  // swap in place
    left++; right--;
  }
  return arr;                            // అదే array modify అయ్యింది
}
console.log(reverseInPlace([1, 2, 3, 4, 5]));  // [5, 4, 3, 2, 1]
```

Amortized Analysis – "average over many operations"

కొన్ని operations అప్పుడప్పుడు ఖరీదైనవి, కానీ ఎక్కువసార్లు చౌక. **Amortized complexity** = చాలా operations మీద average cost. ఒక్కో ఖరీదైన operation ని అన్నిటి మీద "పంచడం."

క్లాసిక్ ఉదాహరణ – Dynamic Array (`push`):

- JS array full అయినప్పుడు, engine కొత్త, **రెట్టింపు size** array allocate చేసి, పాత elements అన్నీ copy చేస్తుంది → ఆ ఒక్క push O(n).

- కానీ ఇది అరుదుగా జరుగుతుంది (size double అయినప్పుడే). మిగతా అన్ని pushes O(1).
- n pushes మొత్తం cost ≈ 2n → ఒక్కో push **amortized O(1)**.

అంటే: individual worst-case O(n) అయినా, **సగటున (amortized) O(1).** అందుకే `arr.push()` ని O(1) అంటాం.

```
// Dynamic array doubling ని simulate చేసి, "పంచడం" చూద్దాం
function simulatePushes(n) {
  let capacity = 1, size = 0, totalCopies = 0;
  for (let i = 0; i < n; i++) {
    if (size === capacity) {            // full → grow (ఖరీదైన step)
      capacity *= 2;                    // capacity double
      totalCopies += size;              // పాత elements copy
    }
    size++;                             // push
  }
  console.log(`${n} pushes → ${totalCopies} total copies`);
  console.log(`per push avg = ${(totalCopies / n).toFixed(2)} (amortized O(1))`);
}
simulatePushes(1000);
// 1000 pushes → 1023 total copies
// per push avg = 1.02 (amortized O(1))  — n పెరిగినా ~2 దగ్గరే ఉంటుంది
```

Hash Map (`set`) కూడా అలాగే – load factor threshold దాటితే *rehash* (అన్ని keys కొత్త, పెద్ద table కి) → O(n) ఒక్కసారి, కానీ amortized O(1) per insert.

Time vs Space Trade-off

చాలా problems లో **time తగ్గించాలంటే space వాడాలి**, vice versa. ఇది core engineering decision:

Strategy	Time	Space	ఉదాహరణ
Hash map వాడటం	తగ్గుతుంది	పెరుగుతుంది	Two Sum: $O(n^2)$→O(n) time, కానీ O(n) space
In-place modify	same	తగ్గుతుంది	Array reverse: O(1) space
Memoization (DP)	తగ్గుతుంది	పెరుగుతుంది	Fibonacci: $O(2^n)$→O(n) time, O(n) space
Recompute	పెరుగుతుంది	తగ్గుతుంది	store చేయకుండా మళ్ళీ లెక్కించడం

Interview లో "ఇక్కడ time తగ్గించడానికి O(n) extra space వాడొచ్చు" అని trade-off explicit గా చెప్పడం strong signal.

Key Points

- **Space complexity = extra (auxiliary) memory**, input storage కాదు.
- **Recursion depth = call stack space** – code లో array కనపడకపోయినా O(depth). చాలా మంది మర్చిపోతారు!
- **In-place = O(1) extra space** – input ని modify చేస్తూ (ఉదా two-pointer swap).
- **Amortized = many ops మీద average.** `push` / `set` occasional O(n) అయినా **amortized O(1).**

- **Time-space trade-off** = core engineering decision; interview లో explicit గా చెప్పు.

Interview దృష్టి

Q: ఈ recursive solution O(1) space అన్నావు – sure నా? A: జాగ్రత్త! Recursion ఎప్పుడూ కనీసం O(depth) call stack space వాడుతుంది. Tree DFS లో balanced అయితే O(log n), skewed అయితే O(n). "కొత్త array లేదు కాబట్టి O(1)" అనేది call stack ని మర్చిపోయే classic తప్పు.

Q: `arr.push()` O(1) అంటావా O(n) అంటావా? A: **Amortized O(1).** అప్పుడప్పుడు (array full అయినప్పుడు) resize+copy కోసం O(n), కానీ ఇది అరుదు కాబట్టి n operations మీద average O(1). Worst-case single push O(n) – కానీ interview లో push ని O(1) గా treat చేయడం సరైనది.

---

5. Common Complexity Cheat-sheet

వివరణ

ఈ topic ఒక **reference** – interview ముందు, revision లో చూసుకోవడానికి. ఈ tables ని బట్టీ కొట్టద్దు; అర్థం చేసుకుని గుర్తుంచుకో (ఎందుకు hash map O(1)? ఎందుకు tree O(log n)?). ఒక్కసారి internal working అర్థమైతే, ఈ numbers automatic గా గుర్తుంటాయి.

Real-life Scenario

> **ఈ cheat-sheet = మెకానిక్ దగ్గర ఉండే "torque specs" chart.** అనుభవం ఉన్న మెకానిక్‌కి కూడా ప్రతి bolt కి ఎంత torque అనేది చార్ట్‌లో చూస్తాడు – గుర్తున్నా, confirm చేసుకుంటాడు. అలాగే నువ్వు ఏ data structure ఎంచుకోవాలో decide చేసేముందు ఈ chart తలలో ఉండాలి – "lookup fast కావాలా? → hash map O(1). Ordered + fast కావాలా? → balanced BST O(log n)."

Data Structure Operations – Big-O (average / worst)

Data Structure	Access	Search	Insert	Delete	Space	గమనిక
Array (static)	O(1)	O(n)	O(n)	O(n)	O(n)	index instant; middle insert/delete shift
Dynamic Array	O(1)	O(n)	O(1)*	O(n)	O(n)	*end push amortized O(1)
Hash Map / Set	–	O(1) / O(n)	O(1) / O(n)	O(1) / O(n)	O(n)	avg O(1); worst O(n) (అన్ని collisions)

Data Structure	Access	Search	Insert	Delete	Space	గమనిక
Linked List (singly)	O(n)	O(n)	O(1)†	O(1)†	O(n)	†node ఉంటే O(1); head insert O(1)
Stack	O(n)	O(n)	O(1)	O(1)	O(n)	push/pop top మాత్రమే
Queue / Deque	O(n)	O(n)	O(1)	O(1)	O(n)	ends మీద ops O(1)
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)	unbalanced అయితే O(n)!
Heap (binary)	–	O(n)	O(log n)	O(log n)	O(n)	min/max peek O(1); arbitrary search O(n)
Trie	–	O(L)	O(L)	O(L)	O(alphabet×nodes)	L = word length
Union-Find	–	O(α(n))	O(α(n))	–	O(n)	α = inverse Ackermann ≈ O(1)

(dash `–` = ఆ operation ఆ structure కి typical గా apply కాదు.)

Sorting Algorithms – Big-O

Algorithm	Best	Average	Worst	Space	Stable?	గమనిక
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✅	teaching only; slow
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	❌	ఎప్పుడూ n^2; fewest swaps
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	✅	small/nearly-sorted కి best
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✅	guaranteed n log n; extra space
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	❌	avg fastest; bad pivot → n^2
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	❌	in-place; poor cache locality
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	✅	k = value range; integers only
Radix Sort	O(nk)	O(nk)	O(nk)	O(n+k)	✅	k = digits; integers/strings

JS `Array.prototype.sort()` – engines లో typically **Timsort** (merge + insertion hybrid) → O(n log n), stable (ES2019+). కానీ **default comparator strings గా sort చేస్తుంది** – numbers కి తప్పకుండా comparator ఇవ్వాలి (Topic 7 gotcha).

Input Size → Expected Complexity (అత్యంత useful!)

Interview లో **constraints నీకు expected solution చెప్తాయి.** n range చూసి, target complexity guess చేయి:

n (input size)	ఏ complexity పనిచేస్తుంది	సూచన
$n \le 10–12$	$O(n!)$, $O(2^n)$	permutations/backtracking ఓకే
$n \le 20–25$	$O(2^n)$	subsets, bitmask DP
$n \le 100$	$O(n^3)$	triple nested loops ఓకే
$n \le 1,000$	$O(n^2)$	nested loops ఓకే
$n \le 10^5$	O(n log n)	sort / heap / balanced structures
$n \le 10^6–10^7$	O(n) లేదా O(n log n)	single pass / two pointers / sliding window
$n \ge 10^8$	O(log n), O(1)	math / binary search మాత్రమే

Rule of thumb: typical judge ~10^8 operations/second చేస్తుంది. n = 10^5 ఉంటే, $O(n^2) = 10^{10}$ → చాలా slow (TLE); O(n log n) = ~$1.7×10^6$ → instant. అందుకే **constraint చూడగానే target complexity fix చేసుకో.**

Big-O Growth – n పెరిగితే steps

n	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(2^n)$
10	~3	10	~33	100	1,024
100	~7	100	~664	10,000	10^{30}
1,000	~10	1,000	~10^4	10^6	ఊహాకందదు
1,000,000	~20	10^6	~$2×10^7$	10^{12}	–

ఈ table చూస్తే **$O(n^2)$ ఎంత త్వరగా blow up అవుతుందో, O(log n) ఎంత నెమ్మదిగా పెరుగుతుందో** స్పష్టంగా కనపడుతుంది. n=10^6 కి log n కేవలం 20!

Code – complexity ని కళ్ళతో చూద్దాం

```
// ఒకే problem (array లో duplicate ఉందా?) – రెండు approaches, benchmark
function hasDupBrute(arr) {          // O(n²)
  for (let i = 0; i < arr.length; i++)
    for (let j = i + 1; j < arr.length; j++)
      if (arr[i] === arr[j]) return true;
```

```
    return false;
  }

  function hasDupSet(arr) {            // O(n)
    const seen = new Set();
    for (const x of arr) {
      if (seen.has(x)) return true;   // O(1) lookup
      seen.add(x);
    }
    return false;
  }

  const big = Array.from({ length: 20000 }, (_, i) => i);  // no dup, worst case

  console.time("brute O(n^2)");
  hasDupBrute(big);
  console.timeEnd("brute O(n^2)");     // ~సెకన్లలో కొంత (~200-400ms)

  console.time("set O(n)");
  hasDupSet(big);
  console.timeEnd("set O(n)");         // ~1-3ms — వందల రెట్లు వేగం!
```

Key Points

- **Hash map/set = O(1) average** – DSA లో అత్యంత powerful; lookup/insert/delete అన్నీ instant.
- **Balanced BST/heap = O(log n)** – ordered data + fast ops కావాలంటే.
- **Sorting comparison-based limit = O(n log n)** – merge/quick/heap. Integer range తెలిస్తే counting/radix O(n).
- **Constraint (n) → target complexity** – $n \le 10^5$ అంటే O(n log n); $n \le 20$ అంటే $O(2^n)$ ఒకే.
- **Unbalanced BST = O(n)** – ఇది hidden trap; balanced అని assume చేయకు.
- ఈ tables బట్టీ కాదు, **అర్థం చేసుకో** – internal working తెలిస్తే numbers automatic.

Interview దృష్టి

Q: Interviewer "n = 10^5" అన్నాడు – ఏ complexity target చేయాలి? A: O(n log n) లేదా అంతకంటే better. $O(n^2) = 10^{10}$ operations → TLE. అందుకే sort + two pointers, లేదా hash map single pass వైపు ఆలోచించు. **Constraint = free hint.** ఎప్పుడూ n range అడుగు.

Q: Hash map worst case O(n) కదా, అప్పుడు దాన్ని O(1) ఎలా అంటావు? A: Practical గా, మంచి hash function + resizing తో collisions అరుదు, అందుకే **average O(1)** అని treat చేస్తాం (ఇది real-world లో నిజం). Worst case O(n) అనేది అన్ని keys ఒకే bucket కి hash అయితే – adversarial/rare. Interview లో average O(1) చెప్పి, అడిగితే worst case mention చేయి.

Part 2 – Recursion & Core Techniques

> ఇప్పటిదాకా mindset + measurement నేర్చుకున్నాం. ఇప్పుడు **techniques** – recursion (చాలా advanced topics కి పునాది: trees, graphs, backtracking, DP), sorting, searching, మరియు అన్ని ముఖ్యమైన patterns overview. Recursion ముఖ్యంగా చాలా మందికి కష్టం అనిపిస్తుంది – దాన్ని మనం మొదటి principle నుండి, call stack తో సహా, పూర్తిగా విప్పుతాం.

---

6. Recursion from scratch

వివరణ

Recursion = ఒక function **తనను తానే** పిలుచుకోవడం (self-call), problem ని చిన్న చిన్న అదే-రకమైన sub-problems గా విడగొట్టేదాకా.

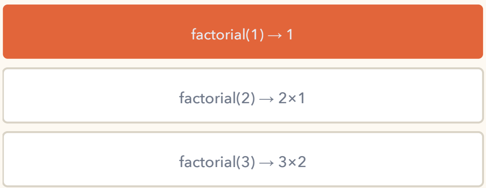
ప్రతి recursion కి **తప్పనిసరిగా 2 భాగాలు** ఉండాలి:

1. **Base case** – recursion **ఆగే** condition (అత్యంత చిన్న, direct answer తెలిసిన case). ఇది లేకపోతే infinite recursion → **stack overflow** crash.
2. **Recursive case** – problem ని **చిన్నదిగా చేసి** తనను తానే పిలుచుకోవడం, base case వైపు కదులుతూ.

కీలక mindset: **"నేను మొత్తం problem solve చేయను. ఒక్క చిన్న step చేసి, మిగతాది 'నా చిన్న version' చేస్తుందని నమ్ముతాను."** ఈ నమ్మకమే (leap of faith) recursion అర్థం చేసుకోవడానికి తాళం.

RECURSION · CALL STACK మరియు RECURSION TREE

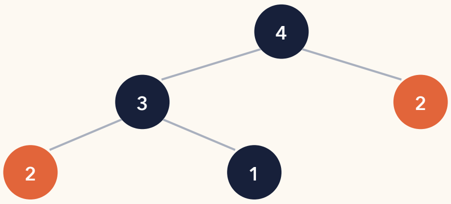
CALL STACK · factorial(3)



↑ base case – ఇక్కడ ఆగకపోతే **stack overflow**

పైనుంచి కిందికి పేరుకుంటుంది · base case దగ్గర ఆగి వెనక్కి కరుగుతుంది

RECURSION TREE · fib(4) – ఒకే పని పదే పదే



2 రెండుసార్లు లెక్కించబడింది

n పెరిగితే $O(2^n)$

→ memo చేర్చితే O(n)

ప్రతి recursion కి మూడు ప్రశ్నలు

1 · Base case ఏమిటి? (ఎక్కడ ఆగాలి)
2 · Recursive case problem ని ఎలా చిన్నది చేస్తుంది?
3 · Base case వైపు నిజంగా కదులుతున్నామా? – లేకపోతే అనంత recursion

Real-life Scenario

> **Recursion = రష్యన్ నేస్టింగ్ dolls (matryoshka).** ఒక పెద్ద బొమ్మ తెరిస్తే లోపల చిన్న బొమ్మ, దాన్లో ఇంకా చిన్నది... చివరికి తెరవలేని అతి చిన్న బొమ్మ (**base case**).
>
> "మొత్తం ఎన్ని బొమ్మలు?" లెక్కపెట్టాలంటే: **"1 (ఈ బొమ్మ) + లోపలి బొమ్మల count."** లోపలి count ఎలా? అదే process మళ్ళీ! చివరి బొమ్మ (base case) దగ్గర "లోపల 0" అని ఆగుతావు.
>
> మరో analogy: **సినిమా hall లో "నేను ఏ row లో ఉన్నా?"** అని తెలుసుకోవడం. ముందు వ్యక్తిని అడుగుతావు: "నువ్వు ఏ row?" వాడు వాడి ముందు వ్యక్తిని అడుగుతాడు... మొదటి row వ్యక్తి (**base case**) "నేను row 1" అంటాడు. తర్వాత వెనక్కి: వాడి వెనక వ్యక్తి "row 2," ఆ వెనక "row 3"... ఇదే recursion – **ముందుకు breaking down, వెనక్కి building up.**

Base case + Recursive case – factorial

```
// factorial(n) = n × (n−1) × (n−2) × ... × 1
function factorial(n) {
  // BASE CASE: అత్యంత చిన్న case — direct answer
  if (n <= 1) return 1;           // 0! = 1! = 1
  // RECURSIVE CASE: n × (దాని చిన్న version)
  return n * factorial(n - 1);    // n ని చిన్నదిగా (n−1) చేసి పిలుస్తున్నాం
}
console.log(factorial(5));        // 120  (5×4×3×2×1)
```

Call stack visualization – లోపల ఏం జరుగుతుంది

`factorial(5)` పిలిస్తే, ప్రతి call **return అయ్యేదాకా stack మీద wait** చేస్తుంది. ముందు **అడుగుకి వెళ్ళడం** (winding), తర్వాత **పైకి రావడం** (unwinding):

```
winding down (deeper calls stacked):
  factorial(5) → 5 * factorial(4)      ┐ wait
    factorial(4) → 4 * factorial(3)    │ wait
      factorial(3) → 3 * factorial(2)  │ wait
        factorial(2) → 2 * factorial(1)│ wait
          factorial(1) → 1  ← BASE CASE ┘ ఇక్కడ ఆగి, వెనక్కి తిరుగుతుంది

unwinding up (results bubble back):
          factorial(1) = 1
        factorial(2) = 2 * 1  = 2
      factorial(3) = 3 * 2    = 6
    factorial(4) = 4 * 6      = 24
  factorial(5) = 5 * 24       = 120   ← final answer
```

ముఖ్యం: ఏకకాలంలో 5 frames stack మీద ఉన్నాయి → **O(n) call stack space** (Topic 4 గుర్తుందా?). ప్రతి frame తన `n` value, తను ఎక్కడ ఆగిందో గుర్తుంచుకుంటుంది.

Recursion tree – fibonacci (branching)

కొన్ని recursions ప్రతి step లో **ఒకటి కంటే ఎక్కువ calls** చేస్తాయి → tree లా విస్తరిస్తాయి:

```
// fib(n) = fib(n-1) + fib(n-2), fib(0)=0, fib(1)=1
function fib(n) {
  if (n < 2) return n;             // BASE: fib(0)=0, fib(1)=1
  return fib(n - 1) + fib(n - 2);  // రెండు recursive calls (branching!)
}
console.log(fib(6));               // 8
```

```
fib(5) recursion tree:
                fib(5)
              /        \
         fib(4)        fib(3)
challenges/    \        /    \
     fib(3)  fib(2)  fib(2) fib(1)
     /  \    /  \    /  \
   ...  ...(చాలా repeated work! fib(3), fib(2) మళ్ళీ మళ్ళీ)
```

ఇక్కడ **అదే sub-problems మళ్ళీ మళ్ళీ లెక్కిస్తున్నాం** → O(2^n), చాలా slow. దీన్ని **memoization** (లెక్కించినది cache చేయడం) తో O(n) కి తగ్గించడమే **Dynamic Programming** (Topic 9 + DSA_10 doc). ఇదే recursion → DP కి bridge.

ఎలా recursively ఆలోచించాలి – 3 steps

ఏ recursive problem కైనా ఈ 3 questions అడుక్కో:

1. **Base case ఏమిటి?** – అత్యంత చిన్న input కి direct answer ఏది? (empty? n=0? single node?)
2. **Problem ని ఎలా చిన్నదిగా చేయాలి?** – ప్రతి call ఎలా base case వైపు కదులుతుంది? (n-1? array half? tree child?)
3. **Sub-problem results ని ఎలా combine చేయాలి?** – చిన్న version answer తో మొత్తం answer ఎలా build చేయాలి? (multiply? add? merge?)

"Leap of faith": recursive call correctly పనిచేస్తుందని నమ్ము. లోపలికి mentally dig చేయకు – base case + combine logic సరిగా ఉంటే, మొత్తం సరిగా పనిచేస్తుంది.

మరిన్ని examples

```
// 1. Array sum – recursively
function arraySum(arr, i = 0) {
  if (i === arr.length) return 0;          // BASE: elements అయిపోయాయి
  return arr[i] + arraySum(arr, i + 1);    // ఈ element + మిగతా sum
}
console.log(arraySum([1, 2, 3, 4]));       // 10

// 2. Reverse a string – recursively
function reverse(str) {
  if (str.length <= 1) return str;         // BASE: ఖాళీ/1 char already reversed
  return reverse(str.slice(1)) + str[0];   // మిగతా reverse + మొదటి char చివర్లో
}
console.log(reverse("hello"));             // "olleh"

// 3. Power – x^n
function power(x, n) {
```

```
  if (n === 0) return 1;                  // BASE: x^0 = 1
  return x * power(x, n - 1);             // x × x^(n-1)
}
console.log(power(2, 10));                // 1024
```

Recursion vs Iteration

అంశం	Recursion	Iteration (loops)
విధానం	Function తనను తానే పిలవడం	Loop (for/while) తో repeat
Space	O(depth) call stack	సాధారణంగా O(1)
Readability	Tree/graph/divide-conquer కి natural, clean	Linear tasks కి simple
Risk	Deep అయితే **stack overflow**	Overflow లేదు
Speed	Function call overhead (కొంచెం slow)	సాధారణంగా వేగం
ఎప్పుడు	Trees, backtracking, D&C, DP	Simple linear scans, counting

Rule: structure recursive అయితే (tree, nested) → recursion natural. Purely linear అయితే → iteration తరచూ better. చాలా recursions ని iteration (+ explicit stack) గా rewrite చేయవచ్చు.

Gotchas (సాధారణ తప్పులు)

- **Base case మర్చిపోవడం / తప్పు** → infinite recursion → `Maximum call stack size exceeded` (stack overflow). ✅ ఎప్పుడూ base case మొదట రాయి.
- **Base case వైపు కదలకపోవడం** – `factorial(n)` లోపల `factorial(n)` పిలిస్తే (n తగ్గించకుండా) → infinite. ✅ ప్రతి call input చిన్నదిగా అవ్వాలి.
- **Fibonacci లా exponential blowup** – memoization లేకుండా branching recursion $O(2^n)$. ✅ overlapping subproblems ఉంటే cache చేయి.
- **చాలా deep recursion** ($n=10^6$) → JS default stack ~10^4-10^5 frames → overflow. ✅ deep linear recursion ని iteration గా మార్చు.

Key Points

- **Recursion = base case + recursive case.** Base case లేకపోతే stack overflow.
- **Call stack:** winding (deeper) then unwinding (results bubble up). Depth = O(depth) space.
- **3 questions:** base case? ఎలా చిన్నదిగా? results ఎలా combine? + **leap of faith.**
- **Branching recursion (fib)** = repeated work → $O(2^n)$; **memoization** → **DP** → O(n).
- Recursion = trees/backtracking/D&C కి natural; deep linear కి iteration better.
- **Recursion అర్థమైతే** trees, graphs, backtracking, DP అన్నీ సులభం – ఇది foundation.

Interview దృష్టి

Q: ఈ recursion time/space complexity ఎలా లెక్కించాలి? A: **Time** = (calls సంఖ్య) × (per-call work). Linear recursion (factorial) = n calls × O(1) = O(n). Branching (fib) = $O(2^n)$. **Space** = max

recursion depth (call stack) = linear కి O(n), balanced tree కి O(log n).

Q: Recursion ని iteration గా ఎప్పుడు మార్చాలి? A: Depth చాలా ఎక్కువ అయ్యి stack overflow ప్రమాదం ఉంటే (deep linear recursion), లేదా tail-call లేని language లో performance కావాలంటే. Tree/graph traversal ని explicit stack/queue తో iterative గా రాయవచ్చు – DFS (stack), BFS (queue).

---

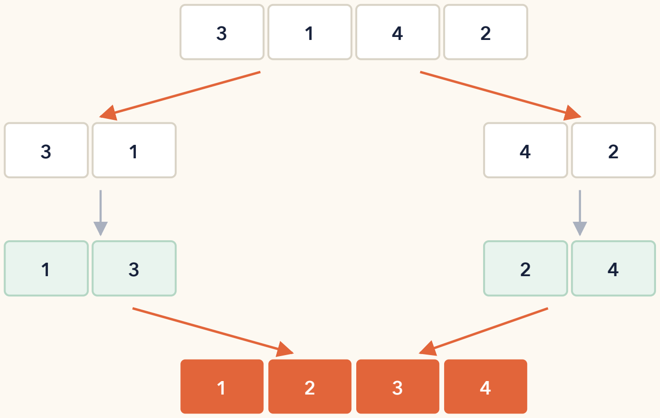
7. Sorting essentials

వివరణ

Sorting = elements ని ఒక క్రమంలో (ascending/descending) అమర్చడం. ఇది DSA లో అత్యంత fundamental – ఎందుకంటే **sorted data మీద చాలా problems దానంతట సులభం అవుతాయి** (binary search possible, duplicates పక్కపక్కన, two pointers work). Interview లో నువ్వు scratch నుండి sort రాయకపోవచ్చు (built-in వాడతావు), కానీ **merge sort & quick sort లోపల ఎలా పనిచేస్తాయో తెలియడం తప్పనిసరి** – అవి divide-and-conquer + recursion కి perfect examples.

SORTING · MERGE SORT యొక్క విభజన

MERGE SORT – విడగొట్టి, క్రమపరిచి, కలపడం



Merge Sort

ఎప్పుడూ O(n log n) – worst case కూడా
STABLE (సమాన elements క్రమం మారదు)
ఖరీదు: O(n) extra space

Quick Sort

సగటున O(n log n), worst $O(n^2)$
In-place – O(log n) space మాత్రమే
Random pivot తో worst case దాదాపు రాదు

ఎప్పుడు ఏది: Stability కావాలంటే merge sort (objects ని పలు keys మీద sort చేసేటప్పుడు ముఖ్యం). Memory పరిమితమైతే quick sort. JavaScript యొక్క `Array.sort()` నేడు Timsort – merge + insertion కలయిక, stable.

Real-life Scenario

> **Sorting = పేకాట (cards) ని చేతిలో అమర్చడం.** నీకు cards ఇచ్చినప్పుడు, నువ్వు వాటిని sort చేస్తావు – కానీ ఎలా?
>
> - **Insertion sort** = ఒక్కో card తీసుకుని, అప్పటికే sorted అయిన cards లో సరైన చోట పెట్టడం. (మనం సహజంగా cards ఇలాగే అమరుస్తాం!)
> - **Selection sort** = అన్నిటిలో అతి చిన్నది వెతికి ముందు పెట్టు, తర్వాత రెండో చిన్నది... (ప్రతిసారి scan).
> - **Merge sort** = cards ని రెండు గుంపులుగా విడగొట్టి, ఒక్కో గుంపు sort చేసి, తర్వాత రెండిటినీ **కలిపి (merge)** అమర్చడం – ఇద్దరు friends సగం సగం sort చేసి కలిపినట్టు.
> - **Quick sort** = ఒక card ని "pivot" గా ఎంచుకుని, దాని కంటే చిన్నవి ఎడమ, పెద్దవి కుడి పెట్టి, ఆ రెండు వైపులా recursively repeat.

Simple sorts (brief – teaching purpose)

మూడు $O(n^2)$ sorts – interview లో వాడవు కానీ concept కి పునాది:

```
// BUBBLE SORT – పక్కపక్క pairs swap చేస్తూ; పెద్దవి "bubble up"
function bubbleSort(arr) {
  for (let i = 0; i < arr.length - 1; i++) {
    let swapped = false;
    for (let j = 0; j < arr.length - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {                        // పక్క element పెద్దదా?
        [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];   // swap
        swapped = true;
      }
    }
    if (!swapped) break;    // ఏమీ swap కాలేదు = already sorted → early exit (best
O(n))
  }
  return arr;
}

// SELECTION SORT – ప్రతిసారి min వెతికి ముందుకి తెచ్చు
function selectionSort(arr) {
  for (let i = 0; i < arr.length - 1; i++) {
    let minIdx = i;
    for (let j = i + 1; j < arr.length; j++)
      if (arr[j] < arr[minIdx]) minIdx = j;            // అతి చిన్నది వెతుకు
    [arr[i], arr[minIdx]] = [arr[minIdx], arr[i]];     // ముందుకి swap
  }
  return arr;
}

// INSERTION SORT – sorted భాగంలో సరైన చోట insert (nearly-sorted కి best!)
function insertionSort(arr) {
  for (let i = 1; i < arr.length; i++) {
    const key = arr[i];
    let j = i - 1;
    while (j >= 0 && arr[j] > key) {   // key కంటే పెద్దవాటిని కుడికి జరుపు
      arr[j + 1] = arr[j];
      j--;
```

```
    }
    arr[j + 1] = key;                  // సరైన చోట key పెట్టు
  }
  return arr;
}
console.log(insertionSort([5, 2, 9, 1, 5, 6, 3])); // [1, 2, 3, 5, 5, 6, 9]
```

Merge Sort (deep) — guaranteed O(n log n), stable

Idea (divide & conquer): array ని సగానికి విడగొట్టు → ప్రతి సగం recursively sort చేయి → రెండు sorted halves ని merge చేయి.

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;              // BASE: 0/1 element already sorted
  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));    // ఎడమ సగం sort
  const right = mergeSort(arr.slice(mid));      // కుడి సగం sort
  return merge(left, right);                    // రెండు sorted halves కలుపు
}

// రెండు sorted arrays ని ఒక sorted array గా merge చేయడం (కీలక step)
function merge(a, b) {
  const result = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length) {        // రెండిటిలో చిన్నదాన్ని తీసుకో
    if (a[i] <= b[j]) result.push(a[i++]);      // <= → stable (equal అయితే ఎడమది
ముందు)
    else result.push(b[j++]);
  }
  while (i < a.length) result.push(a[i++]);     // మిగిలిన a
  while (j < b.length) result.push(b[j++]);     // మిగిలిన b
  return result;
}
console.log(mergeSort([5, 2, 9, 1, 5, 6, 3])); // [1, 2, 3, 5, 5, 6, 9]
```

ఎందుకు O(n log n)? విడగొట్టడం = log n levels (ప్రతిసారి సగం). ప్రతి level లో మొత్తం merge work = O(n). కాబట్టి n × log n. **ఎప్పుడూ** O(n log n) — worst case కూడా (guaranteed). Trade-off: **O(n) extra space** (merge కి కొత్త arrays). **Stable** (equal elements order preserve). Linked lists, external/huge data కి ideal.

Quick Sort (deep) — avg O(n log n), in-place

Idea: ఒక **pivot** ఎంచుకో → array ని rearrange చేయి: pivot కంటే చిన్నవి ఎడమ, పెద్దవి కుడి (**partition**) → pivot ఇప్పుడు దాని final స్థానంలో → ఎడమ, కుడి భాగాలని recursively quick sort.

```
function quickSort(arr, lo = 0, hi = arr.length - 1) {
  if (lo < hi) {
    const p = partition(arr, lo, hi);   // pivot final స్థానంలో పెట్టు
    quickSort(arr, lo, p - 1);          // ఎడమ భాగం
    quickSort(arr, p + 1, hi);          // కుడి భాగం
  }
  return arr;                           // IN-PLACE — అదే array modify
}
```

```
// Lomuto partition: చివరి element ని pivot గా తీసుకో
function partition(arr, lo, hi) {
  const pivot = arr[hi];
  let i = lo - 1;                       // "చిన్నవాటి" boundary
  for (let j = lo; j < hi; j++) {
    if (arr[j] < pivot) {              // pivot కంటే చిన్నదా?
      i++;
      [arr[i], arr[j]] = [arr[j], arr[i]];   // చిన్నవి ఎడమకి తెచ్చు
    }
  }
  [arr[i + 1], arr[hi]] = [arr[hi], arr[i + 1]];  // pivot ని మధ్యలో పెట్టు
  return i + 1;                         // pivot యొక్క final index
}
console.log(quickSort([5, 2, 9, 1, 5, 6, 3])); // [1, 2, 3, 5, 5, 6, 9]
```

Avg O(n log n), worst O(n²) (already-sorted array + last-element pivot → అన్నీ ఒక వైపు). Fix: **random pivot** లేదా median-of-three. **In-place** (O(log n) stack), cache-friendly → practical గా often fastest. **Not stable.**

Merge vs Quick – comparison

అంశం	Merge Sort	Quick Sort
Average time	O(n log n)	O(n log n)
Worst time	O(n log n) ✅ guaranteed	O(n²) (bad pivot)
Space	O(n) extra	O(log n) (in-place)
Stable?	✅ అవును	❌ కాదు
Practical speed	కొంచెం slow (copying)	తరచూ fastest (cache)
ఎప్పుడు	stability కావాలి, linked list, huge/external data	general in-memory, memory-tight

Counting Sort (idea) – O(n+k), comparisons లేవు!

Elements integers, range (k) చిన్నదైతే, **compare చేయకుండా** count చేసి place చేయవచ్చు → O(n+k), comparison-based O(n log n) limit ని దాటేస్తుంది.

```
function countingSort(arr) {
  if (arr.length === 0) return arr;
  const max = Math.max(...arr);
  const count = new Array(max + 1).fill(0);
  for (const x of arr) count[x]++;          // ప్రతి value ఎన్నిసార్లు?
  const result = [];
  for (let v = 0; v <= max; v++)
    while (count[v]-- > 0) result.push(v);   // count ప్రకారం place
  return result;
}
console.log(countingSort([4, 2, 2, 8, 3, 3, 1])); // [1, 2, 2, 3, 3, 4, 8]
```

range k పెద్దదైతే (ఉదా 0 నుండి 10^9) waste – అప్పుడు comparison sort better. Radix sort = counting sort ని digits మీద apply చేసేది.

JS `Array.sort()` gotcha (అత్యంత ముఖ్యం!)

```
// ❌ TRAP: default sort elements ని STRINGS గా convert చేసి lexicographic sort
console.log([10, 2, 1, 20, 3].sort());          // [1, 10, 2, 20, 3]  ← తప్పు!
// ("10" < "2" ఎందుకంటే '1' < '2' — string comparison)

// ✅ Numbers కి తప్పకుండా comparator ఇవ్వాలి:
console.log([10, 2, 1, 20, 3].sort((a, b) => a - b));  // [1, 2, 3, 10, 20]
ascending
console.log([10, 2, 1, 20, 3].sort((a, b) => b - a));  // [20, 10, 3, 2, 1]
descending
```

Comparator rule: `(a, b) => a - b` → negative అయితే a ముందు, positive అయితే b ముందు, 0 అయితే same. Numbers కి ఇది తప్పనిసరి! `.sort()` **in-place** (original array modify) – return కూడా చేస్తుంది కానీ original మారుతుంది. JS engines Timsort వాడతాయి → O(n log n), **stable (ES2019+).**

Stability అంటే ఏమిటి?

Stable sort = equal keys ఉన్న elements వాటి **original relative order** ని preserve చేస్తుంది. ఉదా: employees ని ముందు name తో sort చేసి, తర్వాత stable గా department తో sort చేస్తే – ఒకే department లో వాళ్ళు name order లోనే ఉంటారు. Multi-key sorting కి ఇది కీలకం.

Key Points

- **Sorted data = చాలా problems సులభం** (binary search, two pointers, dedup).
- **Merge sort:** divide & conquer, **guaranteed O(n log n)**, **stable**, O(n) space.
- **Quick sort:** partition around pivot, **avg O(n log n)**, in-place O(log n), **worst $O(n^2)$** (random pivot తో fix), not stable.
- **Counting sort:** integers + చిన్న range → O(n+k), comparisons లేవు.
- **JS `.sort()` gotcha:** default = string sort! Numbers కి `(a,b)=>a-b` తప్పనిసరి; in-place; stable (ES2019+).
- **Stable** = equal elements original order preserve – multi-key sort కి ముఖ్యం.

Interview దృష్టి

Q: JS లో numbers array ని sort ఎలా? A: `arr.sort((a, b) => a - b)`. Default `arr.sort()` elements ని strings గా చూస్తుంది కాబట్టి `[10, 2]` → `[10, 2]` (తప్పు). ఇది interview live-coding లో అత్యంత common bug – comparator మర్చిపోవద్దు.

Q: Merge vs quick – ఏది ఎంచుకుంటావు? A: Worst-case guarantee లేదా stability కావాలంటే merge sort (O(n log n) always). Memory tight అయి average performance చాలు అంటే quick sort (in-place). Practice లో languages built-in sort తరచూ hybrid (Timsort/introsort) – రెండింటి బలాలు కలిపి.

Q: O(n log n) కంటే fast sort సాధ్యమా? A: Comparison-based sorts కి O(n log n) theoretical limit. కానీ data integers + range తెలిస్తే counting/radix sort O(n) సాధించవచ్చు (compare చేయవు కాబట్టి limit apply కాదు).

---

8. Searching essentials

వివరణ

Searching = collection లో ఒక element (లేదా condition satisfy చేసే స్థానం) వెతకడం. రెండు fundamental approaches:

- **Linear search** – మొదటి నుండి ఒక్కో element check → **O(n).** Unsorted data కి ఇదే మార్గం.
- **Binary search** – data **sorted అయి ఉంటే**, ప్రతిసారి search space ని **సగం తగ్గించడం → O(log n).** DSA లో అత్యంత powerful, elegant technique.

కీలక condition: binary search కి data **sorted** అయి ఉండాలి (లేదా monotonic property ఉండాలి). ఇదే దాని superpower మరియు limitation.

BINARY SEARCH · ప్రతి అడుగులో సగం

ప్రతి అడుగులో సగం పరిధి పోతుంది

ముందస్తు షరతు – ఇది మర్చిపోవద్దు

Array SORTED అయ్యుండాలి. లేకపోతే "ఏ వైపు వెళ్ళాలి" అని నిర్ణయించలేం.
Sort చేయడం O(n log n) – ఒక్కసారే వెతికితే linear scan O(n) మేలు.
పలుసార్లు వెతికితే – ఒకసారి sort చేసి, తర్వాత ప్రతి search O(log n).

Real-life Scenario

> **Binary search = dictionary లో పదం వెతకడం.** "Telugu" అనే పదం కోసం నువ్వు మొదటి page నుండి ఒక్కో page తిప్పవు (అది linear – O(n)). బదులుగా **మధ్య page తెరుస్తావు.** "M" కనిపిస్తే – "T" తర్వాత వస్తుంది కాబట్టి **మొదటి సగం మొత్తం వదిలేసి**, రెండో సగంలో మళ్ళీ మధ్యకి వెళ్తావు. ప్రతిసారి **సగం pages అదృశ్యం.** 1000 pages → కేవలం ~10 steps లో పదం దొరుకుతుంది.
>
> **ఇది ఎందుకు పనిచేస్తుంది?** Dictionary **sorted** (alphabetical). Pages randomly అమర్చి ఉంటే, ఈ trick పనిచేయదు – మొదటి నుండి వెతకాల్సిందే. అందుకే **binary search కి sorted data తప్పనిసరి.**
>
> అదే idea – "నేను ఊహించిన number ఏమిటి? (1-100)" game. నువ్వు 50 అంటావు, "పెద్దది" అంటే 75, "చిన్నది" అంటే 62… ప్రతి guess తో సగం options పోతాయి. 100 numbers → గరిష్టంగా 7 guesses.

Linear Search – simple, works everywhere

```
function linearSearch(arr, target) {
  for (let i = 0; i < arr.length; i++) {
    if (arr[i] === target) return i;    // దొరికింది → index return
  }
  return -1;                            // లేదు
}
console.log(linearSearch([4, 2, 7, 1, 9], 7));  // 2
// Time: O(n) worst (target చివర్లో/లేకపోతే). Data sorted అవసరం లేదు.
```

Binary Search – standard template

```
// arr SORTED (ascending) అయి ఉండాలి. target ఉంటే దాని index, లేకపోతే -1.
function binarySearch(arr, target) {
  let lo = 0, hi = arr.length - 1;      // search boundaries (both inclusive)
  while (lo <= hi) {                    // range ఖాళీ అయ్యేదాకా
    const mid = lo + Math.floor((hi - lo) / 2);  // overflow-safe mid
    if (arr[mid] === target) return mid;         // సరిగ్గా దొరికింది
    else if (arr[mid] < target) lo = mid + 1;    // target కుడి సగంలో
    else hi = mid - 1;                            // target ఎడమ సగంలో
  }
  return -1;                            // లేదు
}
const a = [1, 3, 5, 7, 9, 11];
console.log(binarySearch(a, 7));   // 3
console.log(binarySearch(a, 8));   // -1  (లేదు)
```

3 gotchas గుర్తుంచుకో:

1. `mid = lo + (hi-lo)/2` – `(lo+hi)/2` కంటే safe (పెద్ద numbers లో overflow నివారిస్తుంది; JS లో పెద్ద ప్రమాదం కాదు కానీ మంచి అలవాటు).
2. `lo <= hi` (inclusive hi template లో) – `<` వాడితే off-by-one bug.

3. `mid + 1` / `mid - 1` – mid ని exclude చేయాలి, లేకపోతే **infinite loop** (lo/hi కదలవు).

ఎందుకు O(log n)? ప్రతి iteration search space సగం → n → n/2 → n/4 → ... → 1. అది $\log_2(n)$ steps. $n=10^6$ → కేవలం ~20 steps!

Variant 1: Leftmost / first occurrence (lower bound)

Duplicates ఉన్నప్పుడు target యొక్క **మొదటి** occurrence (లేదా target ≥ ఉన్న మొదటి index) కావాలంటే – ఈ **half-open template** అత్యంత reliable:

```
// arr[i] >= target అయ్యే మొదటి index return (target insert position కూడా ఇదే)
function lowerBound(arr, target) {
  let lo = 0, hi = arr.length;          // hi EXCLUSIVE (length, length-1 కాదు)
  while (lo < hi) {                      // '<' (inclusive template కాదు)
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] < target) lo = mid + 1;  // చిన్నది → కుడికి
    else hi = mid;                         // >= target → mid ని ఉంచు, ఎడమ కొనసాగించు
  }
  return lo;                             // మొదటి index where arr[i] >= target
}
const b = [1, 2, 2, 2, 3, 4];
console.log(lowerBound(b, 2));  // 1  (మొదటి 2)
console.log(lowerBound(b, 5));  // 6  (అన్నిటి కంటే పెద్దది → insert at end)
```

Variant 2: Rightmost / upper bound + count

```
// arr[i] > target అయ్యే మొదటి index (target తర్వాతి స్థానం)
function upperBound(arr, target) {
  let lo = 0, hi = arr.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (arr[mid] <= target) lo = mid + 1;  // <= → కుడికి (target ని దాటాలి)
    else hi = mid;
  }
  return lo;
}
const b2 = [1, 2, 2, 2, 3, 4];
// target ఎన్నిసార్లు ఉంది? = upperBound - lowerBound
console.log(upperBound(b2, 2) - lowerBound(b2, 2));  // 3  (మూడు 2లు)
```

Variant 3: Binary search on the "answer" (predicate)

Binary search కి array అవసరం లేదు! ఏదైనా **monotonic predicate** (false...false, true...true) మీద "మొదటి true" వెతకవచ్చు. ఇది "minimum X such that condition holds" problems కి కీలకం (ఉదా Koko Eating Bananas, Ship Packages).

```
// [lo, hi) range లో pred(x) మొదటిసారి true అయ్యే x వెతుకు
// (pred: false,false,...,false,TRUE,true,...true అని monotonic అనుకో)
function firstTrue(lo, hi, pred) {
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (pred(mid)) hi = mid;              // true → ఇది కావొచ్చు, ఎడమ చూడు
    else lo = mid + 1;                    // false → కుడి చూడు
```

```
  }
  return lo;
}
// x² >= 50 అయ్యే మొదటి x (0..20 లో)?
console.log(firstTrue(0, 20, x => x * x >= 50));  // 8  (8²=64 ≥ 50, 7²=49 < 50)
```

Linear vs Binary – comparison

అంశం	Linear Search	Binary Search
Time	O(n)	O(log n)
Data sorted అవసరమా?	❌ అవసరం లేదు	✅ తప్పనిసరి
Implementation	చాలా simple	jaగ్రత్త (off-by-one traps)
Random access అవసరమా?	లేదు (linked list ఓకే)	✅ అవును (array, index)
ఎప్పుడు	unsorted / small / linked list	sorted / large / monotonic
n=10^6 steps	10^6	~20

"ఎప్పుడు sort చేసి binary search చేయాలి?"

- ఒకే search అయితే: sort O(n log n) + search O(log n) > linear O(n). **సింగిల్ search కి sort worth కాదు.**
- **చాలా searches** అయితే: ఒకసారి sort O(n log n), తర్వాత ప్రతి search O(log n). q queries → O(n log n + q log n) ≪ O(nq). **Sort worth.**
- Data ఇప్పటికే sorted అయితే: binary search నేరుగా.

Gotchas (సాధారణ తప్పులు)

- **Unsorted array మీద binary search** → తప్పు/random results. ✅ ముందు sorted అని ensure చేయి.
- **Infinite loop** – `lo = mid` (mid+1 కాదు) వాడితే lo కదలదు. ✅ boundaries ఎప్పుడూ మారేలా చూడు.
- **Off-by-one** – `lo <= hi` vs `lo < hi`, `hi = length` vs `length-1` కలపడం. ✅ ఒకే template consistent గా వాడు (inclusive OR half-open – mix చేయకు).
- **Overflow** – `(lo+hi)/2`. ✅ `lo + (hi-lo)/2`.

Key Points

- **Linear O(n)** – unsorted/small కి; **Binary O(log n)** – sorted data తప్పనిసరి.
- **Binary search 3 rules:** overflow-safe mid, correct loop condition, mid±1 (infinite loop నివారణ).
- **Variants:** standard (exact), **lowerBound/upperBound** (duplicates, insert position, count), **firstTrue predicate** (binary search on answer).
- Half-open template (`lo<hi`, `hi=length`) bound-variants కి అత్యంత reliable.

- **చాలా queries → sort + binary search worth**; single query కి linear చాలు.

Interview దృష్టి

Q: Binary search off-by-one bugs ఎలా avoid చేయాలి? A: ఒకే template బట్టీ కొట్టి ప్రతిసారి అదే వాడు. నేను recommend చేసేది half-open (`lo<hi` , `hi=arr.length` , `pred(mid)→hi=mid` else `lo=mid+1`) – ఇది exact, lowerBound, firstTrue అన్నిటికీ పనిచేస్తుంది. మధ్యలో templates mix చేయకు.

Q: "Rotated sorted array లో search" ఎలా? A: ఇది binary search variant – ప్రతి step లో ఏ సగం sorted గా ఉందో గుర్తించి, target ఆ range లో ఉందా చూసి, తగిన వైపు వెళ్లాం. Still O(log n). (DSA_01 doc లో deep drill.)

Q: "Minimum capacity/speed such that..." problems ఎలా చేస్తావు? A: **Binary search on the answer.** Answer range మీద monotonic predicate ("ఈ capacity తో పని అవుతుందా?") define చేసి firstTrue వెతుకుతాం. Array search కాదు, కానీ అదే O(log(range)) technique.

---

9. The Pattern Catalog

వివరణ

Interview problems దాదాపు అన్నీ **10-12 patterns** లో ఏదో ఒకటే. కొత్త problem చూడగానే "ఇది ఏ pattern?" అని గుర్తుపట్టగలిగితే – 80% పని అయిపోయింది. ఈ topic ప్రతి pattern ని **2-3 lines + "ఎప్పుడు వాడాలి" recognition signals** తో overview చేస్తుంది. **లోతైన drills companion docs లో** (DSA_01-DSA_10) ఉంటాయి; ఇది నీ **mental index** – problem చూసి సరైన tool ఎంచుకోవడానికి.

Real-life Scenario

> **Patterns = వైద్యుడి "differential diagnosis" checklist.** అనుభవం ఉన్న డాక్టర్ symptom చూడగానే "జ్వరం + దగ్గు → ఈ 4 జబ్బుల్లో ఒకటి" అని తక్షణం narrow చేస్తాడు – ప్రతి patient ని scratch నుండి figure out చేయడు. అనుభవం ఉన్న engineer కూడా "sorted array + pair → two pointers", "subarray + condition → sliding window" అని problem యొక్క *signals* చూసి pattern ని instant గా recall చేస్తాడు. **ఈ recall speed అనేదే experience.** పేటర్న్లు బాగా తెలిస్తే, interview లో "ఆలోచించడం" కంటే "గుర్తుపట్టడం" ఎక్కువ.

The 12 Core Patterns

1. Two Pointers – రెండు indices ని array/string మీద కదిలించడం (opposite ends, లేదా same direction). Nested loop $O(n^2)$ ని O(n) కి తగ్గిస్తుంది.

> **ఎప్పుడు:** sorted array + pair/triplet కావాలి; palindrome check; array reverse; remove duplicates in-place; "two sum on sorted."

2. Sliding Window – contiguous subarray/substring మీద ఒక "window" ని slide చేయడం, ప్రతిసారి scratch నుండి కాకుండా incremental update. $O(n^2) \rightarrow O(n)$.

> **ఎప్పుడు:** "contiguous subarray/substring" + "longest/shortest/max sum/at most K" లాంటి phrases; fixed-size window average.

3. Prefix Sum – cumulative sums ముందే compute చేసి, ఏ range sum అయినా O(1) లో పొందడం.

> **ఎప్పుడు:** చాలా "range sum" queries; "subarray sum equals K" (prefix sum + hash map); 2D grid region sums.

4. Fast & Slow Pointers (Floyd's) – రెండు pointers వేర్వేరు speeds తో (ఒకటి 1 step, ఒకటి 2 steps).

> **ఎప్పుడు:** linked list లో cycle detection; middle node కనుక్కోవడం; "happy number" లాంటి cycle problems.

5. BFS (Breadth-First Search) – level-by-level traversal, **queue** వాడి. Unweighted graph లో **shortest path** guarantee.

> **ఎప్పుడు:** "shortest path/minimum steps" in unweighted graph/grid; level-order tree traversal; "nearest/closest."

6. DFS (Depth-First Search) – ఒక path ని చివరిదాకా explore చేసి, తర్వాత backtrack. **Recursion/stack.**

> **ఎప్పుడు:** "all paths"; connected components; tree traversals; cycle detection; grid "islands"; topological sort.

7. Backtracking – అన్ని candidates ని systematically try చేసి, invalid అయితే వెనక్కి వెళ్ళి (undo) వేరే option. "Choose → explore → un-choose."

> **ఎప్పుడు:** "all permutations/combinations/subsets"; N-Queens; Sudoku; word search; constraint satisfaction.

8. Dynamic Programming (DP) – overlapping subproblems + optimal substructure. Sub-results cache (memoization/tabulation) చేసి recompute నివారించడం.

> **ఎప్పుడు:** "min/max cost/ways to..."; "count of ways"; choices with overlapping subproblems; "can we reach/partition?"; sequences (LIS, edit distance, knapsack).

9. Greedy – ప్రతి step లో locally optimal choice చేసి, అది globally optimal అవుతుందని ఆశించడం (proof అవసరం).

> **ఎప్పుడు:** "maximum/minimum" + local choice global కి దారితీస్తుంది; interval scheduling; Huffman; "jump game"; coin change (కొన్ని denominations).

10. Divide & Conquer – problem ని independent sub-problems గా విడగొట్టి, solve చేసి, combine.

> **ఎప్పుడు:** merge sort/quick sort; "find in rotated/2D sorted"; majority element; closest pair; ప్రతిదీ సగం చేయగలిగే structure.

11. Heap / Top-K – priority queue (min/max heap) తో "అత్యంత చిన్న/పెద్ద K" efficiently maintain చేయడం. O(n log k).

> **ఎప్పుడు:** "top K / K largest / K closest / K frequent"; "median of stream"; merge K sorted lists; scheduling by priority.

12. Union-Find (Disjoint Set) – elements ని groups గా, fast union + connectivity check ($\alpha(n) \approx O(1)$).

> **ఎప్పుడు:** "connected components"; "number of islands (dynamic)"; cycle detection in undirected graph; Kruskal's MST; "accounts merge."

Code – pattern signal → chosen tool (mini examples)

```
// SIGNAL: "sorted array, pair with sum = target" → TWO POINTERS
function twoSumSorted(arr, target) {
  let lo = 0, hi = arr.length - 1;
  while (lo < hi) {
    const sum = arr[lo] + arr[hi];
    if (sum === target) return [lo, hi];
    else if (sum < target) lo++;      // sum చిన్నది → ఎడమ pointer కుడికి
    else hi--;                        // sum పెద్దది → కుడి pointer ఎడమకి
  }
  return [];
}
console.log(twoSumSorted([1, 3, 4, 5, 7, 11], 9));  // [2, 3]  (arr[2]+arr[3] = 4+5 =
9)

// SIGNAL: "longest contiguous subarray, sum ≤ limit" → SLIDING WINDOW
function longestSubarrayAtMost(arr, limit) {
  let left = 0, sum = 0, best = 0;
  for (let right = 0; right < arr.length; right++) {
    sum += arr[right];                // window కి కుడి element చేర్చు
    while (sum > limit) sum -= arr[left++];  // limit దాటితే ఎడమ నుండి తగ్గించు
    best = Math.max(best, right - left + 1); // valid window size
  }
  return best;
```

```
}
console.log(longestSubarrayAtMost([1, 2, 1, 0, 3], 4));  // 4 ([1,2,1,0])

// SIGNAL: "subarray sum equals K, count" → PREFIX SUM + HASH MAP
function subarraySumCount(arr, k) {
  const count = new Map([[0, 1]]);   // prefix sum 0 ఒకసారి (empty prefix)
  let sum = 0, result = 0;
  for (const x of arr) {
    sum += x;
    result += count.get(sum - k) || 0;  // sum-k ఇంతకుముందు వచ్చిందా?
    count.set(sum, (count.get(sum) || 0) + 1);
  }
  return result;
}
console.log(subarraySumCount([1, 1, 1], 2));  // 2
```

Recognition Signals – decision table (అత్యంత useful)

Problem లో ఈ signal కనిపిస్తే	ఈ pattern వాడు
"sorted array" + pair/triplet	Two Pointers / Binary Search
"contiguous subarray/substring" + longest/max/min	Sliding Window
చాలా "range sum" queries	Prefix Sum
linked list + cycle/middle	Fast & Slow Pointers
"shortest path/min steps" (unweighted)	BFS
"all paths / connected / islands"	DFS
"all permutations/combinations/subsets"	Backtracking
"min/max/count of ways" + overlapping	Dynamic Programming
"max/min" + local greedy choice	Greedy
"top K / K-th / K frequent/closest"	Heap
"connected groups / union / cycle (undirected)"	Union-Find
"sorted + search / minimize maximum"	Binary Search (on array/answer)

ఎలా master చేయాలి

1. **ప్రతి solve చేసిన problem ని ఒక pattern కి tag చేయి.** "ఇది sliding window problem" – ఇలా mental index build అవుతుంది.
2. **Pattern-wise practice** (random కాదు) – వరుసగా 8-10 sliding window problems చేస్తే, ఆ pattern permanent గా brain లో fix అవుతుంది.
3. **Signals ని బట్టీ కొట్టు** – పైన table లో phrases ని problem statements తో connect చేయడం అలవాటు చేసుకో.

4. **Companion docs వాడు** – DSA_01 (arrays/strings), DSA_02 (two pointers/sliding window), ... DSA_10 (DP) – ప్రతి pattern ని LeetCode-150 problems తో deep drill చేస్తాయి.

Key Points

- ~**12 patterns** interview problems దాదాపు అన్నిటినీ cover చేస్తాయి. Pattern గుర్తిస్తే solution 80% అయ్యింది.
- **Signals → pattern:** sorted+pair→two pointers; contiguous subarray→sliding window; shortest path→BFS; all combinations→backtracking; top-K→heap; overlapping subproblems→DP.
- ఈ topic = **mental index** (overview); **deep drills companion docs (DSA_01-DSA_10) లో.**
- **Pattern-wise practice** (random కాదు) + **ప్రతి problem ని tag చేయడం** = fastest mastery.

Interview దృష్టి

Q: కొత్త problem లో pattern గుర్తుపట్టలేకపోతే? A: Input shape + keywords చూడు (పైన signals table). ఏదీ match కాకపోతే: (1) brute force రాయి, (2) "ఎక్కడ repeated work?" అని చూడు – repeated lookups→hash map, repeated subproblems→DP, sorted structure→binary search. చాలా optimizations ఈ మూడిటిలో ఒకటే.

Q: ఒక problem కి రెండు patterns fit అయితే? A: సాధారణం! ఉదా "subarray sum = K" ని prefix-sum+hashmap తో O(n), లేదా (all positive అయితే) sliding window తో చేయవచ్చు. Constraints (negatives ఉన్నాయా?) ఏది valid అనేది decide చేస్తాయి. రెండూ mention చేసి, ఎందుకు ఒకటి ఎంచుకున్నావో చెప్పడం strong signal.

Part 3 – Data Structures FROM SCRATCH

> ఇది గుండె (heart) of DSA. ప్రతి data structure ని **మొదటి principle నుండి** – memory లో ఎలా ఉంటుంది, ఎందుకు ఏ operation ఏ complexity, ఎప్పుడు వాడాలి, JS లో ఎలా వాడాలి/implement చేయాలి – పూర్తిగా విప్పుతాం. ఇక్కడ ప్రతి structure కి **runnable JS** ఉంటుంది. ఇవి బాగా అర్థమైతే, ఏ problem కైనా సరైన tool ఎంచుకోగలవు. **Read once, never forget.**

---

10. Arrays & Dynamic Arrays

వివరణ

Array = ఒకే type elements ని **memory లో వరుసగా (contiguous)** పెట్టే data structure. ఇదే అత్యంత fundamental, అత్యంత వాడే structure.

కీలక superpower: **index access O(1).** ఎందుకు? ఎందుకంటే elements memory లో వరుసగా ఉంటాయి. కాబట్టి `arr[i]` address = `base_address + i × element_size` – ఒక్క multiplication + addition → constant time, i ఎంత పెద్దదైనా.

రెండు రకాలు:

- **Static array** – fixed size (C/Java లో). Create అయ్యేటప్పుడే size fix.
- **Dynamic array** – auto-grow (JS `Array`, Python `list`, Java `ArrayList`, C++ `vector`). Full అయితే internally పెద్దది allocate చేసి copy చేస్తుంది.

Real-life Scenario

> **Array = రైలు bogies వరుస (numbered seats).** ఒక platform మీద bogies వరుసగా ఉన్నాయి – bogie 0, 1, 2, 3... నీకు "bogie 5" కావాలంటే, మొదటి నుండి లెక్కపెట్టవు – **నేరుగా** 5వ దానికి వెళ్తావు (index O(1)), ఎందుకంటే అవి వరుసగా, సమాన దూరంలో ఉన్నాయి.
>
> కానీ **మధ్యలో కొత్త bogie చేర్చాలంటే?** bogie 2 దగ్గర కొత్తది పెట్టాలంటే, దాని వెనక ఉన్న అన్ని bogies ని ఒక్కో స్థానం **వెనక్కి జరపాలి** (shift) → O(n). అలాగే మధ్యలో ఒకటి తీసేస్తే, ఖాళీ పూడ్చడానికి మిగతావి ముందుకు జరగాలి. **అందుకే array లో index access super-fast కానీ middle insert/delete slow.**

ARRAY · వరుస MEMORY వల్ల O(1) ACCESS

MEMORY LAYOUT – వరుసగా, పక్కపక్కనే

	10	20	30	40
	0	1	2	3
address:	1000	1004	1008	1012

అందుకే arr[i] O(1)

address = base + i × elementSize – ఒక్క గుణకారం. వెతకడం లేదు.

వేగం ఎక్కడ

index తో access O(1)
చివర push/pop amortised O(1)
CPU cache friendly – పక్కపక్కనే ఉన్నాయి

నెమ్మది ఎక్కడ

మధ్యలో insert/delete O(n) – అన్నీ జరపాలి
వెతకడం O(n) (sorted అయితే O(log n))

Internal Working – memory layout

```
Index:     0    1    2    3    4
          ┌────┬────┬────┬────┬────┐
Array:    │ 10 │ 20 │ 30 │ 40 │ 50 │
          └────┴────┴────┴────┴────┘
Address: 100  104  108  112  116     (ప్రతి int 4 bytes అనుకో)

arr[3] కావాలి? → address = 100 + 3×4 = 112 → నేరుగా అక్కడికి. O(1)!
మధ్యలో index 1 దగ్గర insert? → 20,30,40,50 అన్నీ కుడికి shift → O(n).
```

Dynamic array growth (amortized O(1) push – Topic 4 గుర్తుందా): capacity full అయితే, engine **రెట్టింపు** capacity array allocate చేసి, పాత elements copy చేసి, కొత్తది add చేస్తుంది. ఈ copy అరుదు కాబట్టి, n pushes మీద average O(1).

Operations + Big-O

Operation	Big-O	ఎందుకు
Access `arr[i]`	O(1)	address math, direct jump
Update `arr[i] = x`	O(1)	direct write
Push (end)	O(1)*	*amortized (occasional resize)
Pop (end)	O(1)	చివరిది తీసేయడం, shift లేదు
Insert (start/middle)	O(n)	తర్వాతి elements అన్నీ shift
Delete (start/middle)	O(n)	ఖాళీ పూడ్చడానికి shift
Search (unsorted)	O(n)	ప్రతి element check
Search (sorted)	O(log n)	binary search

Key insight: ends O(1), middle/start O(n). ఇదే array యొక్క fundamental trade-off.

ఎప్పుడు వాడాలి

- **వాడు:** index తో fast access కావాలి; data size తెలిసి ఉంది/మారదు; ends మీద ఎక్కువ ops (stack లా); iterate/scan; sorted data + binary search.
- **వాడొద్దు (వేరేది better):** start/middle లో frequent insert/delete (→ linked list); fast key-based lookup (→ hash map); FIFO ends (→ deque/queue).

JS Array reality (ముఖ్యం!)

JS `Array` "true array" కాదు – ఇది **special object** (keys = string indices). V8 లాంటి engines దీన్ని **optimize** చేస్తాయి: elements అన్నీ same type (అన్నీ numbers) అయితే internally packed contiguous array గా store చేసి C-array speed ఇస్తాయి. కానీ holes (`arr[100]=x` on empty) లేదా mixed types వాడితే "dictionary mode" కి పడిపోయి slow అవుతుంది. **DSA లో: consistent types వాడు, holes create చేయకు.**

Common Operations – JS code

```
// ---- Creation ----
const a = [10, 20, 30];
const zeros = new Array(5).fill(0);          // [0,0,0,0,0]
const range = Array.from({ length: 5 }, (_, i) => i);  // [0,1,2,3,4]
const grid = Array.from({ length: 3 }, () => new Array(3).fill(0)); // 2D 3x3

// ---- Access & update: O(1) ----
console.log(a[1]);          // 20
a[1] = 99;                  // [10, 99, 30]

// ---- End operations: O(1) amortized ----
a.push(40);                 // add end → [10, 99, 30, 40]
a.pop();                    // remove end → 40, a = [10, 99, 30]

// ---- Start operations: O(n) (shift all!) ----
a.unshift(5);               // add start → [5, 10, 99, 30]
a.shift();                  // remove start → 5, a = [10, 99, 30]

// ---- Middle insert/delete: splice O(n) ----
a.splice(1, 0, 55);         // index 1 దగ్గర 55 insert → [10, 55, 99, 30]
a.splice(1, 1);             // index 1 దగ్గర 1 element delete → [10, 99, 30]

// ---- Search ----
console.log(a.indexOf(99)); // 1  (O(n))
console.log(a.includes(99));// true (O(n))

// ---- Iterate ----
for (let i = 0; i < a.length; i++) { /* index తో */ }
for (const x of a) { /* value తో */ }
a.forEach((x, i) => { /* value + index */ });

// ---- Transform (కొత్త array return, O(n)) ----
console.log([1, 2, 3].map(x => x * 2));        // [2, 4, 6]
console.log([1, 2, 3, 4].filter(x => x % 2));  // [1, 3]
console.log([1, 2, 3, 4].reduce((s, x) => s + x, 0)); // 10

// ---- Copy (shallow) ----
const copy1 = [...a];       // spread
```

```
const copy2 = a.slice();    // slice (no args = full copy)

// ---- 2D array access ----
grid[1][2] = 7;             // row 1, col 2
```

Static vs Dynamic Array

అంశం	Static Array	Dynamic Array (JS)
Size	fixed (create టైం)	auto-grow
Resize	సాధ్యం కాదు	double + copy (amortized O(1) push)
Memory	ఖచ్చితం	కొంచెం extra (unused capacity)
ఉదా	C `int[10]`, Java `int[]`	JS `Array`, Python `list`

Gotchas (సాధారణ తప్పులు)

- `new Array(3)` = 3 *empty slots* (length 3, values undefined/holes), కాదు `[3]`. `.fill()` తో initialize చేయి.
- **2D array తప్పు:** `new Array(3).fill(new Array(3).fill(0))` → అన్ని rows **అదే** array reference! ఒక cell మారిస్తే అన్ని rows మారతాయి. ✅ `Array.from({length:3}, () => new Array(3).fill(0))`.
- **loop లో `arr.unshift/splice`** = ప్రతిసారి O(n) → మొత్తం O(n²). ✅ end ops (push/pop) prefer చేయి.
- **`arr.includes()` in loop** → O(n²). ✅ Set వాడు.
- **Sparse arrays** (holes) engine ని slow mode కి → contiguous ఉంచు.

Key Points

- **Array = contiguous memory → index O(1)** (address math). ఇదే core superpower.
- **Ends O(1) (push/pop), start/middle O(n) (shift/splice).** ఇదే fundamental trade-off.
- **Dynamic array = auto-grow (double+copy), push amortized O(1).**
- **Search unsorted O(n), sorted O(log n) (binary search).**
- JS array = optimized object; **consistent types + no holes** ఉంచితే fast.
- **2D array trap:** ప్రతి row కి కొత్త array create చేయి (`Array.from`), fill(sharedArray) కాదు.

Interview దృష్టి

Q: Array vs hash map – ఎప్పుడు ఏది? A: Index/position తో access, order matters, iterate → array. Key-based fast lookup (O(1)), "exists?", frequency count → hash map. చాలా array problems hash map తో O(n²)→O(n) అవుతాయి.

Q: JS లో middle నుండి delete O(1) చేయవచ్చా? A: Order matter కాకపోతే – trick: ఆ element ని **last element తో swap చేసి pop** (O(1)). Order matter అయితే splice O(n) తప్పదు, లేదా linked list వాడాలి.

Q: 2D array ని ఎలా correctly create చేస్తావు? A: `Array.from({length: rows}, () => new Array(cols).fill(0))`. `fill(new Array(...))` వాడితే అన్ని rows అదే reference share చేస్తాయి – classic bug.

---

11. Strings

వివరణ

String = characters యొక్క sequence. అంతర్గతంగా ఇది **characters array లాంటిదే** – కానీ ఒక కీలక తేడా: **చాలా languages లో strings immutable (మార్చలేనివి).** JS లో కూడా strings immutable.

Immutable అంటే: ఒకసారి create అయిన string ని మార్చలేం. "మార్చినట్టు" కనిపించే ప్రతి operation (concat, replace, uppercase) నిజానికి **కొత్త string create చేస్తుంది**, పాతది అలాగే ఉంటుంది. ఇది DSA లో performance కి భారీ implication – ఎందుకంటే ఒక string ని loop లో మళ్ళీ మళ్ళీ modify చేస్తే, ప్రతిసారి కొత్త string → $O(n^2)$.

Real-life Scenario

> **String immutability = రాతితో చెక్కిన శిలాశాసనం (stone inscription).** ఒకసారి రాతిపై చెక్కాక, ఒక్క అక్షరం మార్చాలన్నా – పాత రాయిని చెరిపి కొత్తది చెక్కలేవు. **కొత్త రాయి తీసుకుని, మొత్తం మళ్ళీ చెక్కాలి.**
>
> అందుకే ఒక పెద్ద inscription కి 100 అక్షరాలు ఒక్కొక్కటిగా చేర్చాలంటే – ప్రతిసారి మొత్తం రాయి మళ్ళీ చెక్కడం భయంకరమైన వృథా ($O(n^2)$). బదులుగా **అన్ని అక్షరాలు ఒక notebook (array) లో రాసుకుని, చివర్లో ఒక్కసారే రాతిపై చెక్కడం (`array.join('')`)** తెలివైన పని. ఇదే DSA లో string building యొక్క golden rule.

Internal Working – immutability యొక్క ఖర్చు

```
// ❌ O(n²) – ప్రతి += కొత్త string create చేస్తుంది
function buildBad(n) {
  let s = "";
  for (let i = 0; i < n; i++) {
    s += "a";      // ప్రతిసారి: పాత s (length i) copy + 'a' → కొత్త string
  }                // 1+2+3+...+n = n²/2 copies → O(n²)!
  return s;
}

// ✅ O(n) – array లో push చేసి, చివర్లో ఒక్కసారే join
function buildGood(n) {
  const parts = [];
  for (let i = 0; i < n; i++) {
    parts.push("a");   // array push O(1)
  }
  return parts.join("");  // ఒక్కసారే string build → O(n)
```

```
}
console.log(buildGood(5));  // "aaaaa"
```

Golden rule: string ని loop లో బట్టీ కడుతుంటే – **array లో parts collect చేసి, చివర్లో** `.join('')`**.** ఇది DSA string problems లో అత్యంత common optimization.

Character operations

```
const s = "hello";

// ---- Access char: O(1) ----
console.log(s[0]);           // "h"
console.log(s.charAt(1));    // "e"

// ---- Char code (ASCII/Unicode): comparisons/hashing కి ----
console.log(s.charCodeAt(0));         // 104  ('h' code)
console.log(String.fromCharCode(104));// "h"

// ---- 'a' నుండి offset (frequency arrays కి classic trick) ----
const idx = "c".charCodeAt(0) - "a".charCodeAt(0);  // 2
console.log(idx);            // 2  ('a'=0, 'b'=1, 'c'=2 ... 26-letter array index)

// ---- Length: O(1) ----
console.log(s.length);       // 5
```

`charCodeAt` **trick:** lowercase letters కి frequency count చేయాలంటే, size-26 array వాడి `arr[c.charCodeAt(0) - 97]++` – hash map కంటే fast (fixed alphabet).

DSA కి ముఖ్యమైన JS string methods

```
const str = "Hello World";

// ---- Search ----
str.indexOf("World");        // 6   (లేకపోతే -1)  O(n)
str.includes("World");       // true               O(n)
str.startsWith("Hello");     // true
str.endsWith("World");       // true

// ---- Extract (కొత్త string, original మారదు) ----
str.slice(0, 5);             // "Hello"   (start, end-exclusive)
str.substring(6);            // "World"
str.slice(-5);               // "World"   (negative = చివరి నుండి)

// ---- Transform (అన్నీ కొత్త string) ----
str.toLowerCase();           // "hello world"
str.toUpperCase();           // "HELLO WORLD"
"  hi  ".trim();             // "hi"      (whitespace తీసేయడం)
str.replace("World", "JS");  // "Hello JS"  (మొదటిది మాత్రమే)
str.replaceAll("l", "L");    // "HeLLo WorLd"

// ---- Split / join (string ↔ array — DSA కి కీలకం) ----
"a,b,c".split(",");          // ["a", "b", "c"]
"hello".split("");           // ["h","e","l","l","o"]  (chars array)
["a", "b", "c"].join("-");   // "a-b-c"
```

```
"one two three".split(" ");  // ["one","two","three"]

// ---- Char array తో manipulate (immutability workaround) ----
const chars = "hello".split("");  // array → mutable!
chars[0] = "H";                   // ఇప్పుడు మార్చవచ్చు
console.log(chars.join(""));      // "Hello"

// ---- Repeat, padding ----
"ab".repeat(3);              // "ababab"
"5".padStart(3, "0");        // "005"
```

String ని modify చేయడం (immutable workaround)

String మార్చాలంటే: **array గా convert → modify → join back.**

```
// String reverse — chars array తో
function reverseString(s) {
  return s.split("").reverse().join("");
}
console.log(reverseString("hello"));  // "olleh"

// ఒక char మార్చడం
function replaceCharAt(s, i, ch) {
  const arr = s.split("");   // mutable array
  arr[i] = ch;
  return arr.join("");
}
console.log(replaceCharAt("hello", 0, "H"));  // "Hello"
```

Common DSA string patterns

```
// 1. Palindrome check — two pointers, O(n) time O(1) space
function isPalindrome(s) {
  let l = 0, r = s.length - 1;
  while (l < r) {
    if (s[l] !== s[r]) return false;
    l++; r--;
  }
  return true;
}
console.log(isPalindrome("racecar"));  // true

// 2. Character frequency — hash map లేదా size-26 array
function charFreq(s) {
  const freq = new Map();
  for (const ch of s) freq.set(ch, (freq.get(ch) || 0) + 1);
  return freq;
}
console.log(charFreq("aabbc"));  // Map { a→2, b→2, c→1 }

// 3. Anagram check — frequency compare
function isAnagram(a, b) {
  if (a.length !== b.length) return false;
  const count = new Array(26).fill(0);
  for (let i = 0; i < a.length; i++) {
    count[a.charCodeAt(i) - 97]++;   // a లో ఉన్నవి +
```

```
    count[b.charCodeAt(i) - 97]--;   // b లో ఉన్నవి -
  }
  return count.every(c => c === 0);  // అన్నీ 0 → same chars
}
console.log(isAnagram("listen", "silent"));  // true
```

String Operations – Big-O

Operation	Big-O	గమనిక
Access `s[i]`, `length`	O(1)	
Concatenation `s + t`	O(n+m)	కొత్త string create
`+=` in loop	**O(n²)** ⚠️	ప్రతిసారి copy – array+join వాడు
`slice/substring`	O(k)	k = substring length
`indexOf/includes`	O(n·m)	naive substring search
`split/join`	O(n)	
Compare `s === t`	O(n)	char-by-char

Key Points

- **Strings immutable** – ప్రతి "modify" కొత్త string create చేస్తుంది.
- **Golden rule:** loop లో string build చేస్తే **array + `.join('')`** (O(n)), `+=` కాదు (O(n²)).
- **Modify workaround:** `split('')` → array modify → `join('')`.
- **`charCodeAt(c) - 97`** = size-26 frequency array trick (fast, fixed alphabet).
- Common patterns: palindrome (two pointers), anagram/frequency (count array), substring (sliding window).

Interview దృష్టి

Q: JS లో string ఎందుకు index తో మార్చలేం (`s[0]='H'` పనిచేయదు)? A: Strings immutable. `s[0]='H'` silently fail (strict mode లో error). మార్చాలంటే `split('')` తో array గా మార్చి, modify చేసి, `join('')`. ఇది భాష design – immutability వల్ల strings safely share/cache అవుతాయి.

Q: పెద్ద string build చేస్తున్నప్పుడు TLE వస్తోంది – ఎందుకు? A: `s += ...` loop లో O(n²) (ప్రతిసారి కొత్త string copy). Fix: array లో parts push చేసి, చివర్లో ఒక్కసారే `.join('')` → O(n). ఇది string problems లో అత్యంత common performance bug.

Q: Frequency count కి hash map vs array-26 – ఏది? A: input lowercase English letters మాత్రమే అని guarantee ఉంటే size-26 array (fast, O(1) space, cache-friendly). Unicode/arbitrary chars అయితే hash map. Interview లో constraint అడిగి decide చేయి.

12. Hash Map & Hash Set

వివరణ

Hash Map (dictionary/hash table) = **key** → **value** pairs ని store చేసి, key తో **O(1) average** లో value ని పొందే structure. **Hash Set** = keys మాత్రమే (values లేవు) – "ఈ element ఉందా?" O(1) లో చెప్తుంది.

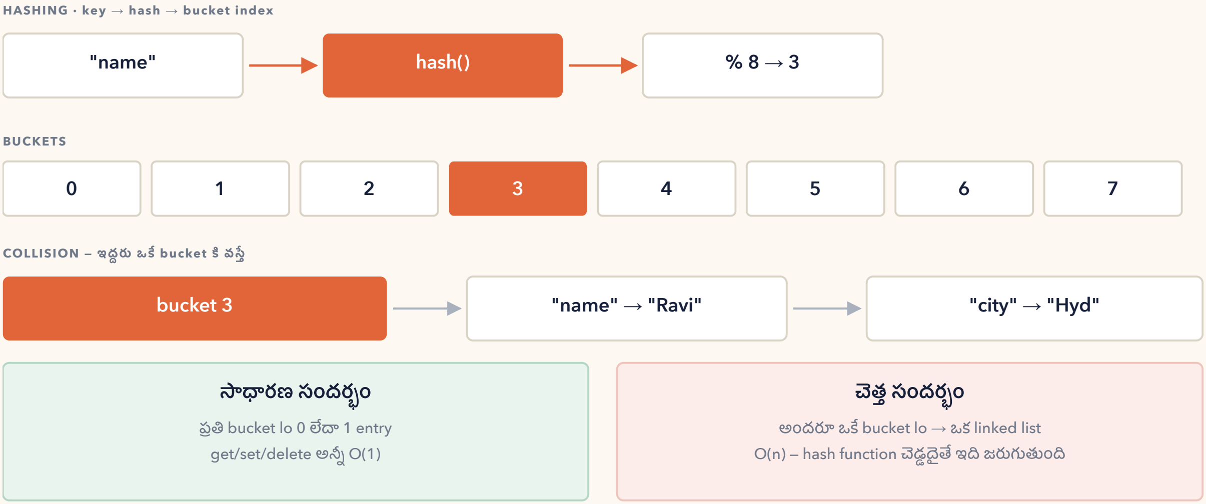
ఇది DSA లో **అత్యంత powerful, అత్యంత వాడే** structure. చాలా $O(n^2)$ brute-force solutions ని O(n) కి తగ్గించే magic wand. "seen before?", "frequency?", "complement exists?" – ఇవన్నీ hash map territory.

Core idea: ఒక **hash function** key ని ఒక array index గా convert చేస్తుంది. కాబట్టి key ఇస్తే, నేరుగా ఆ index కి వెళ్ళి value తీయవచ్చు – searching లేదు, direct jump. అదే O(1).

Real-life Scenario

> **Hash Map = grand library లో "call number" system.** లక్షల పుస్తకాలున్న library లో ఒక పుస్తకం కావాలంటే, shelf shelf కి వెతకవు (అది O(n)). బదులుగా catalog లో పుస్తకం పేరు (**key**) ఇస్తే, ఒక **call number** (ఉదా "QA76.73") వస్తుంది (**hash function**) – అది నేరుగా ఏ floor, ఏ shelf అని చెప్తుంది (**index**). నువ్వు నేరుగా అక్కడికి వెళ్ళి పుస్తకం తీస్తావు – **O(1).**
>
> **Collision అంటే?** ఒకవేళ రెండు పుస్తకాలకి అదే call number వస్తే? ఆ shelf దగ్గర ఆ రెండూ (లేదా కొన్ని) పక్కపక్కన పెడతారు (**chaining**). నువ్వు అక్కడికి వెళ్ళి, ఆ చిన్న గుంపులో సరైనది తీస్తావు. Collisions అరుదుగా, తక్కువగా ఉంటే – ఇప్పటికీ దాదాపు O(1). చాలా పుస్తకాలకి అదే number వస్తే (చెత్త system) → అది O(n) కి దిగజారుతుంది. అందుకే **మంచి hash function** ముఖ్యం.

HASH MAP · HASHING, BUCKETS, COLLISIONS



Load factor 0.75 దాటితే – buckets ని రెట్టింపు చేసి అన్నీ మళ్ళీ hash చేయడం (rehash)

Interview lo అడిగే ప్రశ్న: "O(1) ఎప్పుడూ నిజమేనా?" – కాదు. అది సగటు మాత్రమే. చెడ్డ hash function లేదా ఉద్దేశపూర్వక దాడి (hash flooding) తో O(n) కి దిగజారొచ్చు. అందుకే Java 8+ lo పెద్ద buckets ని red-black tree గా మారుస్తారు – O(log n) హామీ.

Internal Working – hashing + collisions

```
Key "apple" → hash function → 15432 → % 8 (table size) → index 0
Key "banana"→ hash function → 78219 → % 8 → index 3
Key "cherry"→ hash function → 41008 → % 8 → index 0  ← COLLISION with apple!

Table (chaining తో):
  index 0: [("apple", 5), ("cherry", 9)]   ← అదే bucket లో chain (linked list)
  index 1: []
  index 2: []
  index 3: [("banana", 3)]
  ...
```

Steps: (1) key ని hash function integer గా మారుస్తుంది, (2) `% tableSize` తో bucket index వస్తుంది, (3) ఆ bucket లో store/lookup.

Collision = రెండు keys అదే bucket కి. పరిష్కారాలు:

- **Chaining** – ప్రతి bucket ఒక list; collisions అదే list లో append (JS engines typically ఇది వాడతాయి).
- **Open addressing** – collision అయితే తర్వాతి ఖాళీ slot వెతకడం (linear/quadratic probing).

Load factor & resizing: entries/buckets ratio (load factor) threshold దాటితే (ఉదా 0.75), table **రెట్టింపు size** కి grow అయ్యి అన్ని keys **rehash** అవుతాయి → O(n) ఒక్కసారి, కానీ amortized O(1) (Topic 4).

ఎందుకు O(1) average, O(n) worst: మంచి hash function → collisions తక్కువ → దాదాపు direct access (O(1)). చెత్త function/adversarial keys → అన్నీ ఒకే bucket → linked list scan → O(n). Practice లో average O(1).

Operations + Big-O

Operation	Average	Worst	గమనిక
Insert / set	O(1)	O(n)	worst = అన్నీ collide + resize
Lookup / get	O(1)	O(n)	
Delete	O(1)	O(n)	
Has / contains	O(1)	O(n)	
Iterate all	O(n)	O(n)	

Space: O(n). **Trade-off:** fast lookup కోసం extra memory (buckets + entries).

ఎప్పుడు వాడాలి (recognition signals)

- **"seen before?" / "duplicate?"** → hash set.
- **"frequency / count of each"** → hash map (value → count).

- **"complement / pair with sum"** (Two Sum) → hash map (value → index).
- **"group by something"** (anagrams) → hash map (key → list).
- **fast lookup by key** (cache, index) → hash map.
- **వాడొద్దు:** order/sorting కావాలంటే (hash unordered – JS Map insertion order ఇస్తుంది కానీ sorted కాదు); range queries.

JS లో 3 options: Object vs Map vs Set

```
// ---- 1. Object {} — simple string/symbol keys ----
const obj = {};
obj["apple"] = 5;
obj.banana = 3;
console.log(obj["apple"]);        // 5
console.log("apple" in obj);      // true
delete obj.banana;
for (const key in obj) { /* keys */ }
// ⚠️ keys ఎప్పుడూ strings (obj[1] === obj["1"]); prototype keys risk

// ---- 2. Map — any-type keys, size, ordered, cleaner API (DSA కి BEST) ----
const map = new Map();
map.set("apple", 5);
map.set(42, "answer");            // key ఏ type అయినా (number, object కూడా!)
console.log(map.get("apple"));    // 5
console.log(map.has(42));         // true
console.log(map.size);            // 2   (O(1))
map.delete("apple");
for (const [k, v] of map) { /* insertion order లో */ }

// ---- 3. Set — unique values మాత్రమే ("exists?") ----
const set = new Set();
set.add(10);
set.add(10);                      // duplicate ignore
console.log(set.has(10));         // true
console.log(set.size);            // 1
set.delete(10);
const uniq = [...new Set([1, 2, 2, 3, 3, 3])];  // [1, 2, 3] — dedup trick!
```

Object vs Map – ఏది ఎప్పుడు

అంశం	Object `{}`	Map
Key types	strings/symbols మాత్రమే	**ఏదైనా** (object, number, string)
Size	`Object.keys(o).length` O(n)	`map.size` O(1)
Order	modern engines insertion (కానీ integer keys sorted!)	**guaranteed insertion order**
Iteration	`for...in`, `Object.entries`	directly iterable
Prototype pollution	risk (`obj.toString`)	లేదు (clean)

అంశం	Object `{}`	Map
Performance (frequent add/delete)	slower	**optimized**
DSA లో	JSON-like fixed data	**default choice**

DSA లో recommendation: Map & Set వాడు (cleaner, safer, O(1) size, any-key). Object ని fixed/known-string-keys కి మాత్రమే.

Code – hash map తో O(n²) → O(n)

```
// Two Sum — hash map తో single pass O(n)
function twoSum(nums, target) {
  const seen = new Map();               // value → index
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (seen.has(need)) return [seen.get(need), i];
    seen.set(nums[i], i);
  }
  return [];
}
console.log(twoSum([2, 7, 11, 15], 9));  // [0, 1]

// Frequency count — hash map
function topFreq(arr) {
  const freq = new Map();
  for (const x of arr) freq.set(x, (freq.get(x) || 0) + 1);
  return freq;
}
console.log(topFreq([1, 1, 2, 3, 3, 3]));  // Map { 1→2, 2→1, 3→3 }

// Group anagrams — hash map (sorted-key → list)
function groupAnagrams(words) {
  const groups = new Map();
  for (const w of words) {
    const key = w.split("").sort().join("");  // "eat"→"aet", "tea"→"aet"
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
  }
  return [...groups.values()];
}
console.log(groupAnagrams(["eat", "tea", "tan", "ate", "nat"]));
// [["eat","tea","ate"], ["tan","nat"]]

// Dedup + membership
console.log([...new Set([3, 1, 2, 3, 1])]);  // [3, 1, 2]
```

Gotchas (సాధారణ తప్పులు)

- **Object integer keys sorted** – `{2:'a', 1:'b'}` iteration `1` ముందు `2`! insertion order కావాలంటే Map.
- **Object keys always strings** – `obj[1]` and `obj["1"]` same. Number keys distinct కావాలంటే Map.

- **Prototype keys** – `obj["toString"]` inherited property తో clash. ✅ `Object.create(null)` లేదా Map.
- **Object/array ని key గా Object లో** – `[1,2]` key అయితే `"1,2"` string అవుతుంది (lossy). Map ఇలాంటి keys reference గా handle చేస్తుంది.
- `freq.get(x) + 1` when x absent → `undefined + 1 = NaN`. ✅ `(freq.get(x) || 0) + 1`.

Key Points

- **Hash map/set = O(1) average** lookup/insert/delete – DSA లో అత్యంత powerful tool.
- **Hashing:** key → hash → index; **collisions** chaining/open addressing తో; load factor దాటితే resize+rehash.
- **O(1) average, O(n) worst** (అన్నీ collide) – practice లో average O(1) treat చేయి.
- **JS: Map & Set prefer** (any-key, O(1) size, insertion order, no prototype issue); Object fixed-string-keys కి.
- Signals: "seen?/duplicate?" → Set; "frequency" → Map; "complement/pair" → Map; "group by" → Map of lists.

Interview దృష్టి

Q: Hash map ఎలా O(1) ఇస్తుంది? A: Hash function key ని array index గా మారుస్తుంది → direct access, searching లేదు. Collisions (అదే index) chaining తో handle. మంచి function + resizing వల్ల collisions అరుదు → average O(1). Adversarial worst case O(n) కానీ practical గా O(1).

Q: Object vs Map – interview లో ఏది వాడతావు? A: Map – cleaner API, any-type keys, O(1) `.size`, guaranteed insertion order, prototype pollution లేదు. Object ని JSON-like fixed data కి మాత్రమే. Number keys/frequent updates ఉంటే Map తప్పనిసరి.

Q: Hash map "worst case O(n)" – production లో problem కాదా? A: మంచి randomized hash function ఉంటే adversary predict చేయలేడు, collisions balanced. Java 8+ లాంటివి bucket పెద్దదైతే linked list ని balanced tree (O(log n)) గా మారుస్తాయి – worst case bound మెరుగుపరచడానికి. Interview లో average O(1) చెప్పి, worst case aware గా ఉండటం చాలు.

---

13. Linked List

వివరణ

Linked List = **nodes** యొక్క sequence, ఇక్కడ ప్రతి node రెండు భాగాలు కలిగి ఉంటుంది: (1) **data** (value), (2) **pointer** (తర్వాతి node యొక్క reference/address). Array లా contiguous memory లో ఉండవు – nodes memory లో **ఎక్కడైనా** చెల్లాచెదురుగా ఉండవచ్చు, pointers వాటిని కలుపుతాయి.

రకాలు:

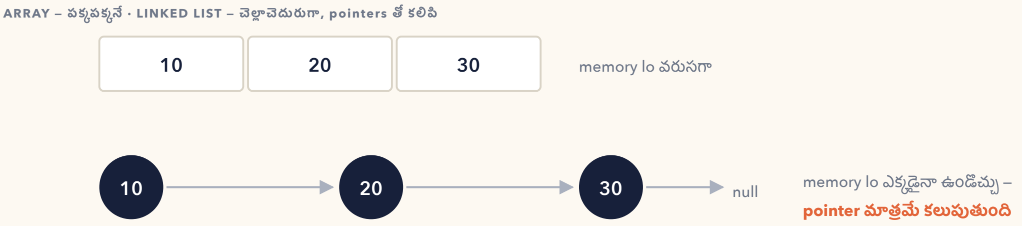
- **Singly linked list** – ప్రతి node **తర్వాతి** (next) ని మాత్రమే చూపిస్తుంది. ఒక్క దిశ.
- **Doubly linked list** – ప్రతి node **తర్వాతి + ముందటి** (next + prev) చూపిస్తుంది. రెండు దిశలు.

Array తో పోలిస్తే fundamental తేడా: **linked list కి index access లేదు** (O(n) – traverse చేయాలి), కానీ **head దగ్గర insert/delete O(1)** (shift అవసరం లేదు – కేవలం pointers మార్చడం).

Real-life Scenario

> **Linked List = నిధి వేట (treasure hunt) chits.** నీకు మొదటి chit (head) దొరుకుతుంది. దానిపై: "బహుమతి: ₹10. తర్వాతి clue మామిడి చెట్టు కింద." మామిడి చెట్టు దగ్గరికి వెళ్తే మరో chit: "బహుమతి: ₹20. తర్వాతి clue బావి దగ్గర"... ఇలా ప్రతి chit **తర్వాతిదాన్ని ఎక్కడ వెతకాలో** చెప్తుంది (pointer). చివరి chit: "ఇదే చివరిది" (**next = null**).
>
> **కీలకం:** నీకు "5వ chit" నేరుగా కావాలంటే? సాధ్యం కాదు – మొదటి నుండి ఒక్కో chit follow అవ్వాల్సిందే (**O(n) access**). కానీ మధ్యలో కొత్త chit **చేర్చడం** చాలా సులభం – ఒక chit మీద address మార్చి, కొత్త chit వైపు చూపిస్తే చాలు (**O(1) insert** – వెనక ఉన్నవాటిని కదపనవసరం లేదు). ఇదే array కి opposite behavior.

LINKED LIST VS ARRAY · POINTERS VS వరుస MEMORY



Linked list గెలిచేది	Array గెలిచేది
మొదట్లో insert/delete O(1) – array lo O(n)	index తో access O(1) – list lo O(n)
పరిమాణం ముందే తెలియనవసరం లేదు	CPU cache – array చాలా వేగం
Resize కోసం copy చేయడం అవసరం లేదు	Pointer కి అదనపు memory (node కి 8 bytes)

నిజం: ఆధునిక CPUs మీద cache locality వల్ల array చాలా సందర్భాల్లో గెలుస్తుంది – Big-O అంతా చెప్పదు

Internal Working – memory layout

```
Array (contiguous):        [10][20][30][40]     ← వరుసగా, index math పనిచేస్తుంది
                            100 104 108 112

Linked List (scattered):
   head → [10 | •]───→ [20 | •]───→ [30 | •]───→ [40 | null]
          addr 500      addr 812     addr 208     addr 640
          (data|next)   (memory లో చెల్లాచెదురు, pointers కలుపుతాయి)

"3వ node" కావాలి? → head నుండి next, next, next follow → O(n).
head దగ్గర insert? → కొత్త node.next = head; head = కొత్త node → O(1)!
```

Operations + Big-O

Operation	Singly	గమనిక
Access / search by index	O(n)	traverse from head (index access లేదు)

Operation	Singly	గమనిక
Insert at head	O(1)	pointer మార్పు మాత్రమే
Delete at head	O(1)	`head = head.next`
Insert/delete at tail	O(n)†	tail కి traverse (†tail pointer ఉంటే O(1))
Insert/delete after given node	O(1)	node reference ఉంటే
Search by value	O(n)	

Space: O(n) + ప్రతి node కి extra pointer memory.

Array vs Linked List

అంశం	Array	Linked List
Memory	contiguous	scattered (pointers)
Index access	**O(1)** ✅	O(n)
Head insert/delete	O(n) (shift)	**O(1)** ✅
Tail insert	O(1) amortized	O(1) (tail pointer తో)
Middle insert (node తెలిస్తే)	O(n)	**O(1)** ✅
Extra memory	తక్కువ	ప్రతి node కి pointer(s)
Cache locality	**మంచిది** ✅	చెడ్డది (scattered)
Size	fixed/dynamic	fully dynamic

సారాంశం: fast index access + iteration → array. Frequent head/middle insert-delete + unknown size → linked list. **Practice లో array ఎక్కువగా గెలుస్తుంది** (cache locality), కానీ linked list = interview favorite (pointer manipulation skill test).

ఎప్పుడు వాడాలి

- **వాడు:** frequent insert/delete at ends/middle (node reference తో); size ఊహించలేని dynamic data; stack/queue implementation; LRU cache (doubly); index access అవసరం లేదు.
- **వాడొద్దు:** random/index access కావాలి (→ array); memory tight (pointers overhead); cache performance ముఖ్యం.

JS Implementation

```
// Node — data + next pointer
class ListNode {
  constructor(val, next = null) {
    this.val = val;
```

```
    this.next = next;
  }
}

class LinkedList {
  constructor() {
    this.head = null;
    this.size = 0;
  }

  // Head దగ్గర add — O(1)
  prepend(val) {
    this.head = new ListNode(val, this.head);  // కొత్తది → పాత head వైపు
    this.size++;
    return this;
  }

  // Tail దగ్గర add — O(n) (tail pointer లేకపోతే)
  append(val) {
    const node = new ListNode(val);
    if (!this.head) {
      this.head = node;
    } else {
      let cur = this.head;
      while (cur.next) cur = cur.next;   // చివరికి traverse
      cur.next = node;
    }
    this.size++;
    return this;
  }

  // Value వెతకడం — O(n)
  find(val) {
    let cur = this.head;
    while (cur) {
      if (cur.val === val) return cur;
      cur = cur.next;
    }
    return null;
  }

  // Value ని delete — O(n)
  delete(val) {
    if (!this.head) return;
    if (this.head.val === val) {         // head అయితే special case
      this.head = this.head.next;
      this.size--;
      return;
    }
    let cur = this.head;
    while (cur.next && cur.next.val !== val) cur = cur.next;
    if (cur.next) {                          // దొరికింది → skip (bypass) చేయి
      cur.next = cur.next.next;
      this.size--;
    }
  }

  toArray() {
    const out = [];
```

```
    let cur = this.head;
    while (cur) { out.push(cur.val); cur = cur.next; }
    return out;
  }
}

const ll = new LinkedList();
ll.append(1).append(2).append(3);
ll.prepend(0);
console.log(ll.toArray());   // [0, 1, 2, 3]
ll.delete(2);
console.log(ll.toArray());   // [0, 1, 3]
```

Common ops – Traverse & Reverse (interview favorites)

```
// Traverse — O(n)
function traverse(head) {
  let cur = head;
  while (cur) {
    // cur.val తో పని చేయి
    cur = cur.next;   // ముందుకి కదులు
  }
}

// REVERSE a linked list — THE classic interview question, O(n) time O(1) space
function reverse(head) {
  let prev = null;         // reversed భాగం యొక్క head
  let cur = head;
  while (cur) {
    const next = cur.next; // (1) తర్వాతిది save (link తెంచేముందు)
    cur.next = prev;       // (2) pointer వెనక్కి తిప్పు
    prev = cur;            // (3) prev ముందుకి
    cur = next;            // (4) cur ముందుకి
  }
  return prev;             // కొత్త head (పాత tail)
}
// విజువల్: null←1←2←3←4  (ప్రతి arrow reverse అయ్యింది)

function toArr(head) { const o = []; let c = head; while (c) { o.push(c.val); c =
c.next; } return o; }
let h = new ListNode(1, new ListNode(2, new ListNode(3, new ListNode(4))));
console.log(toArr(reverse(h)));   // [4, 3, 2, 1]

// Find MIDDLE — fast & slow pointers, O(n) single pass
function findMiddle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;         // 1 step
    fast = fast.next.next;    // 2 steps → fast చివరికి చేరితే slow మధ్యలో
  }
  return slow;                // middle node
}

// Detect CYCLE — Floyd's algorithm, O(n) time O(1) space
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
```

```
      slow = slow.next;
      fast = fast.next.next;
      if (slow === fast) return true;    // కలిస్తే cycle ఉంది
    }
    return false;                        // fast null చేరింది → cycle లేదు
  }
```

Doubly Linked List (brief)

ప్రతి node కి `prev` కూడా → రెండు దిశలా traverse, ఇచ్చిన node ని O(1) లో delete (prev తెలుసు కాబట్టి). Trade-off: ప్రతి node కి extra pointer + మరింత bookkeeping. **LRU cache** implementation కి perfect (O(1) లో ఏ node నైనా తీసి, ముందుకి తేగలం).

Gotchas (సాధారణ తప్పులు)

- **Reverse లో `next` save చేయకపోవడం** – `cur.next = prev` చేసేముందు `next` save చేయకపోతే మిగతా list కోల్పోతావు. ✅ ముందు save.
- **Null pointer** – `cur.next.val` before checking `cur.next` exists → crash. ✅ ఎప్పుడూ null check.
- **Head edge case** – head ని delete/insert చేసేటప్పుడు separate handling అవసరం. ✅ **dummy/sentinel node** వాడితే edge cases తగ్గుతాయి.
- **Cycle detection లో `fast.next` check** – `fast.next.next` కి ముందు `fast && fast.next`. ✅ రెండూ check.

Key Points

- **Node = data + next pointer.** Nodes scattered; pointers కలుపుతాయి (array లా contiguous కాదు).
- **Index access O(n)** (no random access), కానీ **head/known-node insert/delete O(1).** Array కి opposite.
- **Reverse (prev/cur/next 3-pointer dance), find middle & cycle (fast/slow)** – top interview questions.
- **Doubly** = రెండు దిశలు, O(1) node delete; LRU cache కి ideal.
- **Dummy node** edge cases (head insert/delete) ని simplify చేస్తుంది.
- Array vs LL: **cache locality → array గెలుస్తుంది** practice లో; LL = pointer skill test.

Interview దృష్టి

Q: Linked list reverse ఎలా? (అత్యంత common) A: 3 pointers – prev(null), cur(head), next. Loop: next save → `cur.next = prev` → prev/cur ముందుకి. చివర్లో prev కొత్త head. O(n) time, O(1) space. Recursively కూడా చేయవచ్చు కానీ O(n) stack space.

Q: Cycle ఉందా ఎలా detect చేస్తావు, O(1) space లో? A: Floyd's fast & slow – slow 1 step, fast 2 steps. Cycle ఉంటే fast slow ని catch చేస్తుంది (వృత్తంలో పరిగెడితే వేగంగా వెళ్ళేవాడు నెమ్మదిగా వెళ్ళేవాడిని lap చేస్తాడు). null చేరితే cycle లేదు.

Q: Array కి బదులు linked list ఎప్పుడు? A: Head/middle లో frequent insert/delete (node reference తో O(1)), size unpredictable, random access అవసరం లేదు. కానీ cache locality వల్ల practice లో array తరచూ better – interview లో ఈ nuance చెప్పడం maturity చూపిస్తుంది.

---

14. Stack

వివరణ

Stack = **LIFO** (Last In, First Out) principle మీద పనిచేసే data structure. అంటే **చివరిగా పెట్టినది మొదట బయటికి** వస్తుంది. అన్ని operations ఒకే చివర ("**top**") జరుగుతాయి.

రెండు ముఖ్య operations (రెండూ O(1)):

- **push(x)** – top మీద element పెట్టడం.
- **pop()** – top నుండి element తీయడం (return + remove).
- **peek/top()** – top element చూడటం (remove చేయకుండా).

Stack ఒక *concept* (behavior), specific structure కాదు – array తో లేదా linked list తో implement చేయవచ్చు. JS లో array యొక్క `push` / `pop` నేరుగా stack.

Real-life Scenario

> **Stack = ప్లేట్ల గుట్ట (stack of plates) canteen లో.** కడిగిన ప్లేట్లు ఒకదానిపై ఒకటి పెడతారు. కొత్త ప్లేట్ ఎప్పుడూ **పైన** పెడతారు (push), తీసేటప్పుడు కూడా **పైనుంచే** తీస్తారు (pop). అడుగున ఉన్న ప్లేట్ (మొదట పెట్టింది) **చివరిగా** బయటికి వస్తుంది. మధ్యలో నుండి లాగడం కుదరదు – కేవలం top.
>
> మరో perfect analogy: **browser యొక్క Back button.** నువ్వు page A → B → C → D visit చేశావు. "Back" నొక్కితే? చివరిగా వచ్చిన D నుండి వెనక్కి C కి. మళ్ళీ Back → B. **చివరిగా చూసినది మొదట వెనక్కి** – pure LIFO. అలాగే **Ctrl+Z (undo)** – చివరి action మొదట undo.

Internal Working

```
push(1), push(2), push(3):        pop() → 3 తీసుకుంటుంది:

  top → │ 3 │  ← చివరిగా వచ్చింది     top → │ 2 │
        │ 2 │     (మొదట పోతుంది)             │ 1 │
        │ 1 │  ← మొదట వచ్చింది
        └───┘     (చివర పోతుంది)

అన్ని action ఒకే చివర (top). Array తో: push = end కి add, pop = end నుండి remove — రెండూ
O(1).
```

Operations + Big-O

Operation	Big-O	గమనిక
push (add top)	O(1)	array end కి add (amortized)
pop (remove top)	O(1)	array end నుండి remove
peek/top	O(1)	చివరి element చూడటం
isEmpty / size	O(1)	
search (middle)	O(n)	stack ఇందుకు కాదు

Space: O(n).

ఎప్పుడు వాడాలి (recognition signals)

- **"matching / balanced" — parentheses, brackets, tags** (ప్రతి open కి close).
- **"undo / redo / history / back"** — reverse chronological.
- **"nearest / next greater/smaller element"** — monotonic stack.
- **DFS iterative** — recursion ని explicit stack తో replace.
- **Expression evaluation** — infix/postfix, calculator.
- **Function call stack** — recursion యొక్క underlying mechanism (Topic 6).
- **"reverse" ఏదైనా** — stack లో పెట్టి తీస్తే reverse.

JS Implementation

```
// JS array నేరుగా stack — push/pop end మీద O(1)
const stack = [];
stack.push(1);      // [1]
stack.push(2);      // [1, 2]
stack.push(3);      // [1, 2, 3]
console.log(stack[stack.length - 1]);  // 3   (peek/top)
console.log(stack.pop());              // 3   (remove top) → [1, 2]
console.log(stack.length === 0);       // false (isEmpty check)

// Clean wrapper class (interview clarity కోసం)
class Stack {
  constructor() { this.items = []; }
  push(x) { this.items.push(x); }
  pop() { return this.items.pop(); }          // ఖాళీ అయితే undefined
  peek() { return this.items[this.items.length - 1]; }
  isEmpty() { return this.items.length === 0; }
  size() { return this.items.length; }
}
```

⚠️ **జాగ్రత్త:** JS లో stack కి `shift()` / `unshift()` (array start ops) వాడకు — అవి O(n)! Stack కి ఎప్పుడూ `push` / `pop` (end) — O(1).

Use case 1: Balanced Parentheses (classic)

```
// "()[]{}" valid? ప్రతి open bracket కి సరైన close ఉందా?
function isValid(s) {
  const stack = [];
  const pairs = { ")": "(", "]": "[", "}": "{" };  // close → matching open
  for (const ch of s) {
    if (ch === "(" || ch === "[" || ch === "{") {
      stack.push(ch);                  // open → push
    } else {
      // close → top తో match అవుతుందా?
      if (stack.pop() !== pairs[ch]) return false;
    }
  }
  return stack.length === 0;          // అన్ని open లు close అయ్యాయా?
}
console.log(isValid("()[]{}"));       // true
console.log(isValid("([)]"));         // false  (order తప్పు)
console.log(isValid("(]"));           // false
console.log(isValid("((("));          // false  (close కాలేదు)
```

ఎందుకు stack? చివరిగా తెరిచిన bracket **మొదట** close అవ్వాలి – pure LIFO.

Use case 2: Monotonic Stack – Next Greater Element

```
// ప్రతి element కి కుడివైపు మొదటి పెద్ద element (లేకపోతే -1). O(n)!
function nextGreater(nums) {
  const result = new Array(nums.length).fill(-1);
  const stack = [];   // indices – decreasing values (monotonic)
  for (let i = 0; i < nums.length; i++) {
    // current, stack top కంటే పెద్దదైతే – top కి "next greater" ఇదే
    while (stack.length && nums[i] > nums[stack[stack.length - 1]]) {
      result[stack.pop()] = nums[i];
    }
    stack.push(i);    // current index ని push
  }
  return result;
}
console.log(nextGreater([2, 1, 3, 5, 4]));  // [3, 3, 5, -1, -1]
// నిష్ఠగా O(n): ప్రతి index ఒకసారి push, ఒకసారి pop.
```

Use case 3: Iterative DFS (recursion → stack)

```
// Tree/graph ని recursion లేకుండా explicit stack తో traverse
function dfsIterative(root) {
  if (!root) return [];
  const result = [], stack = [root];
  while (stack.length) {
    const node = stack.pop();          // top తీసుకో
    result.push(node.val);
    // children ని push (right ముందు → left ముందు pop అవుతుంది)
    if (node.right) stack.push(node.right);
    if (node.left) stack.push(node.left);
  }
```

```
  return result;
}
// recursion యొక్క call stack ని మనమే explicit గా manage చేస్తున్నాం.
```

Gotchas (సాధారణ తప్పులు)

- `shift()` / `unshift()` **వాడటం** stack కి → O(n)! ✅ push/pop (end) మాత్రమే – O(1).
- **ఖాళీ stack pop** → JS లో `undefined` (crash కాదు, కానీ logic bug). ✅ pop ముందు `isEmpty` check.
- **peek vs pop confusion** – peek remove చేయదు, pop చేస్తుంది.
- **Balanced brackets లో** చివర్లో `stack.length === 0` check మర్చిపోవడం → `"((("` ని valid అనేస్తుంది. ✅ చివర్లో empty ఉండాలి.

Key Points

- **Stack = LIFO** (Last In First Out); అన్ని ops **top** మీద – push/pop/peek అన్నీ **O(1).**
- **JS: array యొక్క `push` / `pop`** నేరుగా stack. `shift` / `unshift` వాడకు (O(n))!
- Signals: **matching/balanced, undo/history/back, next greater (monotonic), iterative DFS, reverse, expression eval.**
- **Recursion యొక్క underlying mechanism = call stack** – recursion ని explicit stack తో iterative గా రాయవచ్చు.
- **Monotonic stack** = "next greater/smaller" problems ని O(n) కి తగ్గించే powerful technique.

Interview దృష్టి

Q: Stack ఎప్పుడు గుర్తుపట్టాలి? A: "matching/balanced brackets", "undo/back", "nearest/next greater element", "reverse", "evaluate expression" – ఇవి కనిపిస్తే stack. అలాగే recursion ని iterative గా మార్చాలంటే explicit stack.

Q: JS లో stack ఎలా implement చేస్తావు? A: Array + `push` / `pop` (రెండూ O(1), array end మీద). `shift` / `unshift` (start) O(n) కాబట్టి stack కి వాడకూడదు. Clarity కోసం చిన్న wrapper class రాయవచ్చు.

Q: Monotonic stack అంటే? A: Stack లో elements ఎప్పుడూ increasing (లేదా decreasing) order లో ఉంచేలా maintain చేయడం – కొత్త element ఆ order break చేస్తే, break అయ్యేవాటిని pop చేస్తూ answer compute చేయడం. "Next greater element", "daily temperatures", "largest rectangle in histogram" – ఇవన్నీ O(n) కి తగ్గుతాయి.

---

15. Queue & Deque

వివరణ

Queue = **FIFO** (First In, First Out) principle – **మొదట పెట్టినది మొదట బయటికి.** Stack కి సరిగ్గా వ్యతిరేకం. రెండు వేర్వేరు చివర్లలో operations:

- **enqueue(x)** – **వెనక** (rear/back) element చేర్చడం.

- **dequeue()** – **ముందు** (front) నుండి element తీయడం.

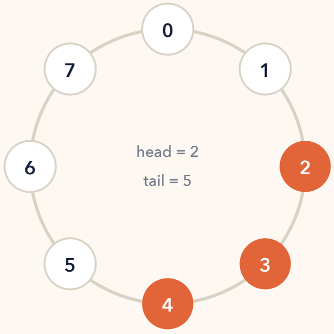
Deque (Double-Ended Queue, "డెక్" అని pronounce) = **రెండు చివర్లలోనూ** add/remove చేయగలిగే queue. అంటే stack + queue రెండిటి superset – front & back రెండిటి మీద push/pop.

Real-life Scenario

> **Queue = ticket counter దగ్గర వరుస (line/queue).** మొదట వచ్చిన వ్యక్తి **ముందు** నిలబడతాడు, మొదట ticket తీసుకుంటాడు (dequeue front). కొత్తగా వచ్చినవాళ్ళు **వెనక** చేరతారు (enqueue rear). **First come, first served** – pure FIFO. మధ్యలో దూరడం (line cutting) కుదరదు!
>
> **Deque = metro platform bench.** నువ్వు ఎడమ చివర నుండి కూడా కూర్చోవచ్చు, కుడి చివర నుండి కూడా; ఎడమ చివర నుండి లేవచ్చు, కుడి చివర నుండీ లేవచ్చు. **రెండు చివర్లూ open.** అందుకే deque తో stack (ఒకే చివర) లేదా queue (రెండు వేర్వేరు చివర్లు) రెండిటినీ simulate చేయవచ్చు.

QUEUE · CIRCULAR BUFFER ఉపాయం

CIRCULAR BUFFER – array ని ఉంగరంలా వాడటం



ఎందుకు circular

సాధారణ array queue lo – dequeue చేసినప్పుడు అన్నీ ఒక అడుగు జరపాలి → O(n).
Circular lo – head index ని ముందుకి జరిపితే చాలు → O(1).
index = (index + 1) % capacity – ఈ ఒక్క line మొత్తం ఉపాయం.

Deque (double-ended)

రెండు చివరల నుంచీ push/pop → O(1)
Sliding window maximum, LRU కి ఇదే కావాలి
JavaScript lo shift() O(n) – పెద్ద queue కి index pointer వాడాలి

Internal Working

```
QUEUE (FIFO):
  enqueue: వెనక చేరు →  [1][2][3]  → dequeue: ముందు నుండి తీయి
                         front    rear
  enqueue(1),(2),(3) then dequeue() → 1 (మొదట వచ్చింది మొదట పోతుంది)

DEQUE (రెండు చివర్లూ):
       ← addFront        addBack →
       ← removeFront     removeBack →
              [1][2][3][4]
```

Circular Queue – ఎందుకు?

Simple array queue లో **dequeue = `shift()` = O(n)** (అన్ని elements ముందుకి shift). దీన్ని పరిష్కరించడానికి **circular queue** (ring buffer): fixed array + రెండు pointers (`front`, `rear`) – array చివరికి చేరితే **మొదటికి wrap around** (modulo `%`). దీనితో enqueue/dequeue **O(1)** – shift అవసరం లేదు. OS scheduling, streaming buffers, bounded queues కి వాడతారు.

Operations + Big-O

Operation	Queue	Deque
enqueue / addBack	O(1)	O(1)
dequeue / removeFront	O(1)†	O(1)
addFront	–	O(1)
removeBack	–	O(1)
peek front/back	O(1)	O(1)
search	O(n)	O(n)

†proper implementation తో (circular array లేదా linked list). **JS array** `shift()` **O(n)** కాబట్టి పెద్ద data కి pointer-based queue వాడాలి (కింద చూడు).

ఎప్పుడు వాడాలి (recognition signals)

- **BFS** (level-by-level traversal, shortest path unweighted) – **queue గుండె.**
- **"process in order / first come first served"** – task scheduling, print queue, request handling.
- **level-order tree traversal.**
- **sliding window maximum/minimum** – **monotonic deque.**
- **producer-consumer, buffering, rate limiting.**
- Deque: front & back రెండిటి మీద ops కావాలంటే (palindrome check, undo+redo).

JS Implementation

```
// ---- ❌ Naive (చిన్న data కి ఓకే, పెద్దదానికి slow) ----
const q = [];
q.push(1);           // enqueue (back) – O(1)
q.push(2);
console.log(q.shift());  // dequeue (front) – ⚠️ O(n)! elements shift అవుతాయి
// చిన్న inputs కి ఓకే, కానీ n పెద్దదైతే O(n²) → TLE.

// ---- ✅ Efficient Queue – head pointer (shift లేదు, O(1) dequeue) ----
class Queue {
  constructor() {
    this.items = {};      // index → value (object as sparse store)
    this.head = 0;        // dequeue index
    this.tail = 0;        // enqueue index
  }
  enqueue(x) { this.items[this.tail++] = x; }        // back కి add – O(1)
  dequeue() {
    if (this.isEmpty()) return undefined;
    const val = this.items[this.head];
    delete this.items[this.head++];                  // front నుండి – O(1)
    return val;
  }
  front() { return this.items[this.head]; }
```

```
  isEmpty() { return this.head === this.tail; }
  size() { return this.tail - this.head; }
}

const queue = new Queue();
queue.enqueue(1); queue.enqueue(2); queue.enqueue(3);
console.log(queue.dequeue());  // 1  (FIFO — మొదట వచ్చింది)
console.log(queue.dequeue());  // 2
console.log(queue.front());    // 3
console.log(queue.size());     // 1
```

Deque — JS లో

```
// చిన్న inputs: JS array యొక్క push/pop (back) + unshift/shift (front)
const dq = [];
dq.push(1);       // addBack   → [1]
dq.unshift(0);    // addFront  → [0, 1]   (⚠️ O(n) array లో)
dq.pop();         // removeBack → [0]
dq.shift();       // removeFront → []
// నిజమైన O(1) deque కి doubly linked list లేదా circular buffer వాడాలి.
```

Use case: BFS (queue యొక్క killer app)

```
// Grid/graph లో BFS — shortest path (unweighted), level-order. Queue కీలకం.
function bfs(graph, start) {
  const visited = new Set([start]);
  const queue = [start];       // (చిన్న demo — పెద్దదానికి head-pointer Queue వాడు)
  const order = [];
  while (queue.length) {
    const node = queue.shift();       // front నుండి తీయి (FIFO)
    order.push(node);
    for (const nbr of graph[node]) {
      if (!visited.has(nbr)) {
        visited.add(nbr);
        queue.push(nbr);              // back కి చేర్చు
      }
    }
  }
  return order;
}
const graph = { A: ["B", "C"], B: ["D"], C: ["D", "E"], D: [], E: [] };
console.log(bfs(graph, "A"));  // ["A", "B", "C", "D", "E"] — level by level
// ⭐ BFS = queue; DFS = stack — ఇది గుర్తుంచుకో.
```

Stack vs Queue — comparison

అంశం	Stack	Queue
Principle	**LIFO** (last in first out)	**FIFO** (first in first out)
Add/remove	ఒకే చివర (top)	వేర్వేరు చివర్లు (back/front)
analogy	ప్లేట్ల గుట్ట	ticket వరుస

అంశం	Stack	Queue
Traversal	**DFS**	**BFS**
JS	`push` / `pop`	head-pointer queue

Gotchas (సాధారణ తప్పులు)

- **`arr.shift()` for dequeue on big data** → O(n) each → O(n^2) total → TLE. ✅ head-pointer queue లేదా linked list.
- **Stack/queue confusion** – pop from where? Stack = same end; queue = opposite end.
- **BFS లో `visited` mark timing** – node ని queue కి **enqueue చేసేటప్పుడే** visited mark చేయి (dequeue అప్పుడు కాదు), లేకపోతే duplicates queue లోకి.
- **Empty dequeue** → undefined; check `isEmpty` ముందు.

Key Points

- **Queue = FIFO** (front నుండి తీయి, back కి చేర్చు); **Stack = LIFO.**
- **Deque = రెండు చివర్లూ** – stack + queue superset.
- **JS array `shift()` O(n)** – big data కి **head-pointer queue** (O(1) dequeue) వాడు.
- **Circular queue** = fixed array + wrap-around (modulo) → O(1) ops, bounded.
- Signals: **BFS/level-order** → **queue**; **scheduling/FCFS** → **queue**; **sliding window max** → **monotonic deque.**
- ⭐ **BFS = queue, DFS = stack** – permanent గా గుర్తుంచుకో.

Interview దృష్టి

Q: JS లో efficient queue ఎలా, `shift()` O(n) కదా? A: Array `shift()` అన్ని elements ముందుకి shift చేస్తుంది → O(n). బదులుగా head/tail pointers ఉన్న object-based queue (dequeue = head++ → O(1)), లేదా linked list (head తీసేయడం O(1)) వాడతాను. BFS లో large graphs కి ఇది తప్పనిసరి.

Q: Queue ఎప్పుడు గుర్తుపట్టాలి? A: "shortest path (unweighted)", "level by level", "process in arrival order", "BFS" – queue. Sliding window max/min కి monotonic deque. తో "first in first out" ఏ context అయినా.

Q: Deque ఎందుకు అవసరం, stack + queue రెండూ ఉన్నాయి కదా? A: రెండు చివర్లలోనూ O(1) add/remove ఒకే structure లో కావాలంటే – ఉదా sliding window maximum (window కి కుడివైపు add, ఎడమ నుండి remove, back నుండి smaller elements pop). Deque ఒక్కటే ఈ మూడూ O(1) లో ఇస్తుంది.

---

16. Trees & Binary Trees & BST

వివరణ

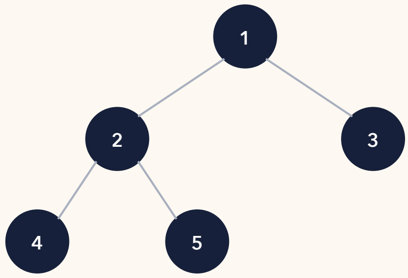
Tree = hierarchical (గొడుగు లాంటి) data structure – ఒక **root** నుండి మొదలై, ప్రతి node కి **children** ఉంటాయి, cycles ఉండవు. Linear structures (array, linked list) కి భిన్నంగా, tree **branching** – ఒక్కో node కి అనేక దారులు.

- **Binary Tree** – ప్రతి node కి **గరిష్టంగా 2 children** (left, right).
- **Binary Search Tree (BST)** – ఒక special binary tree: **ప్రతి node కి, ఎడమ subtree అంతా చిన్నది, కుడి subtree అంతా పెద్దది.** ఈ property వల్ల search/insert/delete O(log n) (balanced అయితే).

Trees ఎక్కడ? File systems (folders), DOM (HTML), org charts, database indexes (B-tree), decision trees, JSON. Interview లో binary tree traversals + BST = అత్యంత common.

TREE TRAVERSALS · PRE, IN, POST

ఒకే tree, మూడు traversal క్రమాలు



INORDER · ఎడమ → root → కుడి
4 · 2 · 5 · 1 · 3 ← BST lo ఇది SORTED

PREORDER · root → ఎడమ → కుడి
1 · 2 · 4 · 5 · 3 ← tree ని copy/serialize

POSTORDER · ఎడమ → కుడి → root
4 · 5 · 2 · 3 · 1 ← delete, size, height

ఏ పేరు దేన్ని సూచిస్తుంది

పేరు ఎప్పుడూ ROOT ని ఎప్పుడు సందర్శిస్తామో చెప్తుంది –
PRE = ముందు · IN = మధ్యలో · POST = తర్వాత. ఎడమ ఎప్పుడూ కుడి కంటే ముందు.
ఈ ఒక్క నియమం గుర్తుంటే మూడింటినీ ఎప్పటికీ మర్చిపోరు.

Real-life Scenario

> **Tree = కుటుంబ వృక్షం (family tree) / company org chart.** పైన CEO (**root**). CEO కింద VPs (**children**). ప్రతి VP కింద managers, వాళ్ళ కింద engineers (**leaves** – ఎవరూ కింద లేరు). ఎవరైనా ఒక్క boss (**parent**) కి report చేస్తారు (root తప్ప). ఇది **hierarchy** – linear కాదు, branching.
>
> **BST = బాగా organize చేసిన phone directory tree.** ఏ node దగ్గరైనా – నీ target ఆ node కంటే చిన్నదా? **ఎడమ వైపు** వెళ్ళు. పెద్దదా? **కుడి వైపు.** ప్రతి step లో **సగం directory** వదిలేస్తావు (binary search లాగే!) → O(log n). ఇదే BST యొక్క magic – sorted order + fast search రెండూ.

Terminology (తప్పకుండా తెలియాలి)

```
        1          ← ROOT (పైన node, parent లేదు)
       / \
      2   3        ← 2,3 = 1 యొక్క CHILDREN; 1 = వాళ్ళ PARENT
     / \   \
```

```
        4   5     6     ← LEAVES (children లేని nodes: 4,5,6)

- Root: పైన node (parent లేదు)
- Leaf: children లేని node (చివరి)
- Parent/Child: పైన/కింద directly connected nodes
- Sibling: అదే parent ఉన్న nodes (4, 5)
- Edge: రెండు nodes మధ్య connection (arrow)
- Depth (of node): root నుండి ఆ node కి edges సంఖ్య (root depth 0)
- Height (of node): ఆ node నుండి deepest leaf కి edges (leaf height 0)
- Height (of tree): root యొక్క height
- Subtree: ఏ node నైనా root గా తీసుకుంటే అది + దాని descendants
- Level: అదే depth ఉన్న nodes (level 0 = root)
```

Depth vs Height గుర్తుంచుకో: Depth పైనుంచి కిందికి (root=0), **Height కిందనుంచి పైకి** (leaf=0). చాలా మంది confuse అవుతారు.

Binary Tree types

రకం	వివరణ
Full	ప్రతి node కి 0 లేదా 2 children (1 కాదు)
Complete	చివరి level తప్ప అన్నీ నిండాయి; చివరిది ఎడమ నుండి fill (heaps ఇలాగే)
Perfect	అన్ని levels పూర్తిగా నిండాయి; అన్ని leaves ఒకే level
Balanced	ఏ node దగ్గరైనా left/right height తేడా ≤ 1 → O(log n) guaranteed
Skewed	ఒక వైపే grow (linked list లా) → O(n) worst

JS Node structure + traversals

```
class TreeNode {
  constructor(val, left = null, right = null) {
    this.val = val;
    this.left = left;
    this.right = right;
  }
}
//        1
//       / \
//      2   3       (ఈ tree మీద traversals చూద్దాం)
//     / \
//    4   5
const root = new TreeNode(1,
  new TreeNode(2, new TreeNode(4), new TreeNode(5)),
  new TreeNode(3));
```

DFS traversals (recursion) — root ని ఎప్పుడు visit చేస్తామో ఆధారంగా 3 రకాలు:

```
// PREORDER: root → left → right  (root మొదట)
function preorder(node, out = []) {
  if (!node) return out;
```

```
  out.push(node.val);          // ROOT మొదట
  preorder(node.left, out);    // తర్వాత left subtree
  preorder(node.right, out);   // తర్వాత right subtree
  return out;
}
console.log(preorder(root));   // [1, 2, 4, 5, 3]
// ఉపయోగం: tree copy/serialize (root ముందు కావాలి)

// INORDER: left → root → right  (root మధ్యలో)
function inorder(node, out = []) {
  if (!node) return out;
  inorder(node.left, out);     // left మొదట
  out.push(node.val);          // ROOT మధ్యలో
  inorder(node.right, out);    // right చివర
  return out;
}
console.log(inorder(root));    // [4, 2, 5, 1, 3]
// ⭐ BST మీద inorder = SORTED order! (అత్యంత ముఖ్యమైన fact)

// POSTORDER: left → right → root  (root చివర)
function postorder(node, out = []) {
  if (!node) return out;
  postorder(node.left, out);   // left మొదట
  postorder(node.right, out);  // right తర్వాత
  out.push(node.val);          // ROOT చివర
  return out;
}
console.log(postorder(root));  // [4, 5, 2, 3, 1]
// ఉపయోగం: tree delete (children ముందు), subtree results combine
```

గుర్తుంచుకో: "pre/in/post" = **root** ని (pre)ముందు / (in)మధ్య / (post)చివర visit చేయడం. Left ఎప్పుడూ right ముందు.

BFS traversal (level-order) – queue వాడి:

```
// LEVEL-ORDER: level by level (queue – BFS). "levels" గా group.
function levelOrder(root) {
  if (!root) return [];
  const result = [], queue = [root];
  while (queue.length) {
    const level = [], n = queue.length;   // ఈ level లో ఎన్ని nodes
    for (let i = 0; i < n; i++) {
      const node = queue.shift();          // front నుండి (FIFO)
      level.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);                    // ఒక level పూర్తి
  }
  return result;
}
console.log(levelOrder(root));  // [[1], [2, 3], [4, 5]]
```

Height (recursion) – tree problems యొక్క foundation

```
// Height = root నుండి deepest leaf కి edges. (empty tree = -1)
function height(node) {
  if (!node) return -1;                    // BASE: empty
  return 1 + Math.max(height(node.left), height(node.right));  // 1 + లోతైన child
}
console.log(height(root));  // 2
// చాలా tree problems ఈ "recurse both children, combine" నమూనా follow చేస్తాయి.
```

BST – property + operations

BST property: ప్రతి node కి – `left subtree అన్ని values < node.val < right subtree అన్ని values`. ఈ ఒక్క rule వల్ల search binary search అవుతుంది.

```
class BST {
  constructor() { this.root = null; }

  // INSERT — O(log n) balanced, O(n) worst (skewed)
  insert(val) { this.root = this._insert(this.root, val); return this; }
  _insert(node, val) {
    if (!node) return new TreeNode(val);   // ఖాళీ చోటు దొరికింది → కొత్త node
    if (val < node.val) node.left = this._insert(node.left, val);   // చిన్నది → ఎడమ
    else if (val > node.val) node.right = this._insert(node.right, val); // పెద్దది → కుడి
    // సమానం → ignore (duplicates handle policy మీద ఆధారం)
    return node;
  }

  // SEARCH — O(log n) balanced. ప్రతి step లో సగం tree వదిలేయి
  search(val) {
    let cur = this.root;
    while (cur) {
      if (val === cur.val) return true;
      cur = val < cur.val ? cur.left : cur.right;  // చిన్నదా ఎడమ, పెద్దదా కుడి
    }
    return false;
  }

  // MIN = అత్యంత ఎడమ node; MAX = అత్యంత కుడి node
  min() { let cur = this.root; while (cur && cur.left) cur = cur.left; return
cur?.val; }
}

const bst = new BST();
[5, 3, 7, 1, 4, 6, 8].forEach(v => bst.insert(v));
console.log(inorder(bst.root));  // [1, 3, 4, 5, 6, 7, 8]  ← SORTED! (BST inorder)
console.log(bst.search(4));      // true
console.log(bst.search(9));      // false
console.log(bst.min());          // 1
```

Operations + Big-O

Operation	BST (balanced)	BST (skewed/worst)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Min/Max	O(log n)	O(n)
Inorder (sorted)	O(n)	O(n)

Space: O(n) storage; traversal recursion O(h) stack (balanced O(log n), skewed O(n)).

⚠️ **Balance ముఖ్యం:** BST లో already-sorted data insert చేస్తే → skewed (linked list) → O(n)! దీన్ని పరిష్కరించేవి **self-balancing trees** (AVL, Red-Black) – auto-rotate చేసి height ని O(log n) లో ఉంచుతాయి. (Java `TreeMap`, C++ `std::map` వీటిపై నడుస్తాయి.)

ఎప్పుడు వాడాలి

- **Tree/BST వాడు:** hierarchical data (files, org, DOM); sorted data + fast search/insert/delete (BST/balanced); range queries ("k నుండి m వరకు values"); "kth smallest" (BST inorder).
- **వాడొద్దు:** simple lookup మాత్రమే కావాలి (→ hash map O(1) faster than BST O(log n)); order అవసరం లేదు.

Gotchas (సాధారణ తప్పులు)

- **Depth vs Height confusion** – depth root నుండి కింద, height leaf నుండి పైకి.
- **BST inorder ≠ sorted అనుకోవడం** – inorder **ఎప్పుడూ** sorted (BST valid అయితే). ఇది BST validate చేయడానికి trick.
- **Unbalanced BST** – sorted insert → O(n). Interview లో "balanced అని assume చేయవచ్చా?" అడుగు.
- **Traversal లో null check** – `node.left` before recursing; base case `if (!node) return`.
- **Recursion stack space** మర్చిపోవడం – skewed tree DFS O(n) stack → deep tree overflow.

Key Points

- **Tree = hierarchical, root + children, no cycles.** Binary tree = గరిష్ఠంగా 2 children.
- **Terminology:** root/leaf/parent/child; **depth (root=0, పైనుంచి), height (leaf=0, కిందనుంచి).**
- **Traversals:** DFS = pre(root first)/in(root mid)/post(root last); BFS = level-order (queue). **root ఎప్పుడు అనేదే తేడా.**
- ⭐ **BST inorder = sorted order** – అత్యంత ముఖ్యమైన fact.
- **BST property:** left < node < right → search/insert/delete **O(log n) balanced, O(n) skewed.**
- **Balance ముఖ్యం** – sorted insert → skewed; self-balancing (AVL/Red-Black) → O(log n) guaranteed.

Interview దృష్టి

Q: 3 DFS traversals తేడా ఎలా గుర్తుంచుకోవాలి? A: "pre/in/post" = **root** ని ఎప్పుడు – pre(మొదట), in(మధ్య), post(చివర). Left ఎప్పుడూ right ముందు. Preorder → serialize/copy; **inorder → BST sorted output**; postorder → delete/bottom-up combine.

Q: BST valid ఎలా check చేస్తావు? A: రెండు మార్గాలు – (1) inorder traversal strictly increasing అయితే valid. (2) ప్రతి node కి valid `(min, max)` range pass చేస్తూ recurse (ఎడమకి max update, కుడికి min update). రెండూ O(n).

Q: BST vs hash map – search రెండూ కావాలంటే ఏది? A: కేవలం lookup అయితే hash map (O(1) > O(log n)). కానీ **sorted order, range queries, kth smallest, min/max, floor/ceiling** కావాలంటే BST (hash map వీటిని ఇవ్వదు). Trade-off: hash map faster point lookup, BST ordered operations.

---

17. Heap / Priority Queue

వివరణ

Heap = ఒక special **complete binary tree** ఇక్కడ ప్రతి parent దాని children తో ఒక **order property** పాటిస్తుంది:

- **Min-heap** – ప్రతి parent ≤ దాని children. అంటే **root = అత్యంత చిన్న element** (minimum).
- **Max-heap** – ప్రతి parent ≥ దాని children. అంటే **root = అత్యంత పెద్ద element** (maximum).

Priority Queue = ఒక abstract concept – "highest priority element మొదట బయటికి." Heap దీన్ని implement చేసే అత్యంత సమర్థవంతమైన data structure. అందుకే "heap" మరియు "priority queue" తరచూ interchangeable గా వాడతారు.

కీలక superpower: min/max element ని **O(1) లో peek**, **O(log n) లో extract/insert.** అందుకే "top K", "kth largest", "median of stream", scheduling problems కి perfect.

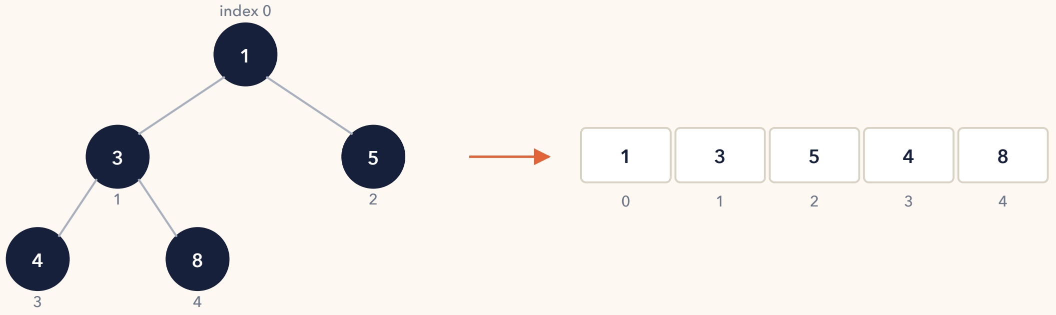
⚠️ **JS లో built-in heap లేదు** – అందుకే interview లో నువ్వు దీన్ని scratch నుండి రాయగలగాలి (కింద పూర్తి implementation).

Real-life Scenario

> **Heap = ఆసుపత్రి ER (emergency room) triage.** Patients వచ్చిన order లో కాదు – **తీవ్రత (priority)** ప్రకారం treat అవుతారు. Heart attack వచ్చినవాడు, చిన్న జ్వరం వాడి కంటే ముందు, అతను తర్వాత వచ్చినా. Nurse ఎప్పుడూ "ఇప్పుడు అత్యంత critical ఎవరు?" అని **O(1) లో** చెప్పగలదు (root). వాడిని treat చేశాక (extract), తర్వాతి అత్యంత critical automatic గా పైకి వస్తాడు (O(log n) reorganize).
>
> **మొత్తం queue ని sort చేయాల్సిన అవసరం లేదు** – కేవలం "top priority ఎవరు" తెలిస్తే చాలు. అదే heap యొక్క తెలివి: fully sorted కాదు, కానీ **min/max ఎప్పుడూ instant.** ఇదే min/max ని O(n log n) sort లేకుండా O(1)+O(log n) లో పొందడం.

HEAP · ARRAY REPRESENTATION యొక్క సొగసు

HEAP ని ARRAY గా – pointers అవసరం లేదు



ఈ మూడు సూత్రాలే మొత్తం heap

parent(i) = (i – 1) / 2 · left(i) = 2i + 1 · right(i) = 2i + 2
Pointers లేవు, nodes లేవు – ఒక సాధారణ array చాలు.
Complete binary tree కాబట్టి ఇది పని చేస్తుంది – ఖాళీలు ఉండవు.

వేగం

peek (min/max) O(1)
insert / extract O(log n)
build from array O(n) – sort కంటే వేగం

ఏం చేయలేదు

"k-వ చిన్నది" నేరుగా చెప్పలేదు
వెతకడం O(n) – sorted కాదు
Heap = పాక్షిక క్రమం మాత్రమే

Internal Working – array representation (elegant trick!)

Heap ఒక tree కానీ **array లో store అవుతుంది** – pointers అవసరం లేదు! Index math తో parent/child navigate చేస్తాం:

```
Min-heap tree:          Array representation:
       1                 index:  0  1  2  3  4  5  6
     /   \               value: [1, 3, 2, 8, 9, 5, 7]
    3     2
   / \   / \             ఏ node i కి:
  8   9 5   7              parent(i) = (i-1) / 2   (floor)
                           left(i)   = 2*i + 1
  (complete tree — ఖాళీలు లేవు)   right(i)  = 2*i + 2
```

ఎందుకు ఇది పనిచేస్తుంది? Heap ఎప్పుడూ **complete tree** (levels ఎడమ నుండి fill, ఖాళీలు లేవు), అందుకే array లో gaps లేకుండా pack అవుతుంది. Node i యొక్క children ఎప్పుడూ 2i+1, 2i+2 – pure arithmetic, pointers అవసరం లేదు. Cache-friendly కూడా.

రెండు core operations:

- **sift-up (bubble up):** insert చేసేటప్పుడు, కొత్త element ని చివర పెట్టి, parent కంటే చిన్నదైతే (min-heap) పైకి swap చేస్తూ, సరైన స్థానం వచ్చేదాకా. O(log n) (height).
- **sift-down (bubble down):** extract చేసేటప్పుడు, చివరి element ని root కి తెచ్చి, children కంటే పెద్దదైతే కిందికి swap చేస్తూ. O(log n).

Operations + Big-O

Operation	Big-O	గమనిక
peek (min/max)	**O(1)**	root = arr[0]

Operation	Big-O	గమనిక
push (insert)	O(log n)	sift-up (height)
pop (extract min/max)	O(log n)	sift-down
heapify (build from array)	**O(n)**	naive O(n log n) కాదు!
search arbitrary	O(n)	heap partial order – random search slow

Space: O(n). **heapify O(n) trap:** array నుండి heap build చేయడం, ఒక్కొక్కటి push (O(n log n)) కాదు – bottom-up sift-down తో **O(n)** (గణిత proof: కింది levels ఎక్కువ nodes కానీ తక్కువ sift depth).

JS Implementation – MinHeap (scratch నుండి)

```
class MinHeap {
  constructor() { this.h = []; }
  size() { return this.h.length; }
  peek() { return this.h[0]; }                    // min — O(1)

  _parent(i) { return (i - 1) >> 1; }             // (i-1)/2 floor
  _left(i)   { return 2 * i + 1; }
  _right(i)  { return 2 * i + 2; }
  _swap(i, j) { [this.h[i], this.h[j]] = [this.h[j], this.h[i]]; }

  // INSERT — చివర పెట్టి, sift-up — O(log n)
  push(val) {
    this.h.push(val);
    let i = this.h.length - 1;
    while (i > 0 && this.h[this._parent(i)] > this.h[i]) {  // parent పెద్దదా? swap
      this._swap(i, this._parent(i));
      i = this._parent(i);                        // పైకి కదులు
    }
  }

  // EXTRACT MIN — root తీసి, చివరి element ని root కి, sift-down — O(log n)
  pop() {
    if (this.h.length === 0) return undefined;
    const top = this.h[0];                        // min (return చేయబోయేది)
    const last = this.h.pop();                     // చివరి element
    if (this.h.length > 0) {
      this.h[0] = last;                            // root కి తెచ్చు
      this._siftDown(0);                           // సరైన స్థానానికి కిందికి
    }
    return top;
  }

  _siftDown(i) {
    const n = this.h.length;
    while (true) {
      let smallest = i;
      const l = this._left(i), r = this._right(i);
      if (l < n && this.h[l] < this.h[smallest]) smallest = l;   // ఎడమ చిన్నదా?
      if (r < n && this.h[r] < this.h[smallest]) smallest = r;   // కుడి చిన్నదా?
      if (smallest === i) break;                   // సరైన స్థానం
      this._swap(i, smallest);
```

```
      i = smallest;                                // కిందికి కదులు
    }
  }
}

const heap = new MinHeap();
[5, 3, 8, 1, 9, 2, 7].forEach(x => heap.push(x));
console.log(heap.peek());   // 1  (min — O(1))
const sorted = [];
while (heap.size()) sorted.push(heap.pop());
console.log(sorted);        // [1, 2, 3, 5, 7, 8, 9]  (extract order = sorted!)
```

Max-heap కావాలా? రెండు మార్గాలు: (1) comparison ని flip (`<` → `>`), లేదా (2) values ని negate చేసి min-heap వాడు (`push(-x)`, `-pop()`). Interview trick: **min-heap లో `-x` పెట్టడం** — max-heap లేనప్పుడు fast hack.

Use case: Top-K & Kth Largest (heap యొక్క killer app)

```
// K largest elements — size-k MIN-heap (O(n log k), sort O(n log n) కంటే better)
function topKLargest(nums, k) {
  const heap = new MinHeap();
  for (const x of nums) {
    heap.push(x);
    if (heap.size() > k) heap.pop();   // size k దాటితే smallest తీసేయి
  }                                     // → heap లో మిగిలేవి k largest
  const res = [];
  while (heap.size()) res.push(heap.pop());
  return res;
}
console.log(topKLargest([3, 1, 5, 12, 2, 11, 9], 3));  // [9, 11, 12]
```

ఎందుకు min-heap for K *largest*? Counter-intuitive కానీ brilliant: size-k min-heap లో **root = k largest లో అతి చిన్నది.** కొత్త element ఆ root కంటే పెద్దదైతే, root ని తీసేసి కొత్తది చేర్చు. చివరికి heap లో top-k మిగులుతుంది. Full sort O(n log n) కంటే O(n log k) better (k చిన్నదైతే).

Heapsort (bonus) — O(n log n) in-place

```
// అన్నిటినీ heap లో పెట్టి, ఒక్కొక్కటి extract → sorted (min-heap → ascending)
function heapSort(arr) {
  const heap = new MinHeap();
  for (const x of arr) heap.push(x);
  const out = [];
  while (heap.size()) out.push(heap.pop());
  return out;
}
console.log(heapSort([5, 2, 8, 1, 9, 3]));  // [1, 2, 3, 5, 8, 9]
```

ఎప్పుడు వాడాలి (recognition signals)

- **"top K / K largest / K smallest / K closest / K frequent"** → heap (size k).
- **"kth largest/smallest element."**

- **"median of a data stream"** → two heaps (max-heap + min-heap).
- **"merge K sorted lists/arrays"** → min-heap of heads.
- **scheduling / priority** – task priority, Dijkstra, Prim's MST.
- **"minimize/maximize repeatedly"** – ప్రతిసారి min/max తీసి process.
- **వాడొద్దు:** fully sorted output అవసరం (→ sort); arbitrary search (→ hash/BST).

Gotchas (సాధారణ తప్పులు)

- **JS లో built-in PQ లేదు** – scratch నుండి రాయాలి (పైన template బట్టీ కొట్టు).
- **Top-K largest కి min-heap** (max కాదు!), top-K smallest కి max-heap – direction confuse అవుతారు.
- **heapify O(n)** అని తెలియకపోవడం – array నుండి build O(n log n) కాదు, bottom-up O(n).
- **Arbitrary element search/delete O(n)** – heap point lookup కి కాదు (index map తో augment అవసరం).
- **Heap ≠ fully sorted** – extract order మాత్రమే sorted; internal array partial order.

Key Points

- **Heap = complete binary tree, array లో store** (parent (i-1)/2, children 2i+1/2i+2 – pointers లేవు).
- **Min-heap root = min, max-heap root = max. peek O(1), push/pop O(log n), heapify O(n).**
- **sift-up (insert), sift-down (extract)** = రెండు core ops, రెండూ O(log n).
- ⭐ **Top-K largest = size-k MIN-heap** (root = smallest of the k). O(n log k) > O(n log n).
- **JS built-in heap లేదు** – implementation బట్టీ కొట్టు. Max-heap hack: `-x` min-heap లో.
- Signals: **top-K, kth largest, median stream (two heaps), merge K sorted, scheduling/priority.**

Interview దృష్టి

Q: "Kth largest element" ఎలా, sort చేయకుండా? A: Size-k **min-heap** – array మీద iterate, ప్రతి element push, size k దాటితే pop (smallest తీసేయి). చివర్లో heap root = kth largest. O(n log k) – full sort O(n log n) కంటే better, ముఖ్యంగా k ≪ n.

Q: JS లో priority queue లేదు – ఏం చేస్తావు? A: MinHeap class scratch నుండి రాస్తా (push=sift-up, pop=sift-down, array-based). Interview కి ఇది తెలియడం తప్పనిసరి. Max-heap కావాలంటే comparison flip లేదా negate values.

Q: "Median of data stream" ఎలా? A: రెండు heaps – max-heap (చిన్న సగం) + min-heap (పెద్ద సగం), balanced. Median = heap tops (max-heap top, min-heap top, లేదా వాటి average). Insert O(log n), median O(1). ఇది two-heap pattern యొక్క classic.

Q: heapify ఎందుకు O(n), O(n log n) కాదా? A: Bottom-up sift-down లో, ఆకుల దగ్గర (ఎక్కువ nodes) sift depth 0, root దగ్గర (ఒక్క node) depth log n. మొత్తం work Σ (nodes × height) geometric series → O(n). ఒక్కొక్కటి push చేస్తే మాత్రమే O(n log n).

18. Graph

వివరణ

Graph = **nodes (vertices)** + వాటిని కలిపే **edges** యొక్క collection. Tree ఒక special graph (cycles లేని, connected, hierarchical) – కానీ general graph లో **cycles ఉండవచ్చు, ఏ node ఏ node తోనైనా connect అవ్వచ్చు.**

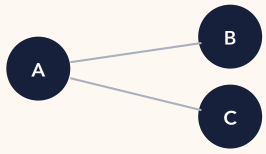
Graphs relationships/networks ని represent చేస్తాయి: social networks (friends), maps (cities+roads), web (pages+links), dependencies (tasks), circuits. **"ఏదైనా X లు Y relationships తో connect అయితే" = graph.**

రకాలు:

- **Directed** – edges కి దిశ (A→B కి, B→A కి కాదు). ఉదా: Twitter follow, task dependency.
- **Undirected** – edges రెండు దిశలా (A–B అంటే A↔B). ఉదా: Facebook friends, roads.
- **Weighted** – edges కి cost/weight (దూరం, ధర). ఉదా: flight prices.
- **Unweighted** – అన్ని edges సమానం.
- **Cyclic / Acyclic** – cycles ఉన్నాయా లేదా (DAG = Directed Acyclic Graph – scheduling కి కీలకం).

GRAPH · LIST మరియు MATRIX

ADJACENCY LIST vs MATRIX



Adjacency List

A → [B, C]
B → [A]
C → [A]
Space: O(V + E)

Adjacency Matrix

3×3 array, 0/1
edge check O(1)
Space: O(V²)

List ఎప్పుడు

Sparse graph (edges తక్కువ) – దాదాపు ఎప్పుడూ పొరుగులని తిరగడం వేగం
Interview lo default ఇదే

Matrix ఎప్పుడు

Dense graph, లేదా "A-B మధ్య edge ఉందా?" ఎక్కువ
$V = 10^5$ అయితే matrix = 10^{10} cells → అసాధ్యం

Directed vs Undirected: undirected lo ప్రతి edge ని **రెండు** దిక్కులా చేర్చాలి – ఇది తరచుగా మర్చిపోతారు

Real-life Scenario

> **Graph = రహదారుల నెట్‌వర్క్ (city map).** ప్రతి నగరం ఒక **node**, ప్రతి రహదారి ఒక **edge**. కొన్ని రహదారులు one-way (**directed**), కొన్ని two-way (**undirected**). ప్రతి రహదారికి దూరం/టోల్ (**weight**). నువ్వు "A నుండి B కి వెళ్ళే మార్గం ఉందా?" (connectivity), "అతి తక్కువ దూరం?" (shortest path), "అన్ని నగరాలు cover చేసే route?" (spanning tree) అని అడగవచ్చు – ఇవన్నీ graph problems.
>
> **Social network** కూడా perfect: ప్రతి వ్యక్తి node, స్నేహం edge. "ఇద్దరి మధ్య ఎన్ని degrees of separation?" = shortest path (BFS). "ఒకే గుంపులో ఎవరెవరు?" = connected components. Graph అంటే **connections యొక్క science.**

Representations – Adjacency List vs Matrix

రెండు ప్రధాన మార్గాలు graph ని store చేయడానికి:

```
Graph (undirected):     0 — 1
                        |   |
                        2 — 3 — 4

ADJACENCY LIST (nodes → neighbors list):   ADJACENCY MATRIX (n×n grid):
  0: [1, 2]                                    0  1  2  3  4
  1: [0, 3]                                  0 [0, 1, 1, 0, 0]
  2: [0, 3]                                  1 [1, 0, 0, 1, 0]
  3: [1, 2, 4]                               2 [1, 0, 0, 1, 0]
  4: [3]                                     3 [0, 1, 1, 0, 1]
                                             4 [0, 0, 0, 1, 0]
  (ప్రతి node కి దాని neighbors)            (matrix[i][j]=1 అంటే edge)
```

అంశం	Adjacency List	Adjacency Matrix
Space	**O(V + E)**	O(V²)
Edge exists? (u,v)	O(degree)	**O(1)**
ప్రతి neighbor iterate	**O(degree)**	O(V)
Sparse graph (తక్కువ edges)	✅ **efficient**	wasteful
Dense graph (చాలా edges)	ఓకే	fine
ఎప్పుడు	**default (చాలా problems)**	dense / fast edge-check అవసరం

DSA లో default = adjacency list (చాలా real graphs sparse – edges ≪ V²). JS లో object/Map (node → neighbors array) గా represent చేస్తాం.

JS Representation

```
// Adjacency list — object లేదా Map
const graph = {
  0: [1, 2],
  1: [0, 3],
  2: [0, 3],
  3: [1, 2, 4],
  4: [3],
};

// Edges list నుండి adjacency list build (undirected)
function buildGraph(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adj[u].push(v);
    adj[v].push(u);      // undirected → రెండు వైపులా (directed అయితే ఈ line తీసేయి)
  }
  return adj;
```

```
}
console.log(buildGraph(3, [[0, 1], [1, 2]]));  // [[1], [0, 2], [1]]
```

BFS template — shortest path (unweighted)

```
// BFS — level by level, QUEUE. Unweighted graph లో shortest path guarantee.
function bfs(graph, start) {
  const visited = new Set([start]);   // ⭐ enqueue అప్పుడే mark (dequeue కాదు)
  const queue = [start];
  const order = [];
  while (queue.length) {
    const node = queue.shift();        // front (FIFO)
    order.push(node);
    for (const nbr of graph[node]) {
      if (!visited.has(nbr)) {
        visited.add(nbr);              // duplicate enqueue నివారణ
        queue.push(nbr);               // back కి
      }
    }
  }
  return order;
}
console.log(bfs(graph, 0));   // [0, 1, 2, 3, 4]  (nearest first)
```

DFS templates — explore deep (recursive + iterative)

```
// DFS RECURSIVE — ఒక path చివరిదాకా, తర్వాత backtrack
function dfs(graph, start) {
  const visited = new Set(), order = [];
  function go(node) {
    visited.add(node);
    order.push(node);
    for (const nbr of graph[node]) {
      if (!visited.has(nbr)) go(nbr);   // recurse deeper
    }
  }
  go(start);
  return order;
}
console.log(dfs(graph, 0));   // [0, 1, 3, 2, 4]

// DFS ITERATIVE — explicit STACK (recursion depth ఎక్కువైతే)
function dfsIterative(graph, start) {
  const visited = new Set(), order = [], stack = [start];
  while (stack.length) {
    const node = stack.pop();          // top (LIFO)
    if (visited.has(node)) continue;
    visited.add(node);
    order.push(node);
    for (const nbr of graph[node]) {
      if (!visited.has(nbr)) stack.push(nbr);
    }
  }
  return order;
}
```

⭐ **BFS = queue (shortest path); DFS = stack/recursion (all paths, connectivity).** ఈ ఒక్క వాక్యం graph problems లో సగం. `visited` set **తప్పనిసరి** – లేకపోతే cycles లో infinite loop.

Use case: Number of Islands (grid = graph)

```
// 2D grid ఒక graph – ప్రతి cell node, పక్క cells (up/down/left/right) neighbors.
// '1' land, '0' water. ఎన్ని islands (connected land groups)?
function numIslands(grid) {
  const rows = grid.length, cols = grid[0].length;
  let count = 0;
  function sink(r, c) {                   // DFS – connected land అంతా '0' చేయి
    if (r < 0 || c < 0 || r >= rows || c >= cols || grid[r][c] !== "1") return;
    grid[r][c] = "0";                     // visited mark (mutate)
    sink(r + 1, c); sink(r - 1, c);       // 4 directions
    sink(r, c + 1); sink(r, c - 1);
  }
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < cols; c++)
      if (grid[r][c] === "1") {            // కొత్త island కనిపించింది
        count++;
        sink(r, c);                        // మొత్తం island ని "sink" చేయి
      }
  return count;
}
console.log(numIslands([
  ["1", "1", "0", "0"],
  ["1", "0", "0", "1"],
  ["0", "0", "1", "1"],
]));   // 2
```

Operations + Big-O (V = vertices, E = edges)

Operation	Adjacency List
BFS / DFS (full traversal)	O(V + E)
Add vertex	O(1)
Add edge	O(1)
Edge exists?	O(degree)
Space	O(V + E)

BFS/DFS = **O(V + E)** – ప్రతి vertex ఒకసారి, ప్రతి edge ఒకసారి (undirected లో రెండుసార్లు). ఇది గుర్తుంచుకో.

ఎప్పుడు వాడాలి (recognition signals)

- **"network / connections / relationships"** – friends, cities, dependencies.
- **"shortest path / minimum steps"** (unweighted) → **BFS.**
- **"connected components / groups / islands" → DFS/BFS/Union-Find.**

- **"cycle detection"** → DFS (colors) / Union-Find.
- **"course schedule / task ordering / dependencies"** → **topological sort** (DAG).
- **"shortest path with weights"** → **Dijkstra** (heap) / Bellman-Ford.
- **grid problems** (maze, islands, flood fill) – grid = implicit graph.

Gotchas (సాధారణ తప్పులు)

- `visited` **set మర్చిపోవడం** → cycles లో infinite loop. ✅ ఎప్పుడూ visited track.
- **BFS లో visited mark timing – enqueue అప్పుడే** mark (dequeue అప్పుడు కాదు), లేకపోతే node multiple times queue లోకి → duplicates/TLE.
- **Directed vs undirected** – undirected లో edge రెండు వైపులా add చేయాలి (`adj[u].push(v); adj[v].push(u)`).
- **Disconnected graph** – అన్ని components cover చేయాలంటే ప్రతి unvisited node నుండి traversal మొదలుపెట్టు (islands లా outer loop).
- **DFS recursion depth** – పెద్ద graph → stack overflow; iterative (explicit stack) వాడు.

Key Points

- **Graph = nodes + edges** (relationships). Directed/undirected, weighted/unweighted, cyclic/acyclic.
- **Adjacency list (O(V+E) space, default)** vs **matrix ($O(V^2)$, O(1) edge check)**. Sparse → list.
- ⭐ **BFS = queue (shortest path unweighted); DFS = stack/recursion (paths, connectivity).**
- `visited` **set తప్పనిసరి** (cycles → infinite loop); BFS లో enqueue అప్పుడే mark.
- **BFS/DFS = O(V + E).** Grid = implicit graph (cells = nodes, adjacent = edges).
- Signals: shortest path→BFS; components/islands→DFS; dependencies/ordering→topological sort; weighted shortest→Dijkstra.

Interview దృష్టి

Q: BFS vs DFS – ఎప్పుడు ఏది? A: **Shortest path / minimum steps (unweighted)** → BFS (level by level, nearest first guarantee). **All paths, connectivity, cycle detection, "does path exist"** → DFS (deep explore, తక్కువ code recursively). Grid islands రెండిటితో చేయవచ్చు. Weighted shortest path అయితే BFS కాదు – Dijkstra.

Q: Graph ని ఎలా represent చేస్తావు? A: సాధారణంగా **adjacency list** (Map/array of arrays) – O(V+E) space, real graphs sparse. Edge existence చాలా చెక్ చేయాలి లేదా dense graph అయితే adjacency matrix (O(1) check, $O(V^2)$ space).

Q: Grid problem ని graph గా ఎందుకు చూడాలి? A: ప్రతి cell ఒక node, adjacent cells (up/down/left/right, కొన్నిసార్లు diagonals) edges. అప్పుడు islands = connected components (DFS/BFS), maze shortest path = BFS, flood fill = DFS. Explicit adjacency list build అవసరం లేదు – directions array తో నేరుగా.

---

19. Trie (Prefix Tree)

వివరణ

Trie ("try" అని pronounce, retrieval నుండి) = **strings ని store చేసే special tree**, ఇక్కడ **ప్రతి node ఒక character** ని represent చేస్తుంది, మరియు root నుండి ఏ node కి **path = ఒక prefix.** అందుకే "prefix tree."

కీలక idea: **common prefixes ఉన్న words ఒకే path share చేస్తాయి.** "cat", "car", "card" – అన్నీ "ca" path share చేస్తాయి, తర్వాత branch అవుతాయి. దీనివల్ల **prefix-based operations చాలా fast** (O(word length), total words సంఖ్యతో సంబంధం లేదు).

Trie superpower: **"ఈ prefix తో మొదలయ్యే words ఉన్నాయా?"** అనేది O(L) లో (L = prefix length) – hash map ఇది చేయలేదు (hash map exact key మాత్రమే, prefix కాదు).

Real-life Scenario

> **Trie = phone లో T9 / autocomplete keyboard.** నువ్వు "c-a-r" type చేస్తుంటే, ప్రతి అక్షరం తర్వాత phone వెంటనే suggestions చూపిస్తుంది: "ca" → car, card, care, cat...; "car" → car, card, care. ఎలా ఇంత fast? ఎందుకంటే ఇది **అన్ని words ని prefix tree లో** organize చేసింది. నువ్వు type చేసిన prefix path ని follow చేసి, ఆ node కింద ఉన్న అన్ని words suggest చేస్తుంది.
>
> మరో analogy: **dictionary యొక్క thumb-index tabs.** పెద్ద dictionary లో side లో "A, B, C..." tabs, ప్రతి tab లోపల "Ca, Ce, Ch..." – నువ్వు "Car" వెతకాలంటే C → Ca → Car అని **అక్షరం అక్షరం navigate.** ప్రతి branching ఒక అక్షరం. ఇదే trie – words ని అక్షరాల paths గా organize చేయడం.

Internal Working – structure

```
"cat", "car", "card", "dog" ని insert చేస్తే:

              root
             /    \
           c        d
           |        |
           a        o
          / \       |
        t*   r      g*        (* = isEnd: ఇక్కడ ఒక పూర్తి word ముగుస్తుంది)
             |
             d*

- root నుండి "c→a→t" path = "cat" (t దగ్గర isEnd=true)
- "c→a→r" = "car"; "c→a→r→d" = "card"
- "ca" common prefix ఒకసారే store అయ్యింది (cat, car, card share)
- isEnd flag: "car" word కానీ "ca" కాదు అని distinguish చేయడానికి
```

ప్రతి node కి: (1) **children** map (character → next node), (2) **isEnd** boolean (ఇక్కడ word ముగిసిందా?).

JS Implementation

```
class TrieNode {
  constructor() {
    this.children = {};      // character → TrieNode
    this.isEnd = false;      // ఇక్కడ ఒక word ముగుస్తుందా?
  }
}

class Trie {
  constructor() { this.root = new TrieNode(); }

  // INSERT — word యొక్క ప్రతి char కి path build. O(L)
  insert(word) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) node.children[ch] = new TrieNode();  // లేకపోతే create
      node = node.children[ch];                                       // ముందుకి
    }
    node.isEnd = true;       // చివరి node = word end mark
  }

  // SEARCH — exact word ఉందా? O(L)
  search(word) {
    const node = this._traverse(word);
    return node !== null && node.isEnd;   // path ఉంది + word end అయితేనే
  }

  // STARTS WITH — ఈ prefix తో ఏదైనా word ఉందా? O(L)  ⭐ trie superpower
  startsWith(prefix) {
    return this._traverse(prefix) !== null;  // path ఉంటే చాలు (isEnd అవసరం లేదు)
  }

  // Helper — string ని follow చేసి చివరి node (లేదా null)
  _traverse(str) {
    let node = this.root;
    for (const ch of str) {
      if (!node.children[ch]) return null;   // path తెగింది → లేదు
      node = node.children[ch];
    }
    return node;
  }
}

const trie = new Trie();
["cat", "car", "card", "dog"].forEach(w => trie.insert(w));
console.log(trie.search("cat"));        // true
console.log(trie.search("ca"));         // false  (prefix, పూర్తి word కాదు)
console.log(trie.startsWith("ca"));     // true   (car/card/cat prefix)
console.log(trie.startsWith("ba"));     // false
console.log(trie.search("card"));       // true
console.log(trie.search("care"));       // false
```

search vs startsWith తేడా: `search` కి చివరి node `isEnd` కూడా true అవ్వాలి (పూర్తి word); `startsWith` కి కేవలం path ఉంటే చాలు (prefix). "ca" path ఉంది కానీ isEnd లేదు → search false, startsWith true.

Operations + Big-O

Operation	Big-O	గమనిక
Insert	O(L)	L = word length
Search (exact)	O(L)	total words తో సంబంధం లేదు!
StartsWith (prefix)	O(L)	hash map చేయలేని పని
Delete	O(L)	
Space	O(total chars × alphabet)	prefixes share అవుతాయి

కీలక insight: operations **word length మీద ఆధారం, total words సంఖ్య మీద కాదు.** 10 lakh words ఉన్నా, "cat" search O(3) – hash map లాగే fast, కానీ **prefix queries కూడా supports.**

Trie vs Hash Set (strings కి)

అంశం	Trie	Hash Set
Exact search	O(L)	O(L) (hashing)
Prefix search	**O(L)** ✅	O(n×L) – అన్నీ scan!
Autocomplete	✅ natural	❌ inefficient
Sorted order (words)	✅ (DFS)	❌
Space	prefixes share (కొన్నిసార్లు తక్కువ)	ప్రతి word పూర్తిగా
Simple exact lookup	overkill	✅ simpler

సారాంశం: కేవలం "word ఉందా?" → hash set (simpler). **Prefix queries, autocomplete, sorted traversal, "words with prefix"** → **trie** (hash set చేయలేదు).

ఎప్పుడు వాడాలి (recognition signals)

- **"autocomplete / type-ahead / suggestions"** → trie.
- **"prefix / starts with / common prefix"** → trie.
- **"dictionary / word search / valid word"** (repeated queries) → trie.
- **"word search in grid / board"** (Boggle) → trie + DFS.
- **"longest common prefix" of many strings** → trie.
- **IP routing, spell check** → trie.
- **వాడొద్దు:** exact lookup మాత్రమే (→ hash set); prefix relevant కాదు.

Gotchas (సాధారణ తప్పులు)

- **isEnd మర్చిపోవడం** – లేకపోతే "car" prefix ఉంటే "ca" ని కూడా word అనేస్తుంది. `search` కి isEnd తప్పనిసరి.
- **search vs startsWith confuse** – search=isEnd check; startsWith=path check.
- **Delete complexity** – నిజంగా node తీసేయడం tricky (ఇతర words ఆ path share చేస్తే తీయకూడదు); తరచూ isEnd=false చాలు.
- **Space** – sparse/long unique strings అయితే trie memory-heavy (ప్రతి char node). Alphabet పెద్దదైతే array బదులు map వాడు.

Key Points

- **Trie = prefix tree** – ప్రతి node ఒక char, root→node path = prefix; common prefixes share.
- Node = **children map + isEnd flag. Insert/search/startsWith అన్నీ O(L)** (L=length, total words కాదు).
- ⭐ **startsWith (prefix query) = trie superpower** – hash set చేయలేదు.
- **search = path + isEnd; startsWith = path మాత్రమే.**
- Signals: **autocomplete, prefix/starts-with, dictionary/word-search, word-search-in-grid (+DFS).**
- Exact lookup మాత్రమే → hash set (simpler); prefix operations → trie.

Interview దృష్టి

Q: Autocomplete ఎలా implement చేస్తావు? A: Trie – అన్ని words insert. User prefix type చేస్తే, ఆ prefix path కి traverse (O(L)), తర్వాత ఆ node కింద DFS చేసి అన్ని complete words (isEnd nodes) collect చేసి suggest. Hash set దీన్ని efficiently చేయలేదు (ప్రతి word check అవసరం).

Q: Trie vs hash map – string storage కి? A: Exact lookup మాత్రమే అయితే hash map/set (simpler, O(L)). కానీ **prefix search, autocomplete, "words starting with", sorted order** కావాలంటే trie – hash map ఇవి చేయలేదు. Trade-off: trie ఎక్కువ memory (nodes), కానీ prefix power.

Q: "Word Search II" (grid లో చాలా words వెతకడం) ఎందుకు trie? A: అన్ని target words ని trie లో పెట్టి, grid మీద DFS చేస్తూ trie ని parallel గా traverse చేస్తాం. ఒక path trie లో లేకపోతే వెంటనే prune (backtrack). ఇది ప్రతి word ని విడిగా వెతకడం (O(words × cells)) కంటే చాలా efficient – shared prefixes ఒకేసారి explore అవుతాయి.

---

20. Union-Find / Disjoint Set

వివరణ

Union-Find (Disjoint Set Union, DSU) = elements ని **వేర్వేరు groups (sets) గా** track చేసి, రెండు అత్యంత fast operations ఇచ్చే data structure:

- **find(x)** – x ఏ group కి చెందుతుంది? (దాని "representative/leader" return).

- **union(x, y)** – x, y గల రెండు groups ని ఒకటిగా కలపడం.

దీని killer feature: **"ఇద్దరు ఒకే group లో ఉన్నారా?" (connected?)** అనేది **దాదాపు O(1)** – connectivity, cycles, grouping problems కి perfect. Graph లో "connected components" ని dynamic గా (edges add అవుతూ ఉంటే) track చేయడానికి BFS/DFS కంటే elegant.

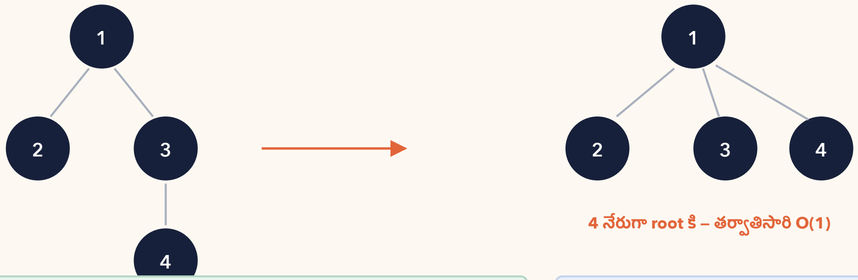
రెండు optimizations దీన్ని దాదాపు O(1) చేస్తాయి: **union by rank/size** + **path compression.** వీటితో amortized **O(α(n))** – α = inverse Ackermann function, ఇది ఆచరణలో ≤ 4 (అంటే practically constant).

Real-life Scenario

> **Union-Find = స్నేహితుల గుంపులు (friend circles) కలవడం.** ఊహించు – ఒక ఊళ్ళో మొదట్లో ప్రతి ఒక్కరూ ఒంటరి (ప్రతివారు తమ సొంత group). తర్వాత "రాము, సీత friends" (union) → ఇద్దరూ ఒకే group. "సీత, భీము friends" → ముగ్గురూ ఒకే group (రాము, సీత, భీము).
>
> ఇప్పుడు "రాము, భీము ఒకే గుంపులో ఉన్నారా?" అని అడిగితే – ప్రతి గుంపుకి ఒక **నాయకుడు (leader)** ఉంటాడు. రాము యొక్క leader ఎవరు? భీము యొక్క leader ఎవరు? **ఇద్దరికీ అదే leader అయితే → ఒకే గుంపు!** (find). కొత్త friendship వస్తే, రెండు గుంపుల leaders ని కలిపి ఒకటే leader చేస్తాం (union).
>
> **Path compression = shortcut.** ఒకసారి "రాము యొక్క leader ఎవరు?" కనుక్కున్నాక, రాము ని నేరుగా leader కి point చేస్తాం – తర్వాతిసారి అడిగితే instant. ఈ shortcut వల్లే దాదాపు O(1).

UNION-FIND · "వీళ్ళిద్దరూ ఒకే గుంపా?"

PATH COMPRESSION · find(4) చేసిన తర్వాత



Path compression

find() చేసినప్పుడు దారిలోని అన్ని nodes ని నేరుగా root కి అతికించడం

Union by rank

చిన్న చెట్టుని పెద్దదాని కింద కలపడం లేకపోతే చెట్టు ఒక పొడవాటి గొలుసు అవుతుంది

రెండూ కలిపితే ఒక్కో operation దాదాపు O(1) – కచ్చితంగా O(α(n)), α = inverse Ackermann (ఆచరణలో ≤ 4)

ఎప్పుడు వాడాలి: connected components · undirected graph lo cycle ఉందా · accounts merge · Kruskal MST. Graph ని traverse చేయకుండా connectivity ప్రశ్నలకి జవాబు కావాలంటే ఇదే సరైన సాధనం.

Internal Working – parent array + tree of leaders

```
ప్రతి element దాని "parent" ని point చేస్తుంది. Group leader = తనకు తానే parent.

మొదట్లో (ప్రతివారు తమ leader):
  index:  0  1  2  3  4  5
  parent:[0, 1, 2, 3, 4, 5]   (అందరూ separate – 6 groups)
```

```
union(0,1), union(1,2):        find(x) = leader వరకు parent follow:
  parent:[0, 0, 0, 3, 4, 5]      find(2) → parent[2]=0 → parent[0]=0 → leader 0
  (0,1,2 అన్నీ leader 0 కి)       find(2)==find(0) → connected!

PATH COMPRESSION: find(2) తర్వాత, 2 ని నేరుగా 0 కి point (tree flatten).
UNION BY RANK: పొట్టి tree ని పొడవు tree కింద attach (tree ఎత్తు తక్కువగా ఉంచడానికి).
```

రెండు optimizations ఎందుకు:

- **Union by rank/size** – ఎప్పుడూ చిన్న tree ని పెద్ద tree కింద కలుపు → tree ఎత్తు తక్కువ → find fast.
- **Path compression** – find చేసేటప్పుడు, దారిలోని అన్ని nodes ని నేరుగా leader కి point → tree దాదాపు flat → తర్వాతి finds instant.

రెండూ కలిస్తే → amortized O(α(n)) ≈ O(1).

JS Implementation

```
class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);  // ప్రతివారు తమ leader
    this.rank = new Array(n).fill(0);                        // tree ఎత్తు (approx)
    this.count = n;                                          // groups సంఖ్య
  }

  // FIND – x యొక్క leader (+ path compression) – O(α(n)) ≈ O(1)
  find(x) {
    if (this.parent[x] !== x) {
      this.parent[x] = this.find(this.parent[x]);   // ⭐ path compression: నేరుగా
leader కి
    }
    return this.parent[x];
  }

  // UNION – x, y groups కలుపు; already కలిసుంటే false (cycle signal) – O(α(n))
  union(x, y) {
    const rx = this.find(x), ry = this.find(y);
    if (rx === ry) return false;                    // ఇప్పటికే ఒకే group (cycle!)
    // UNION BY RANK: పొట్టి tree ని పొడవు tree కింద
    if (this.rank[rx] < this.rank[ry]) this.parent[rx] = ry;
    else if (this.rank[rx] > this.rank[ry]) this.parent[ry] = rx;
    else { this.parent[ry] = rx; this.rank[rx]++; } // సమానం → ఏదైనా, rank++
    this.count--;                                    // రెండు groups → ఒకటి
    return true;
  }

  // CONNECTED – ఒకే group లోనా? – O(α(n)) ≈ O(1)
  connected(x, y) { return this.find(x) === this.find(y); }
}

const uf = new UnionFind(6);        // 6 elements: 0..5
uf.union(0, 1);
uf.union(1, 2);                     // group {0,1,2}
uf.union(3, 4);                     // group {3,4}
console.log(uf.connected(0, 2));    // true   (ఒకే group)
console.log(uf.connected(0, 3));    // false  (వేర్వేరు)
```

```
console.log(uf.count);              // 3     (groups: {0,1,2}, {3,4}, {5})
uf.union(2, 4);                     // రెండు groups కలిసాయి
console.log(uf.connected(0, 3));    // true   (ఇప్పుడు ఒకే group)
console.log(uf.count);              // 2
```

Use case: Cycle Detection in Undirected Graph

```
// ప్రతి edge కి union try; రెండు ends ఇప్పటికే connected అయితే → cycle!
function hasCycle(n, edges) {
  const uf = new UnionFind(n);
  for (const [u, v] of edges) {
    if (!uf.union(u, v)) return true;   // union false = already connected = cycle
  }
  return false;
}
console.log(hasCycle(3, [[0, 1], [1, 2], [2, 0]]));  // true  (triangle → cycle)
console.log(hasCycle(3, [[0, 1], [1, 2]]));          // false (tree, no cycle)
```

Logic: edge (u,v) add చేసేముందు u, v ఇప్పటికే ఒకే group లో ఉంటే – ఈ edge ఒక cycle create చేస్తుంది. ఇది **Kruskal's MST** algorithm లో కూడా అదే idea (cycle create చేసే edges skip).

Operations + Big-O

Operation	Big-O	గమనిక
find	O(α(n)) ≈ O(1)	path compression తో
union	O(α(n)) ≈ O(1)	union by rank తో
connected	O(α(n)) ≈ O(1)	
count components	O(1)	counter maintain
Space	O(n)	parent + rank arrays

α(n) = inverse Ackermann – ఇది ఎంత slowly పెరుగుతుందంటే, విశ్వంలోని atoms సంఖ్యకి కూడా α ≤ 4. అందుకే **practically O(1).**

ఎప్పుడు వాడాలి (recognition signals)

- **"connected components" – dynamic** (edges add అవుతూ) → union-find (static అయితే DFS/BFS కూడా).
- **"cycle detection in undirected graph"** → union-find.
- **"number of provinces / friend circles / groups."**
- **"redundant connection" / "graph valid tree."**
- **"accounts merge / similar strings grouping."**
- **Kruskal's MST** (minimum spanning tree).
- **"are these two in the same set?"** repeated queries.

- **వాడొద్దు:** shortest path (→ BFS/Dijkstra); directed graph cycle (→ DFS colors); components ని విడదీయడం (union-find split support చేయదు).

Gotchas (సాధారణ తప్పులు)

- **Path compression / union by rank లేకపోవడం** → tree skewed → find O(n) → మొత్తం slow. ✅ రెండు optimizations వాడు.
- **Union by rank మర్చిపోయి random attach** → worst case O(n) tree. ✅ rank/size తో attach.
- **Directed graph cycle కి union-find** → పనిచేయదు (undirected మాత్రమే). Directed కి DFS 3-color.
- **find లో `parent[x] !== x` base** miss → infinite recursion.
- **Element removal / set split** – union-find support చేయదు (union మాత్రమే, un-union కాదు).

Key Points

- **Union-Find = groups track; find (leader?), union (merge), connected (same group?).**
- **దాదాపు O(1)** – **union by rank** (tree పొట్టిగా) + **path compression** (find లో flatten) కలిసి O(α(n)).
- **parent array** – leader = తనకు తానే parent; find = leader వరకు follow.
- Signals: **dynamic connected components, undirected cycle detection, grouping/provinces, Kruskal MST, "same set?"**.
- **Undirected మాత్రమే**; split/remove support చేయదు; directed cycle → DFS.

Interview దృష్టి

Q: Union-Find ఎప్పుడు, DFS/BFS ఎప్పుడు (connectivity కి)? A: Graph **static** (అన్ని edges ముందే తెలుసు) + ఒకసారి components కావాలంటే – DFS/BFS సరిపోతాయి. Edges **dynamically add** అవుతూ, మధ్యలో "connected?" queries వస్తూ ఉంటే – **union-find** (ప్రతి query దాదాపు O(1); DFS ప్రతిసారి O(V+E) waste). "Redundant connection", "accounts merge", Kruskal → union-find ideal.

Q: α(n) ఎందుకు practically O(1)? A: Inverse Ackermann function అత్యంత నెమ్మదిగా పెరుగుతుంది – n ఎంత పెద్దదైనా (విశ్వంలోని atoms కంటే ఎక్కువైనా) α(n) ≤ 4. కాబట్టి path compression + union by rank తో amortized cost effectively constant.

Q: Undirected graph లో cycle ఎలా detect చేస్తావు union-find తో? A: ప్రతి edge (u,v) కి union try. Union చేసేముందు u, v ఇప్పటికే connected (same leader) అయితే – ఈ edge cycle create చేస్తుంది → cycle ఉంది. లేకపోతే union చేసి కొనసాగు. O(E·α(n)).

Part 4 – Interview Craft

> Fundamentals + patterns + data structures అన్నీ నేర్చుకున్నాం. కానీ interview లో **అవి తెలియడం సగమే** – మిగతా సగం **వాటిని ఎలా present చేస్తావు** అనేది. ఈ చివరి Part లో communication, time management, ఏ problem కి ఏ pattern (decision guide), మరియు అన్నీ permanent గా గుర్తుండేలా revision strategy + memory tips నేర్చుకుంటాం. Senior/SSE level లో ఇది make-or-break.

---

21. Interview లో ఎలా communicate చేయాలి + Revision Strategy

వివరణ

SSE interview లో interviewer నీ **final code కంటే నీ ఆలోచనా విధానాన్ని** ఎక్కువ చూస్తాడు. Perfect solution silence లో రాయడం కంటే, **think-aloud గా, structured గా** ఒక good solution రాయడం చాలా better. ఎందుకంటే real job లో నువ్వు ఒంటరిగా code రాయవు – design discuss చేస్తావు, trade-offs explain చేస్తావు, team ని convince చేస్తావు. Interview ఆ skill ని test చేస్తుంది.

Golden rule: ఎప్పుడూ **నిశ్శబ్దంగా ఉండకు.** నీ తలలో ఏం జరుగుతోందో మాట్లాడు – wrong idea అయినా, "ఇది try చేద్దామనుకుంటున్నా కానీ ఇది $O(n^2)$ అవుతుంది, better ఉందేమో చూద్దాం" అనేది strong signal.

Real-life Scenario

> **Interview = navigator తో కలిసి driving test.** Examiner (interviewer) పక్కన కూర్చుని ఉన్నాడు. నువ్వు మౌనంగా, హఠాత్తుగా turn తీసుకుంటే – సరైన turn అయినా, examiner కి నీ decision-making కనపడదు, tense అవుతాడు. బదులుగా **"ఇక్కడ signal ఉంది, నేను mirror చూసి, left indicator వేసి, turn తీసుకుంటున్నా"** అని చెప్తూ డ్రైవ్ చేస్తే – examiner కి నీ **process, awareness, safety** అన్నీ కనిపిస్తాయి. చిన్న తప్పు చేసినా, "అయ్యో ఇక్కడ speed ఎక్కువైంది, తగ్గిస్తున్నా" అని self-correct చేస్తే, అది కూడా positive.
>
> **Interview అచ్చం అంతే.** Solution అనే destination కంటే, నువ్వు అక్కడికి ఎలా చేరుతావో (communication, structure, self-correction) అనేదే assess అవుతుంది.

Communication Flow – step by step (UMPIRE aloud)

1. Clarify (అడుగు, assume చేయకు) – 2-3 నిమిషాలు:

- "Input sorted గా ఉంటుందా? Negatives రావచ్చా? Duplicates?"
- "n range ఎంత?" (expected complexity hint – Topic 5).
- "Empty input, single element edge cases ఎలా handle చేయాలి?"

- "ఒకటే valid answer ఉంటుందా, multiple ఉంటే ఏం return చేయాలి?"
- "Output ఏం కావాలి – index? value? boolean?"

2. Examples (అర్థమైందని confirm):

- చిన్న example తో walk through: "input [2,7,11], target 9 అయితే output [0,1] కదా?"
- Edge case example కూడా: "empty అయితే []?"

3. Approach (code ముందు plan చెప్పు – permission తీసుకో):

- "నేను ముందు brute force ఆలోచిస్తా – nested loop, $O(n^2)$."
- "దాన్ని hash map తో O(n) కి తీసుకురావచ్చు అనుకుంటున్నా, ఎందుకంటే..."
- **"ఈ approach తో code మొదలుపెట్టనా?"** – interviewer buy-in తీసుకో.

4. Code (think-aloud గా రాయి):

- "ఇక్కడ hash map create చేస్తున్నా value→index కోసం..."
- Clean, readable – మంచి variable names (`left/right`, `seen`, `count`).
- Struggle అయితే మౌనంగా ఉండకు – "ఇక్కడ edge case ఆలోచిస్తున్నా."

5. Test (dry run – bugs నువ్వే పట్టుకో):

- ఒక example తో line-by-line trace: "i=0, x=2, need=7, map ఖాళీ..."
- Edge cases test: empty, single, duplicates, negatives.
- Bug దొరికితే: "అయ్యో ఇక్కడ off-by-one, fix చేస్తా" – self-correction positive!

6. Complexity + Optimize (evaluate):

- "Time O(n), space O(n)."
- "Space తగ్గించాలంటే sort చేసి two-pointer, కానీ అప్పుడు O(n log n) time – trade-off."
- Follow-up కి ready: "n చాలా పెద్దదైతే / streaming అయితే ఎలా?"

Time Management (45-min interview typical)

దశ	సమయం	ఏం చేయాలి
Clarify + examples	3-5 min	questions, edge cases, expected complexity
Approach + buy-in	5-8 min	brute→optimal, plan చెప్పు, permission
Code	15-20 min	clean, think-aloud
Test + debug	5-8 min	dry run, edge cases, self-correct
Complexity + follow-up	3-5 min	Big-O, trade-offs, optimizations

⚠️ **Stuck అయితే (5+ min):** మౌనంగా struggle చేయకు. "ఇక్కడ ఇలా ఆలోచిస్తున్నా, కానీ ఈ భాగం stuck..." అని చెప్పు – interviewer hint ఇస్తాడు (అది negative కాదు, collaboration). Brute force అయినా రాయి – working slow > broken fast.

ఏ pattern ఎప్పుడు – Decision Guide (interview cheat)

Problem signal	Pattern / DS	Complexity
Sorted array + pair/triplet	Two pointers	O(n)
Sorted + search / "minimize max"	Binary search (array/answer)	O(log n)
Contiguous subarray/substring + longest/max	Sliding window	O(n)
"seen? / duplicate? / frequency / complement"	Hash map / set	O(n)
"top K / kth largest / K closest"	Heap (size k)	O(n log k)
"median of stream"	Two heaps	O(log n) insert
Shortest path / min steps (unweighted)	BFS (queue)	O(V+E)
All paths / connectivity / islands	DFS (stack/recursion)	O(V+E)
Connected components (dynamic) / undirected cycle	Union-Find	O(α(n))
All permutations/combinations/subsets	Backtracking	$O(2^n)/O(n!)$
Min/max/count ways + overlapping subproblems	Dynamic Programming	varies
Matching brackets / next greater / undo	Stack	O(n)
Prefix / autocomplete / word search	Trie	O(L)
Hierarchical data / sorted + range	Tree / BST	O(log n)
Many range-sum queries	Prefix sum	O(1) query
Linked list cycle / middle	Fast & slow pointers	O(n)

Recognition drill: problem statement లో ఈ **keywords** ని pattern కి map చేయడం అలవాటు చేసుకో – ఇది interview లో instant recall ఇస్తుంది.

Revision Strategy – permanent memory కోసం

1. Spaced repetition: ఒక problem solve చేశాక – **1 day, 3 days, 1 week, 1 month** తర్వాత మళ్ళీ చూడు. ప్రతిసారి solution నీకు గుర్తుందా చూడు. మర్చిపోతే మళ్ళీ, గుర్తుంటే interval పెంచు.

2. Pattern-wise, random కాదు: ఒకే pattern (ఉదా sliding window) 8-10 problems వరుసగా చేయి. Random 500 కంటే pattern-wise 150 (Top-Interview-150) చాలా better – brain లో pattern permanent అవుతుంది.

3. Re-solve, re-read కాదు: solution చదవడం ≠ నేర్చుకోవడం. **Blank page నుండి మళ్ళీ రాయి.** చేయగలిగితేనే నిజంగా వచ్చినట్టు.

4. Error log: నువ్వు చేసే mistakes (off-by-one, sort comparator, visited timing) ఒక list లో రాసుకో. Interview ముందు ఈ list చదువు – అవే mistakes మళ్ళీ చేయవు.

5. Explain aloud (Feynman): ఒక problem ని ఎవరికైనా (లేదా గోడకి) **మాటల్లో explain** చేయి. Explain చేయలేకపోతే, అర్థం కాలేదని అర్థం. ఇది ఈ Telugu doc philosophy – analogy తో అర్థమైతే, మర్చిపోవు.

Memory Tips – గుర్తుంచుకోవడానికి mnemonics

- **UMPIRE** – Understand, Match, Plan, Implement, Review, Evaluate.
- ⭐ **BFS = queue, DFS = stack** – permanent.
- **BST inorder = sorted** – permanent.
- **Top-K largest = size-k MIN-heap** (min at root = threshold).
- **Two pointers = sorted; sliding window = contiguous; hash map = complement/frequency.**
- **Complexity ladder:** `O(1) < O(log n) < O(n) < O(n log n) < O(n²) < O(2ⁿ) < O(n!)`.
- `.sort((a,b)=>a-b)` – JS numbers కి comparator తప్పనిసరి.
- `push` / `pop` **O(1),** `shift` / `unshift` **O(n)** – JS array ends.
- **Constraint → complexity:** $n \le 20 \to O(2^n)$, $n \le 10^3 \to O(n^2)$, $n \le 10^5 \to O(n \log n)$, $n \le 10^7 \to O(n)$.

Gotchas (సాధారణ interview తప్పులు)

- **మౌనంగా code రాయడం** → interviewer కి process కనపడదు. ✅ think-aloud.
- **Clarify చేయకుండా assume** → తప్పు problem solve. ✅ ముందు questions.
- **నేరుగా optimal కి jump, fail** → ✅ brute force ముందు (working), తర్వాత optimize.
- **Complexity చెప్పకపోవడం** → ✅ ప్రతి solution కి Big-O + trade-off.
- **Edge cases test చేయకపోవడం** → ✅ empty/single/duplicate/negative dry run.
- **Stuck అయి panic** → ✅ మాట్లాడు, hint అడుగు (collaboration, weakness కాదు).

Key Points

- **Interviewer = process + communication > final code.** ఎప్పుడూ **think-aloud**, మౌనంగా ఉండకు.
- **Flow:** clarify → examples → approach (buy-in) → code → test (dry run) → complexity + optimize.
- **Time:** clarify 5min, approach 8min, code 20min, test 8min, complexity 5min. Stuck → మాట్లాడు.
- **Decision guide** (పైన table) – signal → pattern instant recall.
- **Revision:** spaced repetition + pattern-wise + **re-solve (re-read కాదు)** + error log + explain aloud.
- **Working slow > broken fast.** Self-correction (bug నువ్వే పట్టడం) = positive signal.

Interview దృష్టి

Q: సరైన approach తట్టకపోతే ఏం చెప్పాలి? A: మౌనంగా ఉండకు. "నేను brute force తో మొదలుపెడతా – ఇది $O(n^2)$, కానీ working. ఇప్పుడు ఎక్కడ repeated work ఉందో చూసి optimize చేస్తా..." అని loud గా reason చేయి. Interviewer నీ thinking చూసి hint ఇస్తాడు. Working brute force + optimization path = చాలా strong.

Q: Code లో bug interviewer పట్టుకుంటే? A: Defensive అవ్వకు. "మంచి catch, ఇక్కడ edge case miss అయ్యా, fix చేస్తా" – గ్రహించి, సరిదిద్దడం maturity. ఇంకా better – నువ్వే dry run చేసి ముందే పట్టుకో.

Q: DSA ఎంత practice చాలు SSE కి? A: Quantity కాదు, quality. **Top-Interview-150 ని pattern-wise, deeply** (ప్రతి pattern ఈ doc + companion docs తో అర్థం చేసుకుని) చేస్తే సరిపోతుంది. 500 problems shallow గా కంటే 150 deep గా + re-solve + patterns internalize. ఈ foundations doc + DSA_01-DSA_10 companion docs ఆ roadmap.

Q: ఈ Telugu docs ని ఎలా వాడాలి? A: ఈ **DSA_00 foundations** ని ముందు పూర్తిగా చదివి, ప్రతి data structure ని అర్థం చేసుకో (analogies తో). తర్వాత **DSA_01-DSA_10** companion docs తో ఆ patterns ని LeetCode-150 problems మీద drill చేయి. Foundation strong అయితే, ఏ problem కైనా సరైన tool ఎంచుకోగలవు – అదే SSE interview గెలుపు.

> **ముగింపు:** ఇది DSA foundations యొక్క పూర్తి map – mindset నుండి complexity, recursion, patterns, అన్ని core data structures, interview craft వరకు. దీన్ని ఒకసారి కాదు, **మూడు-నాలుగు సార్లు** చదువు; ప్రతిసారి కొత్తగా అర్థమవుతుంది. తర్వాత companion docs (`DSA_01_Arrays_Strings_Telugu.md` … `DSA_10_DynamicProgramming_Telugu.md`) తో LeetCode Top-Interview-150 ని pattern-by-pattern జయించు. **Foundation ఇక్కడ; practice అక్కడ. రెండూ కలిస్తే – SSE offer నీదే. ఒకసారి చదివితే జీవితంలో మర్చిపోకూడదు!**